TOPDATA

Manual de Integração SDK Inner Acesso Topdata

Guia Completo para Desenvolvedores

TOPDATA

Sumário

1 Introdução à SDK Inner Acesso.....6

1.1 O que é a SDK Inner Acesso?.....6

1.2 A Biblioteca EasyInner.dll: O Coração da Comunicação.....7

1.2.1 Não é uma API REST: Entendendo o Modelo de Comunicação.....8

1.3 Pré-requisitos para Desenvolvimento.....10

1.3.1 Conhecimento Técnico Essencial.....10

1.3.2 Ambiente de Desenvolvimento (Software).....11

1.3.3 Configuração do Projeto.....11

1.4 Instalação e Estrutura dos Recursos.....12

1.4.1 Processo de Instalação da SDK.....12

1.4.2 Acesso aos Exemplos e Documentação.....13

1.4.3 Abrindo e Executando os Exemplos.....13

2 Conceitos Fundamentais da Comunicação.....15

2.1 A Máquina de Estados: Gerenciando a Comunicação Síncrona Sequencial.....15

2.1.1 Por que uma Máquina de Estados? (Orquestrando Operações Síncronas).....15

2.1.2 Estados Essenciais e Transições (Gerenciamento Sequencial).....16

2.1.3 Fluxograma da Máquina de Estados.....20

2.2 Modos de Operação da Catraca: Online vs. Offline.....30

2.2.1 Modo Online: Controle Total pelo Software.....30

2.2.2 Modo Offline: Lógica Interna da Catraca.....31

2.3 Entendendo os Retornos e Códigos de Erro da DLL.....33

2.3.1 Retornos Gerais de Sucesso e Falha.....33

2.3.2 Retornos Específicos e Contextuais.....34

3 Configuração Inicial e Conectividade.....36

3.1 Configuração de Rede da Catraca (Hardware) 36
3.1.1 Endereçamento IP 36
3.1.2 IP do Servidor e Porta de Comunicação 36
3.1.3 Número do Inner 37
3.1.4 Como Configurar (Menu da Catraca ou WebServer) 37
3.2 Configuração de Rede do Servidor (Software) 39
3.2.1 Liberação de Portas no Firewall 39
3.3 Estabelecendo a Conexão via SDK Inner Acesso 40
3.3.1 EasyInner.DefinirTipoConexao() 40
3.3.2 EasyInner.AbrirPortaComunicacao() 40
3.3.3 Sugestão de Código para Conexão Inicial (C# - Exemplo Combinado): 42
4 Comandos Essenciais e Gerenciamento de Acesso 48
4.1 Enviando Configurações para a Catraca 48
4.1.1 O Processo de Montagem da Configuração 48
4.1.2 EasyInner.DefinirQuantidadeDigitosCartao() 52
4.1.3 EasyInner.ConfigurarTipoLeitor() 52
4.1.4 EasyInner.ConfigurarInnerOnline() / EasyInner.ConfigurarInnerOffLine() 53
4.1.5 EasyInner.ConfigurarAcionamento1() / EasyInner.ConfigurarAcionamento2() (Relés) 53
4.1.6 EasyInner.ConfigurarLeitor1() / EasyInner.ConfigurarLeitor2() 54
4.1.7 EasyInner.EnviarConfiguracoes(): Aplicando as Definições 55
4.1.8 Configuração de Mudança Automática Online/Offline 55
4.1.9 Tabela de Comandos para Montar a Configuração 58
4.2 Gerenciamento de Usuários (Modo Offline) 61
4.2.1 EasyInner.DefinirTipoListaAcesso() (Lista Branca/Negra) 61
4.2.2 EasyInner.InserirUsuarioListaAcesso() (Cartão, Horário) 61

4.2.3 EasyInner.EnviarListaAcesso() 62
4.2.4 EasyInner.ApagarListaAcesso() 63
4.3 Recebendo Eventos em Tempo Real (Modo Online) 65
4.3.1 EasyInner.ReceberDadosOnLine(): Capturando Acessos e Eventos 65
4.3.2 Tabela de Origens de Eventos 68
4.4 Coleta de Bilhetes Offline 71
4.4.1 EasyInner.ColetarBilhete(): Recuperando Registros Offline 72
4.5 Liberando o Acesso da Catraca 74
4.6 Outros Comandos Úteis 77
4.6.1 Comandos de Data/Hora 77
4.6.2 Comandos de Sinalização (Bips e LEDs) 78
4.6.3 Comandos de Mensagens no Display 78
4.6.4 Outros Comandos 79
5 Integrações Específicas e Modelos de Catraca 82
5.1 Catraca Linha 3 (Inner Acesso) vs. Catraca Linha 4 (Controle Catraca) 82
5.1.1 Diferenças de Hardware e Firmware 82
5.1.2 Opções de Leitura (Wiegand, Abatrack, QR Code) 85
5.1.3 EasyInner.InserirQuantidadeDigitovariable() 86
5.1.4 Solução para Limitação de Dígitos (Troca de Placa) 88
5.2 Catraca Expedidora de Cartões 88
5.2.1 Fluxo de Operação e Coleta de Bilhetes (EasyInner.ColetarBilhete()) 89
5.2.2 Tipos de Bilhetes da Expedidora 91
5.3 Catraca com Urna Coletora 93
5.3.1 Fluxo de Operação e Validação de Saída 93
5.4 Integração com Leitor Facial 99

6 Boas Práticas e Dicas de Desenvolvimento....101

6.1 Tratamento de Erros e Exceções....101

6.2 Uso de Thread(s) para a Comunicação (Serializando o Acesso à DLL)....101

6.3 Implementação de Logs Detalhados....103

6.4 Estabilidade da Rede e Conectividade....103

6.5 Testes com Kit Integrador e Equipamento Real....104

6.6 Compilação da Aplicação em x86....104

6.7 Gerenciando um Grande Número de Equipamentos....104

7 Troubleshooting: Diagnóstico e Solução de Problemas Comuns....106

7.1 Abordagem Geral para Solução de Problemas....106

7.2 Problemas Comuns e Soluções....106

7.2.1 Problemas de Conexão Inicial....106

7.2.2 Comandos da SDK não Funcionam....108

7.2.3 Leitores não Respondem ou Travam....109

7.2.4 Problemas Específicos da Expedidora....110

7.2.5 Problemas Específicos da Urna Coletora....111

7.3 Ferramentas e Técnicas de Debugging....114

7.4 Quando Contatar o Suporte Topdata....114

8 Suporte e Recursos Adicionais....116

8.1 Canais de Suporte Topdata....116

8.2 Como Solicitar Suporte (Cadastro de Integrador, Informações Necessárias)....117

8.3 Kit Integrador: Como Obter....118

8.4 Links Úteis....118

9 Histórico de Revisões....119

1 Introdução à SDK Inner Acesso

A integração de sistemas de controle de acesso é um pilar fundamental para a segurança e a eficiência operacional de diversos ambientes, desde academias e restaurantes até grandes indústrias e condomínios. A Topdata, reconhecendo essa necessidade, oferece a **SDK Inner Acesso** como a ferramenta principal para desenvolvedores e integradores que desejam criar soluções personalizadas e robustas.

1.1 O que é a SDK Inner Acesso?

A **SDK Inner Acesso** da Topdata é um kit de desenvolvimento de software projetado para facilitar a integração de sistemas externos com os equipamentos de controle de acesso da linha Topdata. Isso inclui uma vasta gama de dispositivos, como:

- **Catracas:** Modelos como Fit, Revolution e Box, que controlam o fluxo de pessoas em entradas e saídas.
- **Coletores de Dados:** Dispositivos que registram eventos e informações.
- **Dispositivos Específicos:** Como catracas expedidoras de cartões e catracas com urna coletora, que possuem funcionalidades especializadas para gerenciamento de comandas e visitantes.

No coração da SDK está a **EasyInner.dll**, uma biblioteca de vínculo dinâmico (DLL) para ambiente Windows. Esta DLL encapsula a complexidade da comunicação de baixo nível via TCP/IP, permitindo que sua aplicação envie comandos e receba eventos dos equipamentos de forma simplificada. Em vez de lidar diretamente com os detalhes do protocolo de comunicação, o desenvolvedor interage com funções de alto nível fornecidas pela DLL.

A SDK acompanha uma vasta documentação detalhada e exemplos de código em diversas linguagens de programação, como **C#**, **Java**, **Delphi** e **Visual Basic 6**. Esses recursos servem como um ponto de partida prático e didático para o desenvolvimento, demonstrando como implementar as funcionalidades essenciais.

É fundamental entender que a SDK Inner Acesso **não é uma API REST** ou um serviço web. A comunicação é direta entre o seu software (servidor) e o equipamento (catraca/coletor), via socket TCP/IP, gerenciada pela DLL. Isso significa que seu software assume o papel de um "servidor" que escuta as conexões dos equipamentos e processa seus eventos em tempo real.

Limitação de Equipamentos Simultâneos: É importante notar que a EasyInner.dll possui uma limitação prática e testada de gerenciar **aproximadamente 30 equipamentos simultaneamente** a partir de uma única instância da DLL (ou seja, em uma única thread de comunicação). Para gerenciar um número maior de catracas, pode ser necessário arquitetar sua solução utilizando múltiplas instâncias da sua aplicação de integração, cada uma responsável por um subconjunto de equipamentos.

Em resumo, a SDK Inner Acesso é a ferramenta essencial para desenvolvedores que buscam criar soluções personalizadas de controle de acesso, garantindo uma integração robusta e eficiente com os equipamentos Topdata, dentro das limitações de escalabilidade por instância da DLL.

1.2 A Biblioteca EasyInner.dll: O Coração da Comunicação

A EasyInner.dll é o componente central da **SDK Inner Acesso**, atuando como a ponte entre o seu software e os equipamentos de controle de acesso da Topdata. Ela é uma **biblioteca de vínculo dinâmico (DLL)** para o ambiente Windows, o que significa que contém um conjunto de funções pré-compiladas que sua aplicação pode chamar para interagir com o hardware.

O que ela faz?

A principal função da EasyInner.dll é abstrair a complexidade da comunicação de rede e do protocolo proprietário dos equipamentos Topdata. Em vez de você precisar entender como montar pacotes de dados em bytes, enviar checksums ou gerenciar timeouts de rede em baixo nível, a DLL oferece funções de alto nível que simplificam essas operações.

Por exemplo, para abrir uma porta de comunicação, você simplesmente chama uma função como EasyInner.AbrirPortaComunicacao(). Para enviar uma configuração, você usa EasyInner.EnviarConfiguracoes(). A DLL se encarrega de:

- **Empacotamento de Dados:** Converte os parâmetros das suas chamadas de função em pacotes de dados no formato que o equipamento entende.
- **Comunicação de Rede:** Gerencia a conexão TCP/IP com a catraca, enviando e recebendo os pacotes.
- **Tratamento de Respostas:** Interpreta as respostas do equipamento e as traduz em retornos compreensíveis para sua aplicação.
- **Gerenciamento de Erros:** Lida com erros de comunicação, como timeouts ou respostas inválidas, retornando códigos de status para sua aplicação.

Compatibilidade:

A EasyInner.dll é desenvolvida para o ambiente Windows. É importante notar que ela é uma **biblioteca de 32 bits (x86)**. Isso significa que, ao desenvolver sua aplicação, você deve configurá-la para compilar ou ser executada como um processo de 32 bits, mesmo que seu sistema operacional seja de 64 bits. A não observância dessa compatibilidade pode resultar em erros como "Can't load IA 32-bit .dll on a AMD 64-bit platform".

Dependências:

Para o correto funcionamento da DLL e dos exemplos, é necessário que o ambiente de execução possua o **.NET Framework 3.5 ou superior** instalado e configurado.

Em suma, a EasyInner.dll é a ferramenta que permite que seu software "fale" com as catracas Topdata de forma eficiente e confiável, sem que você precise se aprofundar nos detalhes técnicos da comunicação de hardware.

1.2.1 Não é uma API REST: Entendendo o Modelo de Comunicação

É crucial para o integrador compreender que a **SDK Inner Acesso** e a **EasyInner.dll** operam sob um modelo de comunicação diferente de uma **API REST** (Representational State Transfer) ou de serviços web modernos. Essa distinção é fundamental para o planejamento e desenvolvimento da sua solução.

O que é uma API REST (e por que a EasyInner.dll não é uma)?

Uma API REST é um conjunto de regras e padrões que permitem que diferentes sistemas se comuniquem pela internet, geralmente usando o protocolo HTTP (Hypertext Transfer Protocol). As interações são baseadas em requisições (como GET, POST, PUT, DELETE) para URLs específicas, e a troca de dados ocorre em formatos padronizados como JSON ou XML.

Características de uma API REST:

- **Baseada em HTTP:** Utiliza os métodos HTTP para as operações.
- **Stateless (Sem Estado):** Cada requisição do cliente para o servidor contém todas as informações necessárias para entender a requisição. O servidor não armazena o estado da sessão do cliente.
- **URLs (Endpoints):** Recursos são identificados por URLs únicas (endpoints).
- **Formato de Dados:** Geralmente JSON ou XML.

Por que a EasyInner.dll não se encaixa nesse modelo?

A EasyInner.dll não utiliza HTTP, URLs ou JSON/XML para sua comunicação. Em vez disso, ela se comunica diretamente com os equipamentos Topdata via **sockets TCP/IP**.

Modelo de Comunicação da EasyInner.dll:

1. **Comunicação Direta por Socket TCP/IP:** Sua aplicação, ao usar a EasyInner.dll, estabelece uma conexão TCP/IP direta com o endereço IP e a porta da catraca (geralmente porta 3570).
2. **Protocolo Proprietário:** A troca de informações ocorre através de um protocolo binário proprietário da Topdata. A EasyInner.dll é responsável por empacotar e desempacotar esses dados binários, traduzindo-os para as funções e retornos que sua aplicação utiliza.
3. **Software como "Servidor":** No contexto dessa comunicação, o seu software (que utiliza a EasyInner.dll) assume o papel de um "servidor" que fica escutando a porta de comunicação definida. As catracas são configuradas para "apontar" para o IP e porta desse seu software, iniciando a conexão e enviando eventos (como a leitura de um cartão).
4. **Stateful (Com Estado):** Embora a comunicação TCP/IP possa ser stateless em alguns contextos, a interação com a EasyInner.dll e a catraca muitas vezes envolve a manutenção de um estado de conexão e de fluxo de comandos (como veremos na Máquina de Estados), onde a sequência de operações é importante.

Implicações para o Desenvolvedor:

- **Desenvolvimento Local:** A integração é feita no nível da aplicação, que precisa ser executada em um ambiente Windows onde a EasyInner.dll esteja acessível. Não é uma integração "na nuvem" no sentido de chamar uma API remota.
- **Controle de Fluxo:** Você precisará gerenciar o ciclo de vida da conexão (abrir, fechar, reconectar) e o fluxo de comandos e eventos, muitas vezes utilizando o conceito de Máquina de Estados (abordado no Capítulo 2).
- **Sem Endpoints Web:** Não há URLs para chamar ou JSON para parsear diretamente da catraca. A interação é via chamadas de função da DLL. Se você precisar de uma API REST para seu próprio sistema (por exemplo, para que um aplicativo web se comunique com a catraca), você terá que desenvolver essa API *em cima* da EasyInner.dll. Ou seja,

sua API REST seria uma camada intermediária que traduz requisições web para chamadas da EasyInner.dll e vice-versa.

Em resumo, a EasyInner.dll oferece uma interface de programação de baixo nível para controle de hardware via rede, exigindo que o desenvolvedor construa a lógica de comunicação e gerenciamento de eventos diretamente em sua aplicação.

1.3 Pré-requisitos para Desenvolvimento

Para iniciar o desenvolvimento de soluções de integração com a **SDK Inner Acesso** e a EasyInner.dll, é fundamental que o ambiente de desenvolvimento e o desenvolvedor possuam os seguintes pré-requisitos:

1.3.1 Conhecimento Técnico Essencial

- **Lógica de Programação:** Sólidos conhecimentos em lógica de programação são indispensáveis para compreender e adaptar os exemplos da SDK, bem como para construir a máquina de estados da comunicação.
- **Linguagem de Programação:** Familiaridade com uma das linguagens suportadas pelos exemplos da SDK é crucial. As linguagens mais comuns são:
    - **C#** (preferencial, pois muitos exemplos e o próprio manual se baseiam nela)
    - **Java**
    - **Delphi**
    - **Visual Basic 6 (VB6)**
- **Programação Orientada a Objetos (POO):** Embora não seja estritamente obrigatório para todos os exemplos, o conhecimento de POO facilita a organização do código e a compreensão da estrutura da SDK.
- **Redes de Computadores (Básico):** Entendimento de conceitos básicos de rede, como:
    - **Endereçamento IP (IPv4):** Compreender o que são IPs, máscaras de sub-rede e gateways.
    - **Protocolo TCP/IP:** Saber que a comunicação ocorre via TCP/IP e o que isso implica (conexão, portas).
    - **Portas de Comunicação:** Entender o conceito de portas TCP e a importância de portas específicas (como a 3570, padrão da catraca).

- **Firewall:** Conhecer a função do firewall e como ele pode bloquear a comunicação.

- **Sistemas Operacionais Windows:** A EasyInner.dll é uma biblioteca para ambiente Windows. Portanto, o desenvolvimento e a execução da aplicação ocorrerão nesse sistema operacional.

1.3.2 Ambiente de Desenvolvimento (Software)

- **Sistema Operacional:**
  - **Windows:** Versões compatíveis (Windows 7, 8, 10, 11 ou Windows Server).
- **IDE (Ambiente de Desenvolvimento Integrado):**
  - **Visual Studio (para C# e VB.NET):** Recomenda-se o Visual Studio 2005, 2008 ou versões mais recentes (como Visual Studio Community, que é gratuito para uso individual e pequenas equipes).
  - **Delphi IDE (para Delphi):** Versões compatíveis com os exemplos fornecidos.
  - **Eclipse ou Netbeans (para Java):** Para desenvolvimento Java.
- **.NET Framework:**
  - **Microsoft .NET Framework 3.5 ou superior:** Essencial para o funcionamento da EasyInner.dll e dos exemplos em C# e VB.NET. Certifique-se de que a versão necessária esteja instalada no ambiente de desenvolvimento e no ambiente de execução da sua aplicação.
- **SDK Inner Acesso:**
  - O instalador da SDK Inner Acesso, que inclui a EasyInner.dll, a documentação e os exemplos de código. Este será o ponto de partida para o seu projeto.

1.3.3 Configuração do Projeto

- **Compilação em 32 bits (x86):** Como a EasyInner.dll é uma biblioteca de 32 bits, sua aplicação deve ser compilada e executada como um processo de 32 bits.
  - **No Visual Studio (C# / VB.NET):** Nas propriedades do projeto, na aba "Compilar" (ou "Build"), defina a "Plataforma de Destino" (ou "Platform Target") para **x86**. Evite "Any CPU" se a DLL for a única dependência de 32 bits, pois "Any CPU" tentará rodar em 64 bits em sistemas 64 bits, causando o erro de carregamento da DLL.

- **No Delphi:** Verifique as configurações de compilação para garantir que o executável seja gerado para a plataforma Win32.
- **No Java:** Certifique-se de que a JVM (Java Virtual Machine) utilizada seja de 32 bits, ou que a ponte JNI/JNA esteja configurada corretamente para carregar DLLs de 32 bits.

Atenção a esses pré-requisitos garantirá um ambiente de desenvolvimento estável e evitará problemas comuns de compatibilidade ao iniciar sua integração.

1.4 Instalação e Estrutura dos Recursos

A **SDK Inner Acesso** é distribuída através de um instalador que tem como principal função registrar as DLLs necessárias no sistema operacional Windows. Os exemplos de código e a documentação, por sua vez, são disponibilizados separadamente em nosso portal do integrador.

1.4.1 Processo de Instalação da SDK

1. **Download do Instalador:** Obtenha o instalador da SDK Inner Acesso diretamente do portal do integrador da Topdata. O link para download é:

Integradores Topdata® - FAQ e downloads para integração com produtos Topdata®

2. Procure pela seção "Integração com Catraca e Coletor de Acesso" ou similar.
3. **Execução do Instalador:** Execute o arquivo baixado. Siga as instruções na tela. Este instalador é responsável por:
    - Registrar a EasyInner.dll e outras DLLs de suporte no sistema operacional, tornando-as acessíveis para suas aplicações.
    - Geralmente, ele instala essas DLLs em diretórios padrão do sistema (como System32 ou SysWOW64) ou em um diretório específico da Topdata, garantindo que o ambiente de execução as encontre.
4. **Conclusão:** Ao final da instalação, as DLLs estarão prontas para serem utilizadas em seus projetos.

1.4.2 Acesso aos Exemplos e Documentação

Diferente de versões anteriores, os exemplos de código e a documentação completa da SDK Inner Acesso não são instalados junto com as DLLs. Eles estão disponíveis para download como pacotes separados em nosso portal do integrador.

1. **Localização no Portal:** Acesse o portal:

Integradores Topdata® - FAQ e downloads para integração com produtos Topdata®

2. Navegue até a seção da **SDK Inner Acesso**. Lá você encontrará links para download de:
    - **Pacotes de Exemplos:** Organizados por linguagem de programação (C#, Java, Delphi, Visual Basic 6).
    - **Documentação:** Manuais de utilização e outros documentos técnicos em formato PDF (disponíveis para download no portal, mas não mais como parte do instalador da SDK).
3. **Estrutura dos Pacotes de Exemplos (Após Download):**

Ao baixar e descompactar um pacote de exemplos (por exemplo, o de C#), você encontrará uma estrutura semelhante a:

```
Exemplos_SDK_Inner_Acesso_CSharp\ (ou similar)
├──ExemploSDK\
│   └──... (Arquivos do projeto C# principal)
└──BaseExemplos\
    └──SDK_Exemplos.mdb (Banco de dados Access para exemplos)
```

- **Projetos de Exemplo:** Cada subpasta contém um projeto completo em uma IDE específica, demonstrando a utilização da EasyInner.dll e a lógica da máquina de estados.
- **Base de Dados para Exemplos:** A pasta BaseExemplos\ pode conter um arquivo .mdb (Microsoft Access) utilizado pelos exemplos para simular um banco de dados de usuários e configurações.

1.4.3 Abrindo e Executando os Exemplos

Para explorar os exemplos e iniciar seu desenvolvimento:

1. **Baixe os Exemplos:** Faça o download do pacote de exemplos na linguagem de sua preferência a partir do portal do integrador.
2. **Descompacte:** Descompacte o arquivo ZIP em um diretório de sua escolha.
3. **Abra na IDE:** Navegue até a pasta do exemplo desejado (e.g., CSharp\ExemploSDK\) e abra o arquivo de projeto (e.g., .sln para Visual Studio, .dproj para Delphi) na sua IDE.
4. **Configure a Compilação (Crucial!):** Lembre-se de configurar a plataforma de destino do seu projeto para **x86** (32 bits), conforme detalhado na seção de Pré-requisitos (1.3.3). Isso é vital para que sua aplicação consiga carregar a EasyInner.dll corretamente.
5. **Compile e Execute:** Compile o projeto e execute-o. Isso permitirá que você veja a aplicação em funcionamento, interaja com ela e comece a entender a lógica por trás da comunicação com os equipamentos Topdata.

Estudar os exemplos é a melhor forma de compreender a aplicação prática dos conceitos da SDK e da máquina de estados. Eles são a base para o desenvolvimento da sua própria solução de integração.

2 Conceitos Fundamentais da Comunicação

Para desenvolver uma integração robusta e eficiente com os equipamentos Topdata utilizando a **SDK Inner Acesso**, é fundamental compreender os conceitos subjacentes à comunicação. Este capítulo explora a arquitetura baseada em máquina de estados, os modos de operação das catracas e a interpretação dos retornos da EasyInner.dll, considerando as características de sua implementação.

2.1 A Máquina de Estados: Gerenciando a Comunicação Síncrona Sequencial

É fundamental entender que a EasyInner.dll é uma biblioteca **bloqueante** e **não thread-safe**. Isso significa:

- **Bloqueante:** Quando você chama uma função da EasyInner.dll (como EasyInner.ReceberDadosOnLine(), EasyInner.EnviarConfiguracoes(), etc.), a execução da thread que fez a chamada é **pausada** até que a operação com a catraca finalize (sucesso, erro, timeout) ou um evento esperado ocorra.
- **Não Thread-Safe:** Múltiplas threads **não devem chamar funções da EasyInner.dll simultaneamente**. O acesso à DLL deve ser serializado, ou seja, apenas uma thread por vez deve interagir com a biblioteca.

Considerando essas características, a **Máquina de Estados Finita (Finite State Machine - FSM)** no contexto da SDK Inner Acesso não é utilizada para gerenciar eventos assíncronos em paralelo entre múltiplas threads da DLL. Em vez disso, ela é a arquitetura recomendada para gerenciar a **sequência de operações síncronas e a alternância entre o gerenciamento de múltiplas catracas em uma única thread dedicada à comunicação com a DLL**.

2.1.1 Por que uma Máquina de Estados? (Orquestrando Operações Síncronas)

Mesmo com a DLL sendo bloqueante, a Máquina de Estados ainda é crucial e oferece vantagens significativas:

- **Orquestração Síncrona:** Gerencia a sequência lógica de chamadas de função da DLL para cada catraca (conectar -> configurar -> operar -> coletar), garantindo que cada passo seja executado na ordem correta e apenas quando o estado anterior for concluído.
- **Alternância entre Equipamentos:** Em uma única thread que interage com a DLL, a FSM permite alternar o processamento entre as diferentes catracas conectadas. A thread

verifica o estado da Catraca 1, executa a operação síncrona para ela, depois passa para a Catraca 2, e assim por diante.

- **Tratamento de Timeouts e Falhas:** A FSM gerencia time-outs de comunicação (que ocorrerão devido à natureza bloqueante das chamadas) e direciona o fluxo para estados de retentativa ou reconexão quando uma operação síncrona falha.
- **Organização e Manutenibilidade:** Estrutura o código de comunicação de forma clara e modular, facilitando a compreensão do fluxo e a manutenção, mesmo com a lógica de alternância sequencial entre catracas.

2.1.2 Estados Essenciais e Transições (Gerenciamento Sequencial)

Em uma única thread dedicada à comunicação com a EasyInner.dll, a Máquina de Estados gerencia a vida de cada catraca individualmente, alternando entre elas no loop principal. Os estados e transições refletem as etapas síncronas da comunicação. O fluxo para o modo online, em particular, segue uma sequência específica de estados para configuração inicial e operação contínua, conforme detalhado nos blocos do fluxograma.

- **ESTADO_CONECTAR:**
  - **Propósito:** Tentar estabelecer a conexão TCP/IP com a catraca específica.
  - **Ação:** Chamar a função EasyInner.TestarConexaoInner(NumeroInner).
  - **Transição:**
    - Retorno 0 (Sucesso): Para ESTADO_ENVIAR_CFG_OFFLINE.
    - Retorno de Erro/Timeout: Para ESTADO_RECONECTAR.
  - *Nota: EasyInner.AbrirPortaComunicacao() é chamada uma única vez pela thread de comunicação no início, antes de entrar no loop principal da máquina de estados.*
- **ESTADO_RECONECTAR:**
  - **Propósito:** Gerenciar a lógica de retentativa para reconectar à catraca após uma falha de comunicação.
  - **Ação:** Implementa um tempo de espera e um número limitado de tentativas de conexão antes de retornar ao estado de conexão inicial.
  - **Transição:** Transita de volta para ESTADO_CONECTAR após o tempo de espera e/ou número de tentativas.

- **ESTADO_ENVIAR_CFG_OFFLINE:**
  - **Propósito:** Montar e enviar as configurações do modo offline para a catraca.
  - **Ação:** Montar o buffer de configuração offline e chamar EasyInner.EnviarConfiguracoes(NumeroInner).
  - **Transição:**
    - Retorno 0 (Sucesso): Para ESTADO_ENVIAR_CONFIGMUD_ONLINE_OFFLINE.
    - Retorno de Erro: Para ESTADO_RECONECTAR.
- **ESTADO_ENVIAR_CONFIGMUD_ONLINE_OFFLINE:**
  - **Propósito:** Montar e enviar as configurações de mudança automática entre modo online e offline.
  - **Ação:** Montar o buffer de configuração de mudança automática e chamar EasyInner.EnviarConfiguracoesMudancaAutomaticaOnLineOffLine(NumeroInner).
  - **Transição:**
    - Retorno 0 (Sucesso): Para ESTADO_ENVIAR_CFG_ONLINE.
    - Retorno de Erro: Para ESTADO_RECONECTAR.
- **ESTADO_ENVIAR_CFG_ONLINE:**
  - **Propósito:** Montar e enviar as configurações do modo online para a catraca.
  - **Ação:** Montar o buffer de configuração online e chamar EasyInner.EnviarConfiguracoes(NumeroInner).
  - **Transição:**
    - Retorno 0 (Sucesso): Para ESTADO_CONFIGURAR_ENTRADAS_ONLINE.
    - Retorno de Erro: Para ESTADO_RECONECTAR.
- **ESTADO_CONFIGURAR_ENTRADAS_ONLINE:**
  - **Propósito:** Configurar as formas de entrada de dados permitidas no modo online.
  - **Ação:** Chamar EasyInner.EnviarFormasEntradasOnLine(NumeroInner, ...).
  - **Transição:**
    - Retorno 0 (Sucesso): Para ESTADO_ENVIAR_MSG_PADRAO.

    - Retorno de Erro: Para ESTADO_RECONECTAR.
- **ESTADO_ENVIAR_MSG_PADRAO:**
    - **Propósito:** Enviar a mensagem padrão a ser exibida no display no modo online.
    - **Ação:** Chamar EasyInner.EnviarMensagemPadraoOnLine(NumeroInner, ...).
    - **Transição:**
        - Retorno 0 (Sucesso): Para ESTADO_POLLING.
        - Retorno de Erro: Para ESTADO_RECONECTAR.
- **ESTADO_POLLING (Modo Online):**
    - **Propósito:** Aguardar a ocorrência de um evento em tempo real na catraca e enviar PingOnline periodicamente.
    - **Ação:** Chamar a função EasyInner.ReceberDadosOnLine(NumeroInner, ...). Se não houver evento imediato, a thread aguarda. Periodicamente, dentro deste estado, também deve ser enviado o comando EasyInner.PingOnline(NumeroInner) para manter a catraca em modo online (se configurado).
    - **Transição:**
        - Retorno 0 (Evento Recebido): Para ESTADO_VALIDAR_ACESSO.
        - Retorno de "Sem Eventos" ou Timeout na Espera: Permanece no estado ESTADO_POLLING.
        - Retorno de Erro na Comunicação: Para ESTADO_RECONECTAR.
- **ESTADO_VALIDAR_ACESSO:**
    - **Propósito:** Processar os dados de um evento recebido (do ReceberDadosOnLine) e decidir se o acesso é permitido ou negado.
    - **Ação:** Lógica da sua aplicação (não envolve chamadas da DLL para validação).
    - **Transição:**
        - Acesso Permitido: Para ESTADO_LIBERAR_CATRACA.
        - Acesso Negado: Para ESTADO_ENVIAR_MSG_ACESSO_NEGADO.
- **ESTADO_ENVIAR_MSG_ACESSO_NEGADO:**

- **Propósito:** Enviar uma mensagem indicando acesso negado no display e aguardar um tempo.
- **Ação:** Chamar EasyInner.EnviarMensagemTemporariaOnLine(NumeroInner, ...) com uma mensagem de acesso negado. Após o envio, a máquina de estados deve aguardar um tempo (definido na lógica do estado) antes de transitar.
- **Transição:**
  - Após tempo de exibição da mensagem: Para ESTADO_CONFIGURAR_ENTRADAS_ONLINE.

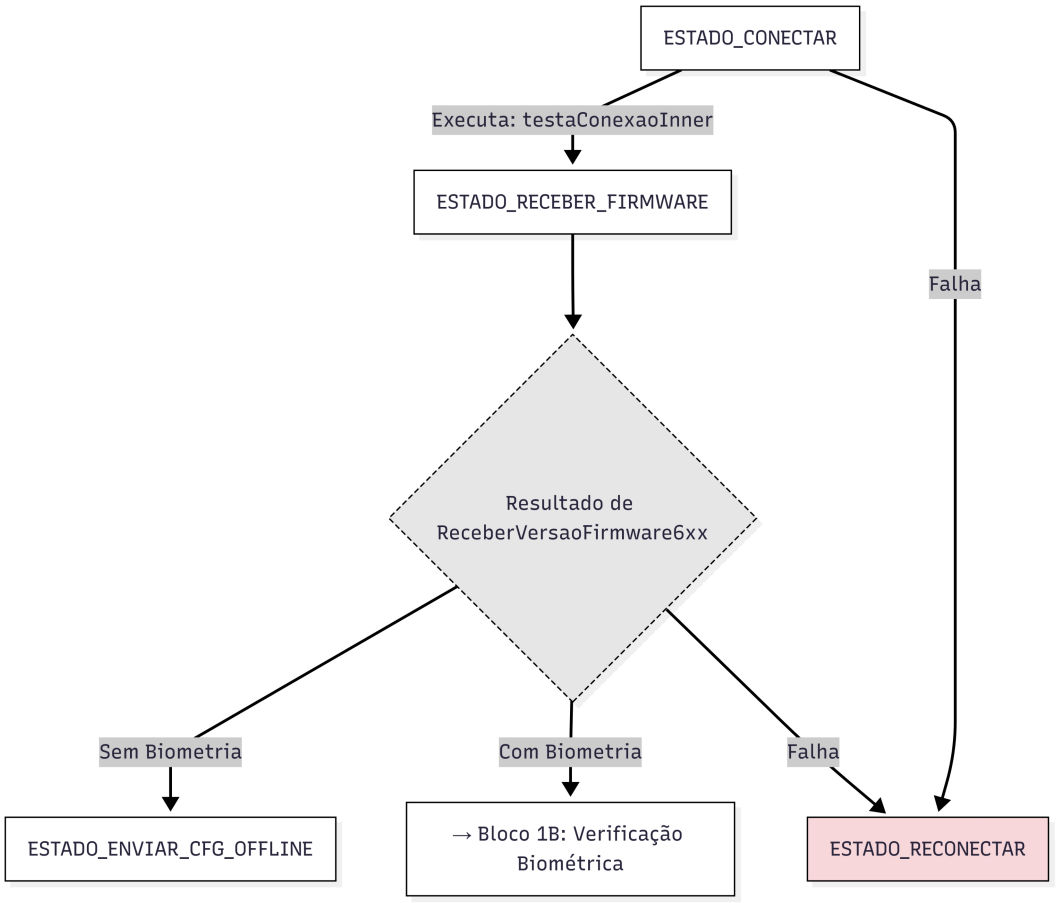
- **ESTADO_LIBERAR_CATRACA:**
  - **Propósito:** Enviar o comando de liberação de giro para a catraca.
  - **Ação:** Chamar uma das funções bloqueantes EasyInner.LiberarCatraca... (NumeroInner).
  - **Transição:**
    - Retorno 0 (Sucesso): Para ESTADO_MONITORA_GIRO_CATRACA.
    - Retorno de Erro: Para ESTADO_RECONECTAR.
- **ESTADO_MONITORA_GIRO_CATRACA:**
  - **Propósito:** Aguardar e confirmar o evento de giro físico após a liberação e enviar PingOnline periodicamente.
  - **Ação:** Chamar a função EasyInner.ReceberDadosOnLine(NumeroInner, ...) esperando especificamente pela Origem 6 (Giro Confirmado) ou Origem 5 (Fim Tempo Acionamento - não girou). Periodicamente, dentro deste estado, também deve ser enviado o comando EasyInner.PingOnline(NumeroInner).
  - **Transição:**
    - Retorno 0 (Evento Recebido - Origem 6 ou 5): Para ESTADO_CONFIGURAR_ENTRADAS_ONLINE.
    - Retorno de "Sem Eventos" ou Timeout na Espera: Permanece no estado ESTADO_MONITORA_GIRO_CATRACA.
    - Retorno de Erro na Comunicação: Para ESTADO_RECONECTAR.
- **ESTADO_COLETAR_BILHETES (Modo Offline ou Expedidora):**

- **Propósito:** Coletar um bilhete armazenado na catraca.
- **Ação:** Chamar a função EasyInner.ColetarBilhete(NumeroInner, ...).
- **Transição:**
    - Retorno 0 (Bilhete Coletado): Processar o bilhete e *permanecer* no estado ESTADO_COLETAR_BILHETES.
    - Retorno de "Sem Bilhetes": Transitar para o próximo estado na sequência (ex: ESTADO_POLLING se a catraca for para online, ou um estado de espera).
    - Retorno de Erro: Para ESTADO_RECONECTAR.

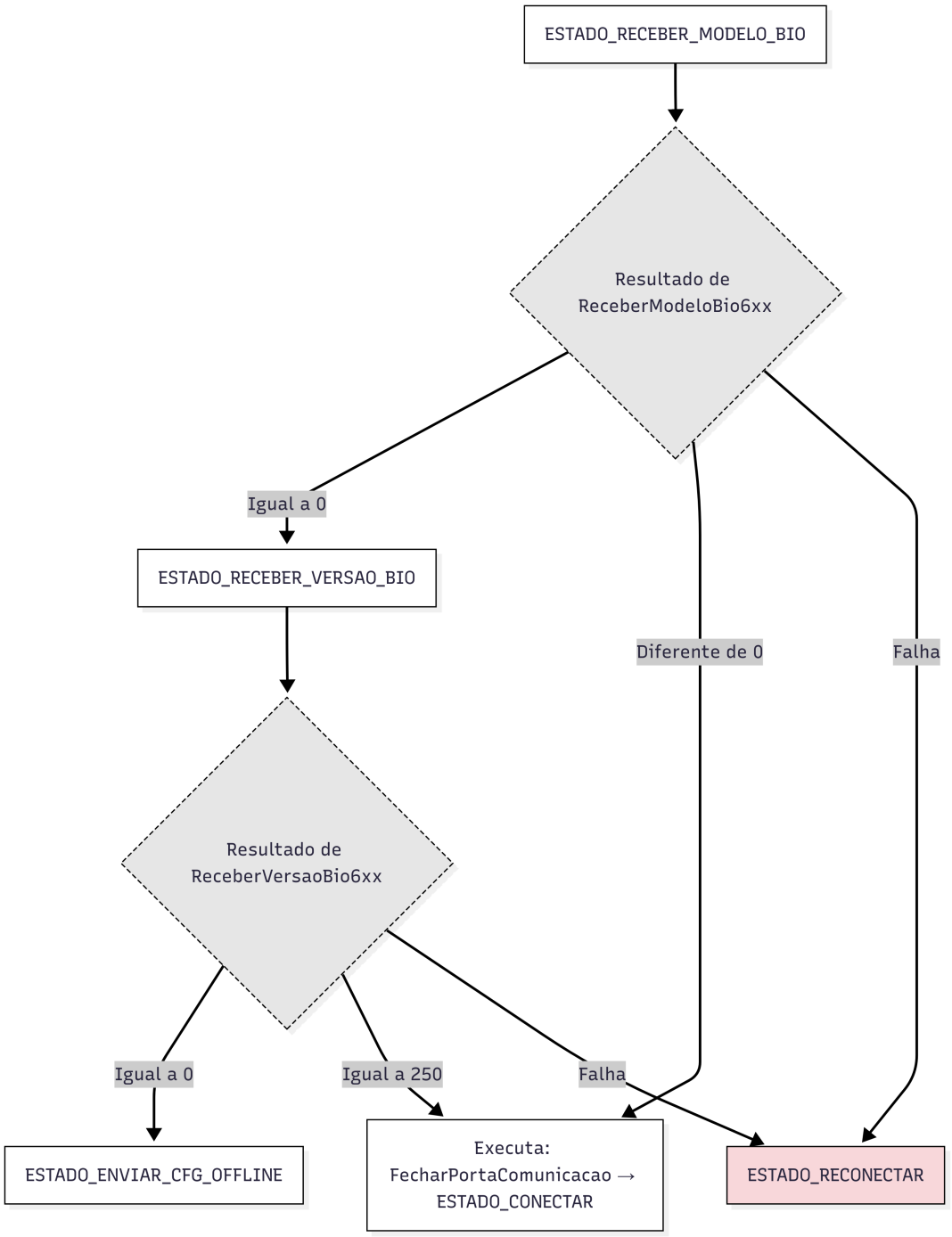
2.1.3 Fluxograma da Máquina de Estados

1) **Conexão Inicial e Leitura de Firmware:**

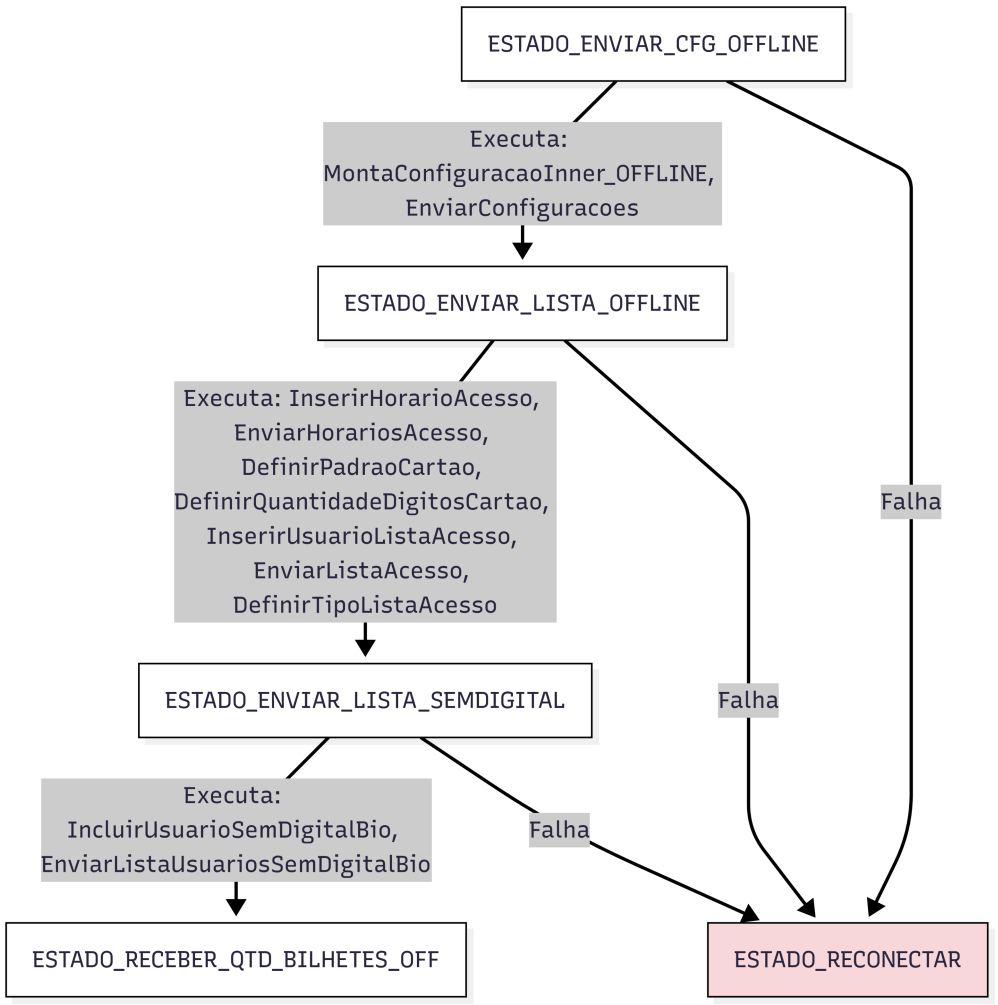
a) **Conexão Inicial:** O sistema inicia tentando conexão com o equipamento. Executa a leitura da versão de firmware. Dependendo do retorno, o fluxo segue diretamente para configuração ou para verificação biométrica.



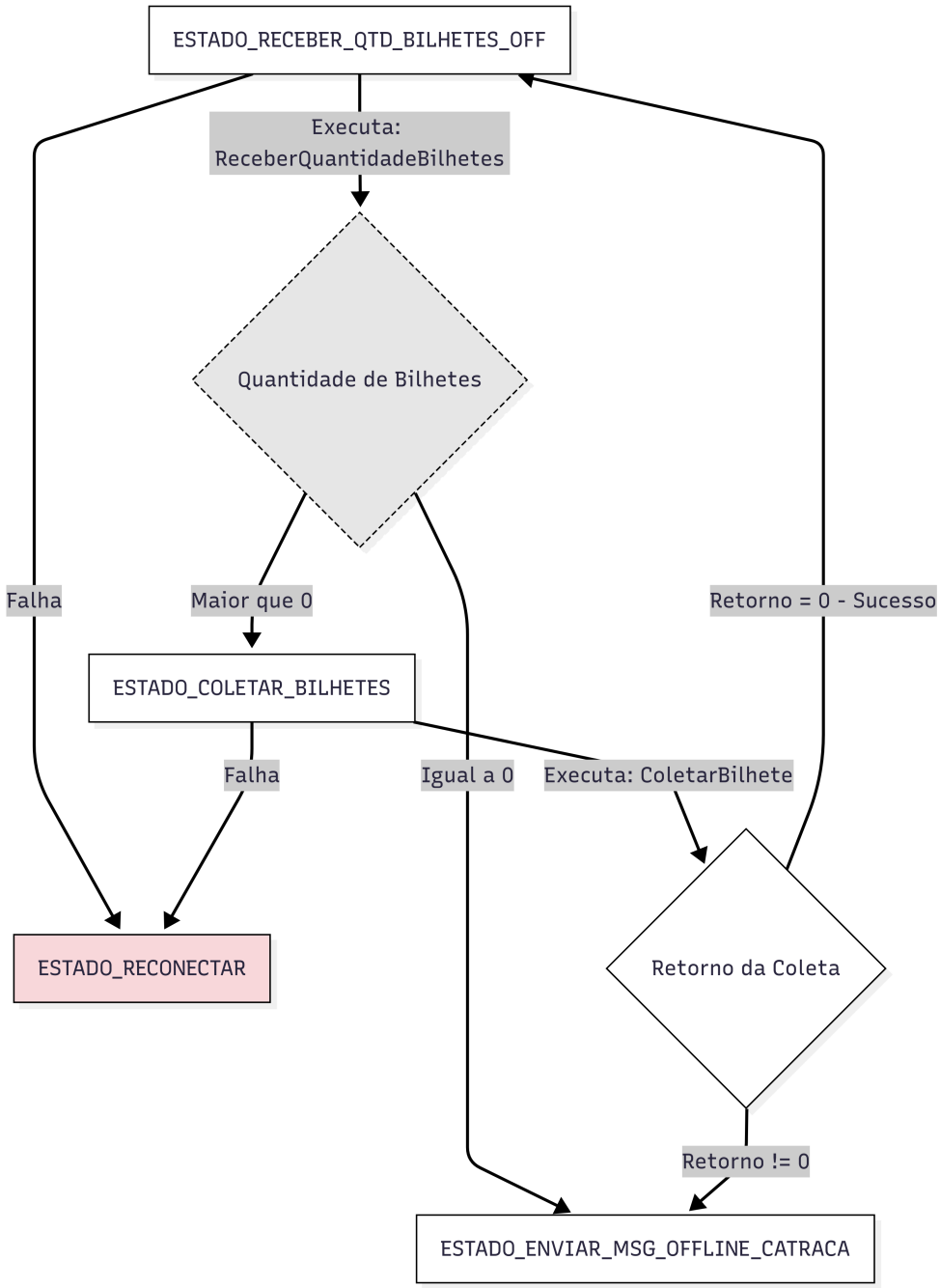
b) **Verificação de Biometria com Modelo e Versão:** Após detectar que o equipamento é biométrico, o sistema verifica o modelo e a versão do módulo biométrico. Dependendo do resultado, continua com a configuração offline ou reinicia a conexão. Todos os caminhos de falha são tratados.


ESTADO_RECEBER_MODELO_BIO
Resultado de ReceberModeloBio6xx
Igual a 0
Diferente de 0
Falha
ESTADO_RECEBER_VERSAO_BIO
Resultado de ReceberVersaoBio6xx
Igual a 0
Igual a 250
Falha
ESTADO_ENVIAR_CFG_OFFLINE
Executa: FecharPortaComunicacao → ESTADO_CONECTAR
ESTADO_RECONECTAR

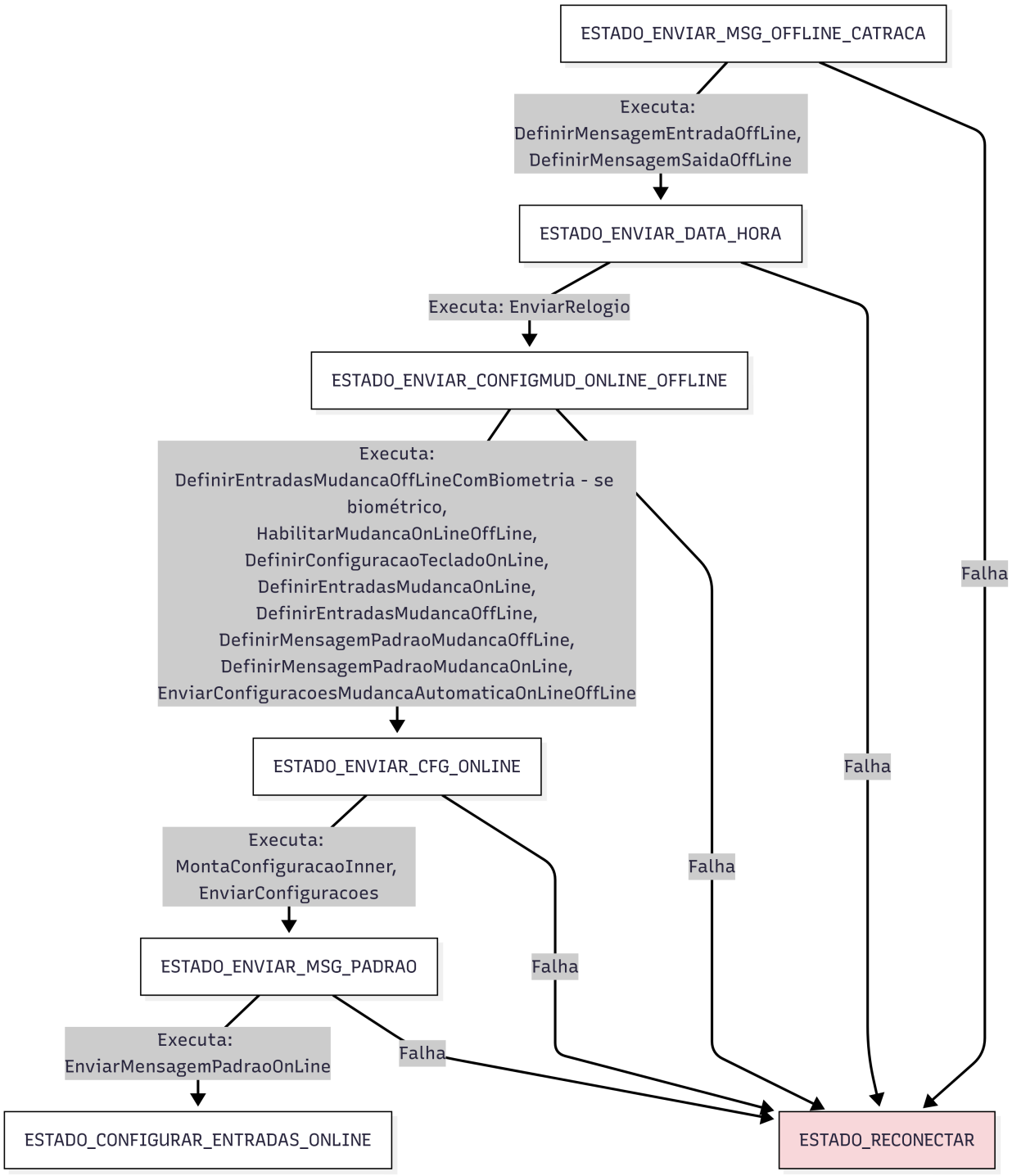
2) **Configuração do Equipamento para Modo Offline:** O sistema define todos os parâmetros necessários para operação offline: horários de acesso, listas de cartões e usuários sem biometria. Em caso de falhas em qualquer etapa, retorna ao estado de reconexão.



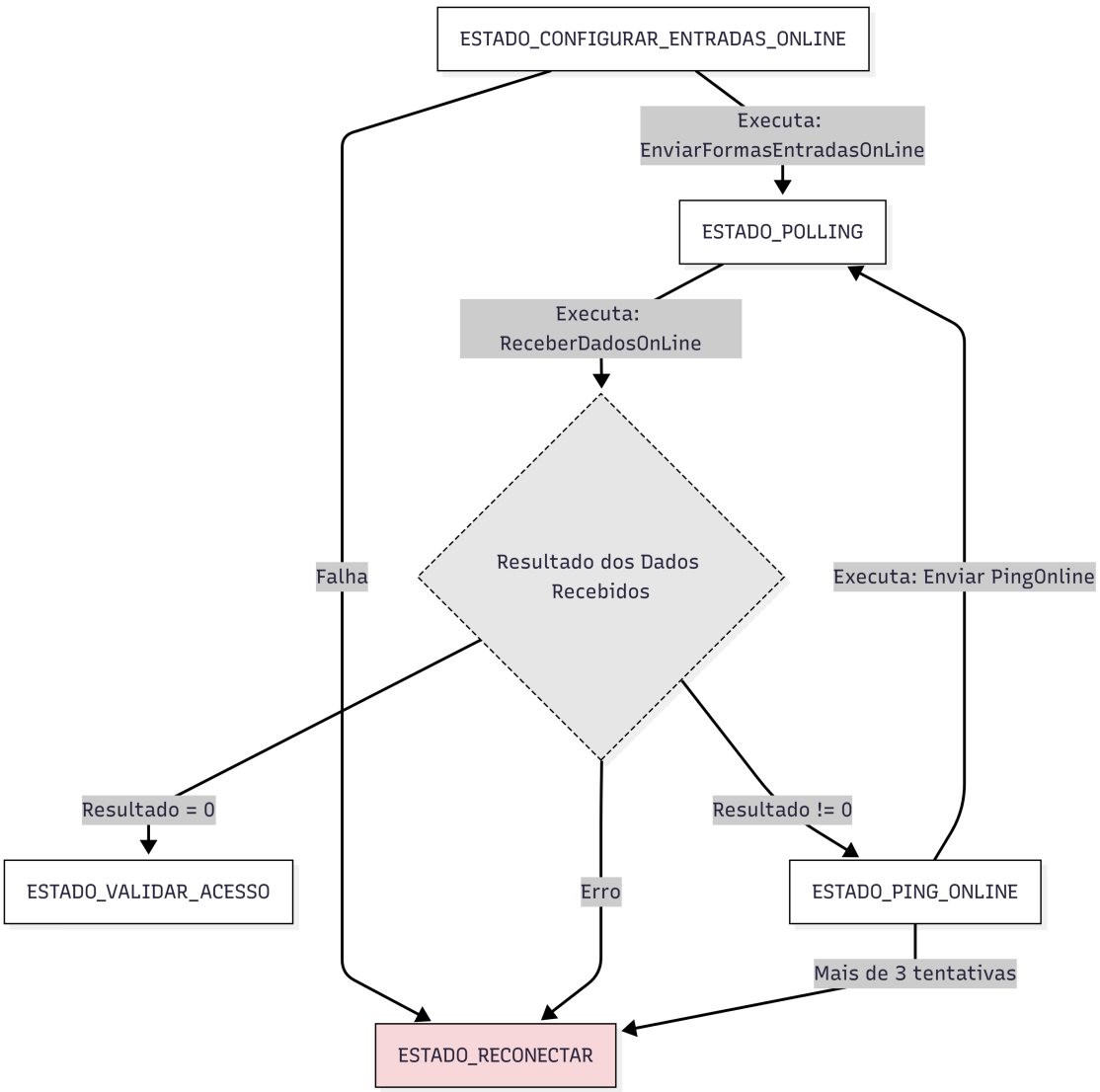
3) **Coleta de Bilhetes Offline com Verificação e Falha:** Após a configuração offline, o sistema coleta bilhetes em ciclo. A cada coleta, verifica se ainda há bilhetes a receber. Quando não houver mais, avança para exibir mensagens padrão. Falhas de leitura redirecionam à reconexão.



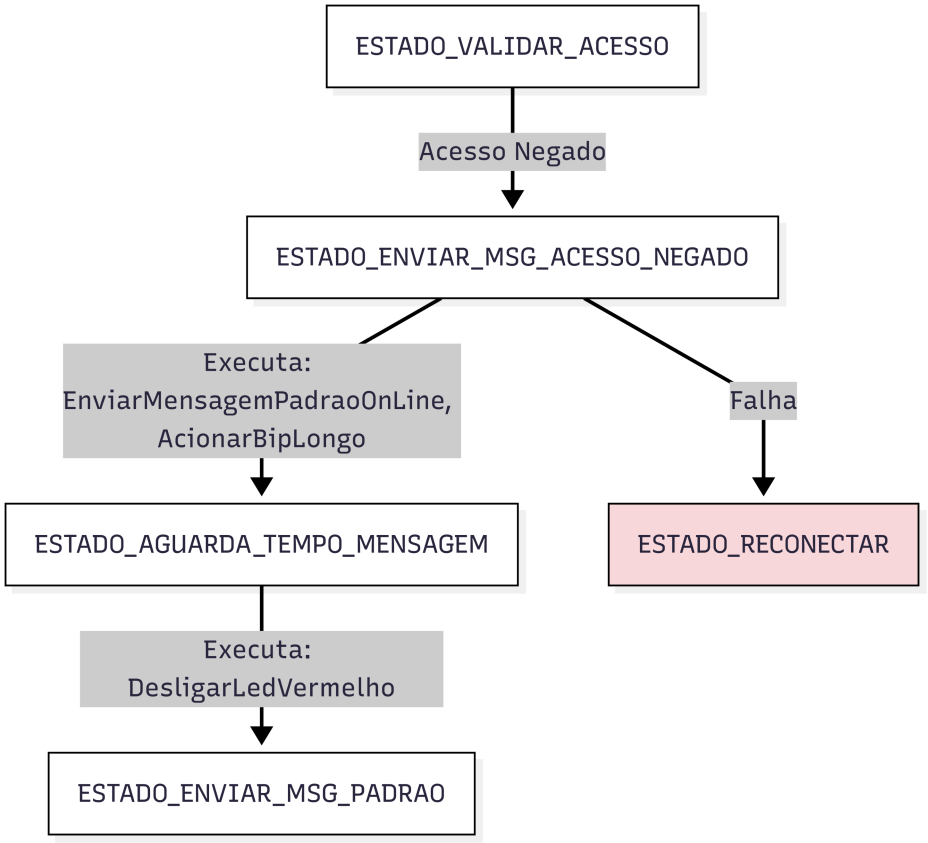
4) **Transição do Modo Offline para o Modo Online:** Após finalizar a coleta de bilhetes, o sistema exibe mensagens padrão, sincroniza o relógio e define os parâmetros que permitem mudança automática entre os modos de operação. Ao final, o Inner está pronto para iniciar operação online.



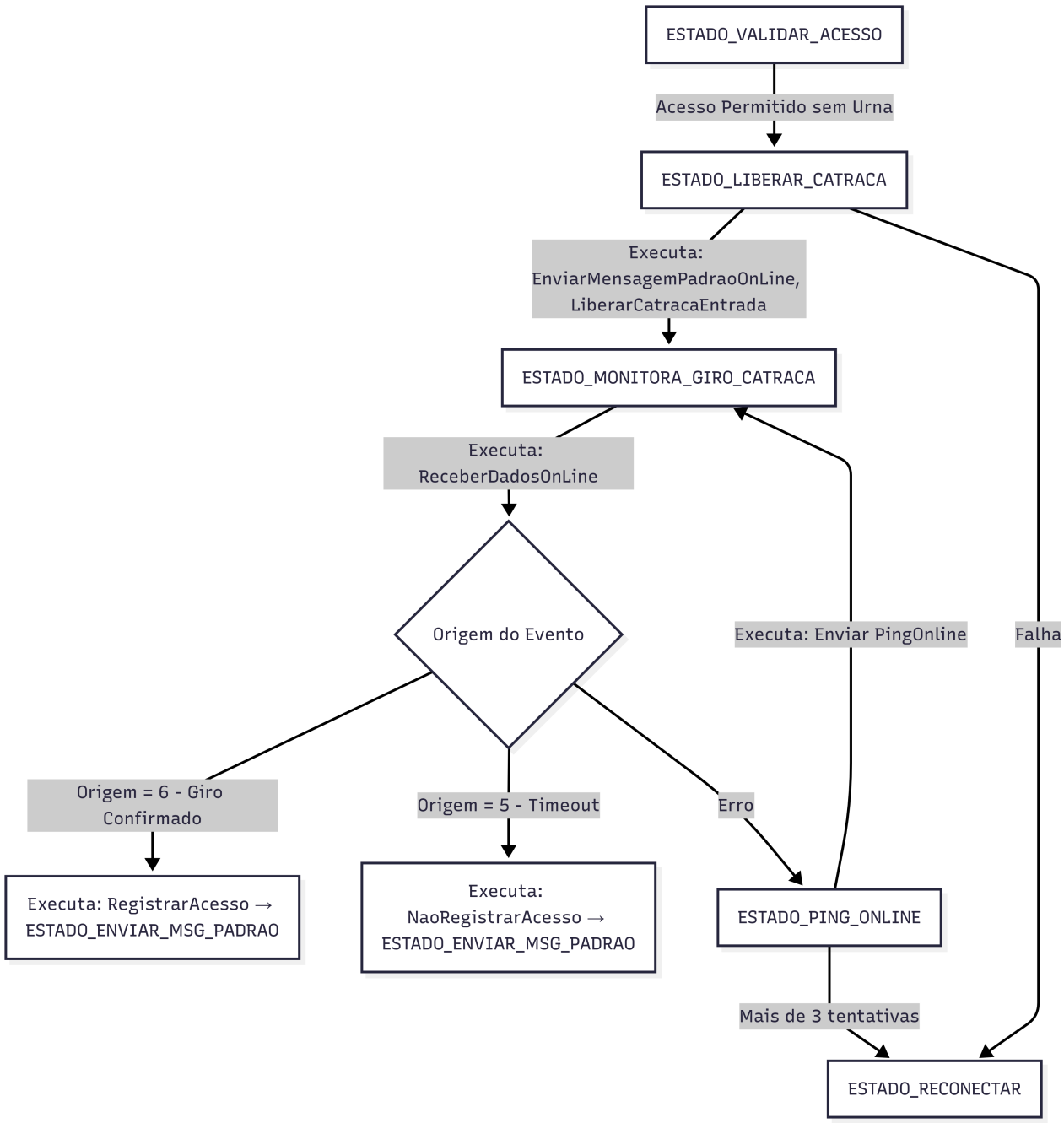
5) **Ciclo de Polling e Validação:** Este bloco mostra o ciclo de operação no modo online: o Inner solicita dados e avalia o resultado. Eventos válidos são validados; falhas ou ausência de eventos resultam em ping ou reconexão.



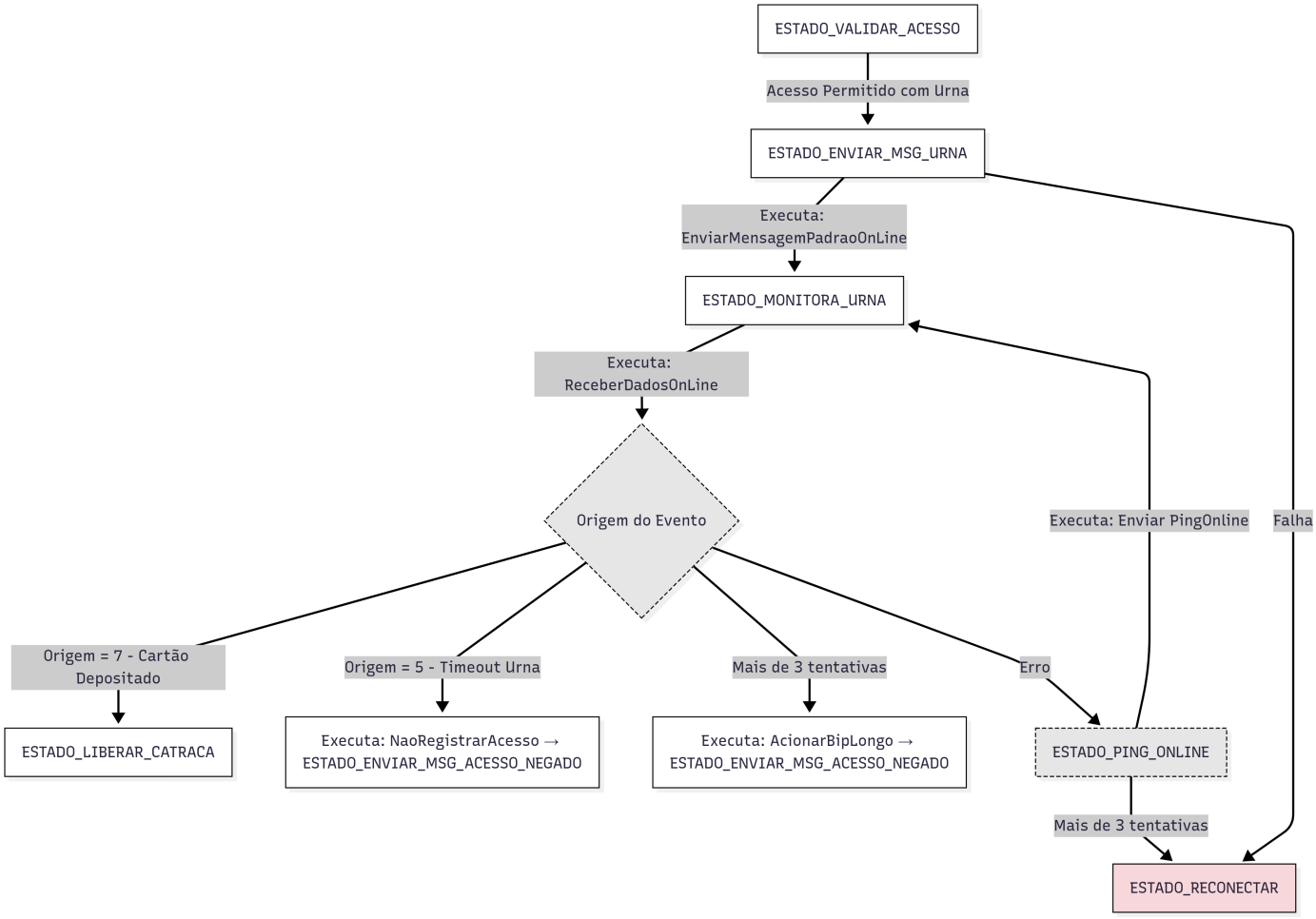
6) **Acesso Negado com Decisão por Tentativas:** Após negar o acesso, o sistema envia mensagem e sinal sonoro. Se o número de tentativas exceder o limite, reconecta. Caso contrário, aguarda e retorna ao fluxo normal.



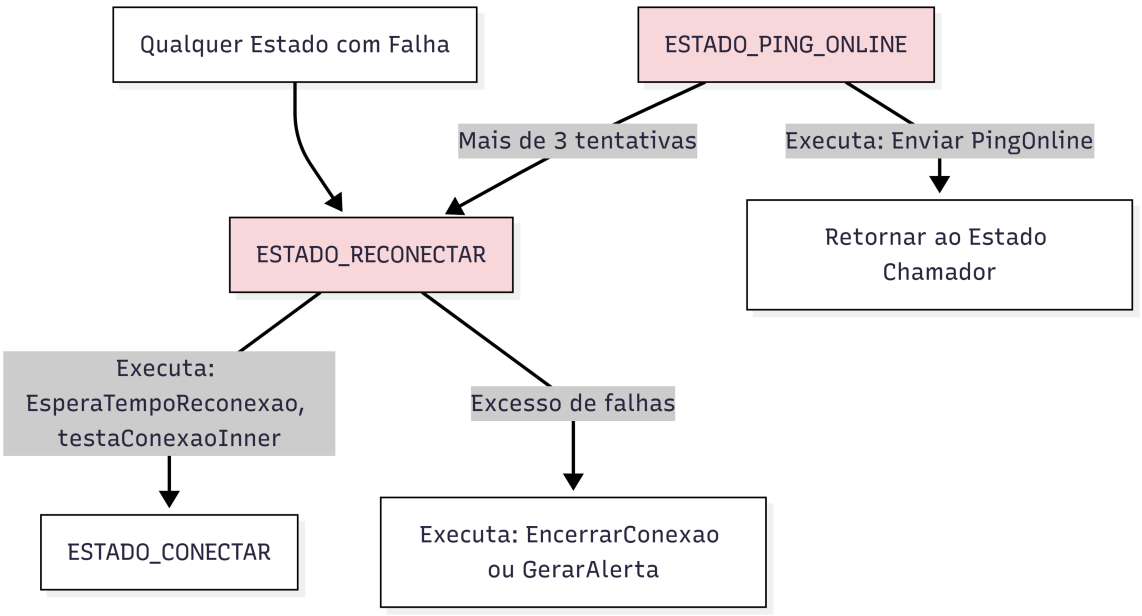
7) **Acesso Autorizado sem Urna com Monitoração de Giro:** Quando o acesso é liberado sem uso de urna, o sistema aciona a catraca e monitora o giro. Se o giro for confirmado, registra o acesso. Se houver falha ou timeout, retorna ao estado padrão.



8) **Acesso Autorizado com Urna e Monitoração de Depósito:** Neste fluxo, após autorização de acesso com urna, o sistema espera a confirmação do depósito do cartão. Se o cartão for recolhido, libera a catraca. Caso contrário, aplica as ações de erro e sinalização sonora antes de retomar o estado padrão.



9) **Tratamento de Falhas e Mecanismo de Reconexão:** Quando ocorre uma falha, o sistema entra no estado de reconexão, tenta restabelecer comunicação e retorna ao início. Caso falhe repetidamente, um alerta técnico ou encerramento é acionado. O ping online é usado para manter a conexão ativa de forma controlada.



A implementação dessa máquina de estados em uma única thread de comunicação é a abordagem necessária devido às características da EasyInner.dll.

2.2 Modos de Operação da Catraca: Online vs. Offline

As catracas Topdata podem operar em dois modos principais, cada um com características e implicações distintas para a lógica da sua aplicação. É fundamental compreender essas diferenças para configurar corretamente o equipamento e desenvolver o fluxo de controle de acesso adequado.

2.2.1 Modo Online: Controle Total pelo Software

No **modo online**, a catraca atua como um terminal "burro" ou "escravo". Isso significa que ela não toma decisões de acesso de forma autônoma. Toda a lógica de validação, permissão e liberação do giro é centralizada no seu **Software de Integração**.

Características do Modo Online:

- **Validação Centralizada:** Quando um usuário tenta acessar (passa um cartão, digita uma senha, usa biometria, etc.), a catraca lê a informação e a envia imediatamente

para o seu software. A catraca aguarda a resposta do software para saber se deve liberar ou bloquear o acesso.

- **Decisão em Tempo Real:** Seu software é o responsável por:
    - Receber o evento da catraca (EasyInner.ReceberDadosOnLine()).
    - Consultar sua base de dados ou regras de negócio para validar o acesso.
    - Enviar o comando de liberação (EasyInner.LiberarCatraca...()) ou de bloqueio para a catraca.
- **Comunicação Contínua:** Exige uma conexão TCP/IP estável e contínua entre o software e a catraca. Seu software deve manter um "polling" constante (chamando EasyInner.ReceberDadosOnLine() em loop) e enviar comandos de "keep-alive" (EasyInner.PingOnline()) para evitar que a catraca considere a conexão inativa e mude para o modo offline.
- **Flexibilidade Máxima:** Oferece a maior flexibilidade para implementar regras de acesso complexas, como limites de refeição, horários flexíveis, anti-passback, integração com outros sistemas (ERP, RH), etc., pois toda a inteligência está no seu software.
- **Configuração:** A catraca é configurada para operar em modo online através da função EasyInner.ConfigurarInnerOnline() e EasyInner.EnviarConfiguracoes().

Quando usar o Modo Online:

- Sistemas que exigem validação em tempo real (ex: consulta a banco de dados centralizado).
- Regras de acesso complexas que não podem ser armazenadas ou processadas na memória limitada da catraca.
- Necessidade de monitoramento e registro imediato de todos os eventos de acesso.
- Integração com leitores faciais ou QR Code que dependem de validação externa.

2.2.2 Modo Offline: Lógica Interna da Catraca

No **modo offline**, a catraca opera de forma autônoma, sem depender de uma conexão contínua com o software para tomar decisões de acesso. A lógica de validação e as permissões são armazenadas diretamente na memória da catraca.

Características do Modo Offline:

- **Validação Local:** A catraca possui uma lista de acesso (lista branca ou lista negra) e horários de acesso configurados em sua memória. Quando um usuário tenta acessar, a catraca valida a informação localmente e decide se libera ou bloqueia o acesso.
- **Independência da Rede:** A catraca continua funcionando mesmo que a conexão com o software seja interrompida. Isso é ideal para ambientes onde a conectividade de rede é instável ou intermitente.
- **Coleta de Bilhetes:** Os eventos de acesso (entradas, saídas, acessos negados) são armazenados na memória da catraca como "bilhetes". Seu software precisará se conectar periodicamente (EasyInner.ColetarBilhete()) para coletar esses bilhetes e sincronizá-los com sua base de dados central.
- **Funcionalidade Limitada:** As regras de acesso são mais simples, baseadas nas listas e horários pré-carregados. Regras complexas como anti-passback global ou validações externas não são possíveis neste modo.
- **Configuração:** A catraca é configurada para operar em modo offline através da função EasyInner.ConfigurarInnerOffLine() e EasyInner.EnviarConfiguracoes(). As listas de acesso são enviadas com EasyInner.EnviarListaAcesso().

Quando usar o Modo Offline:

- Ambientes com conectividade de rede instável ou inexistente.
- Necessidade de operação contínua mesmo com falha no servidor ou na rede.
- Regras de acesso simples (ex: lista de cartões permitidos em horários fixos).
- Catracas expedidoras de cartões (que operam de forma autônoma para dispensar bilhetes e armazenar eventos).

Importante:

Alguns modelos de catraca, como a Catraca Expedidora, operam intrinsecamente no modo offline para suas funções de dispensa de cartões, mesmo que estejam conectadas à rede. Nesses casos, o software se conecta para coletar os bilhetes gerados pela catraca, mas a lógica de dispensa é controlada pela própria placa da catraca.

A escolha entre modo online e offline depende diretamente dos requisitos de negócio e da infraestrutura de rede do cliente.

2.3 Entendendo os Retornos e Códigos de Erro da DLL

Ao interagir com a EasyInner.dll, cada função que você chama retornará um valor que indica o resultado da operação. Compreender esses retornos é fundamental para o tratamento de erros, a depuração da sua aplicação e a construção de uma máquina de estados robusta.

A maioria das funções da EasyInner.dll retorna um valor inteiro (geralmente um byte ou int na linguagem de programação que você estiver usando) que corresponde a um código de status.

2.3.1 Retornos Gerais de Sucesso e Falha

O retorno mais comum e importante é o que indica sucesso ou um erro genérico:

- **0 (Zero) ou RET_COMANDO_OK:**
    - **Significado:** A operação foi executada com sucesso. O comando foi enviado e/ou a resposta foi recebida conforme o esperado.
    - **Exemplo de Uso:**

```
int ret = EasyInner.AbrirPortaComunicacao(3570);

if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
{
  // Porta aberta com sucesso
}
else
{
  // Erro ao abrir a porta
}
```

Nota: O valor 0 é o padrão para sucesso em muitas APIs e DLLs.

- **1 (Um) ou RET_ERRO:**
    - **Significado:** Ocorreu um erro genérico na execução do comando. Este é um retorno abrangente que indica que a operação falhou por alguma razão não especificada por um código de erro mais detalhado. Pode ser causado por:
        - Problemas de comunicação (timeout, conexão instável).
        - Parâmetros inválidos passados para a função.
        - A catraca não está no estado esperado para receber o comando.

- Problemas internos na DLL.

- **Exemplo de Uso:**

```
int ret = EasyInner.EnviarConfiguracoes(numeroInner);
if (ret == (int)Enumeradores.Retorno.RET_ERRO)
{
  // Erro ao enviar as configurações
}
```

2.3.2 Retornos Específicos e Contextuais

Além dos retornos gerais (0 e 1), algumas funções podem retornar códigos específicos que fornecem mais detalhes sobre o motivo do sucesso ou da falha, ou sobre o tipo de evento ocorrido.

Exemplos de Retornos Contextuais Comuns na Linha Acesso:

- **EasyInner.ReceberDadosOnLine():** Esta função, usada no modo online para capturar eventos, retorna 0 (sucesso) quando um evento é recebido. O tipo de evento é então detalhado no parâmetro Origem (veja seção 4.3.2). Se não houver eventos, ela retornará um código indicando "sem eventos" (o valor exato pode ser consultado nos exemplos da SDK).
- **EasyInner.TestarConexaoInner():** Retorna 0 para sucesso na conexão. Outros valores podem indicar que a catraca não respondeu ou que a conexão não foi estabelecida.
- **Retorno 8 (Erro GPF):** Este é um erro crítico que pode ocorrer em diversas funções da DLL, incluindo EasyInner.AbrirPortaComunicacao(). Suas causas mais comuns são:
  - EasyInner.dll não registrada corretamente no sistema.
  - .NET Framework 3.5 (ou compatível) não instalado ou não habilitado no sistema operacional.
  - Incompatibilidade de versão entre a EasyInner.dll e outras DLLs de suporte da Topdata.
  - Problemas de compatibilidade de arquitetura (aplicação 64 bits tentando carregar DLL 32 bits - ver seção 1.3.3 e 6.6).
- **Retornos Específicos de Funções:** Algumas funções podem ter retornos específicos documentados em seus exemplos. Por exemplo:

- Retornos relacionados à configuração de leitores (ex: tipo de leitor inválido).
- Retornos relacionados ao envio de listas offline (ex: quantidade de dígitos inválida no cartão).
- Retornos relacionados ao status do mecanismo (ex: urna cheia - Origem 20).
- Códigos de erro comuns em operações de lista offline (Exemplos): 128 (Padrão/Quantidade/Cartão inválido), 129 (Quantidade de dígitos do cartão inválido), 130 (Horário inválido).

Onde Encontrar os Códigos de Erro Detalhados:

A principal fonte para consultar a lista completa de códigos de retorno e seus significados para cada função são os **exemplos de código da SDK Inner Acesso**.

- **Exemplos de Código:** Os projetos de exemplo (em C#, Java, Delphi, VB6) demonstram o uso das funções da EasyInner.dll e, frequentemente, incluem o tratamento de seus retornos. Ao analisar o código-fonte dos exemplos, você encontrará como os diferentes códigos de sucesso e erro são interpretados no contexto de cada função.
- **Suporte Topdata:** Para dúvidas mais aprofundadas sobre códigos de erro específicos ou o protocolo de comunicação em nível de bytes, o canal de suporte da Topdata (desenvolvimento@topdata.com.br) é o recurso mais indicado.

Um bom tratamento de retornos e erros é o que diferencia uma integração básica de uma solução profissional e resiliente.

3 Configuração Inicial e Conectividade

Antes de sua aplicação começar a interagir com as catracas Topdata, é essencial garantir que os equipamentos estejam corretamente configurados na rede e que seu software esteja preparado para estabelecer a comunicação. Este capítulo detalha as configurações de hardware e os primeiros comandos da **EasyInner.dll** para iniciar a conectividade.

3.1 Configuração de Rede da Catraca (Hardware)

Para que sua aplicação possa se comunicar com as catracas, cada equipamento precisa estar corretamente configurado na rede TCP/IP. Esta configuração é geralmente feita diretamente no equipamento, antes de iniciar a aplicação de integração.

3.1.1 Endereçamento IP

Cada catraca (seja Linha 3, Linha 4, Expedidora ou com Urna) deve possuir um endereço IP único na rede local. A escolha entre IP Fixo e DHCP depende da sua infraestrutura e necessidades:

- **IP Fixo (Recomendado):** Atribua um endereço IP estático e exclusivo para cada catraca. Isso é altamente recomendado para dispositivos de infraestrutura como catracas, pois garante que o endereço IP nunca mude, facilitando a comunicação e o troubleshooting.
- **DHCP:** A catraca pode obter um endereço IP automaticamente de um servidor DHCP na rede. Se optar por DHCP, é aconselhável configurar uma **reserva de IP (DHCP Reservation)** no seu servidor DHCP. Isso garante que a catraca sempre receba o mesmo endereço IP, mesmo que ele seja atribuído dinamicamente.

Máscara de Sub-rede e Gateway:

Configure a **Máscara de Sub-rede** e o **Gateway Padrão** de cada catraca de acordo com os parâmetros da rede local onde elas estão instaladas. Isso é crucial para que as catracas possam se comunicar dentro da rede e, se necessário, alcançar o servidor onde sua aplicação está rodando (especialmente se estiverem em sub-redes diferentes ou em nuvem via VPN).

3.1.2 IP do Servidor e Porta de Comunicação

Este é um dos pontos mais críticos da configuração:

- **IP do Servidor:** Em cada catraca, você precisa configurar o endereço IP do servidor onde o seu Software de Integração (que utiliza a SDK Inner Acesso) estará rodando. É para este IP que a catraca tentará se conectar para enviar eventos e receber comandos.
- **Porta TCP:** Configure também a porta TCP que seu software escutará. O padrão da EasyInner.dll para comunicação com catracas é a **porta 3570**. Certifique-se de que a porta configurada na catraca seja a mesma que sua aplicação escutará.

3.1.3 Número do Inner

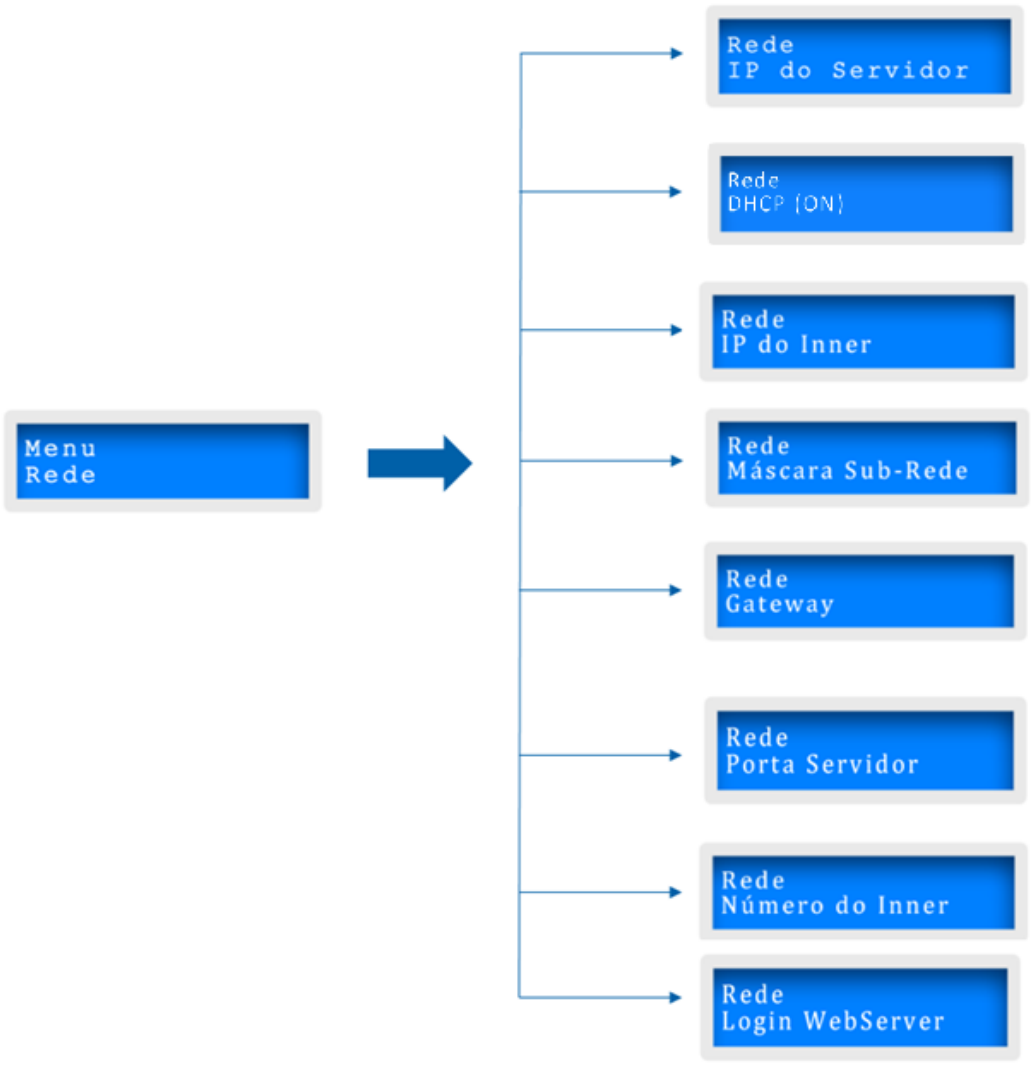
Cada catraca na rede deve ter um **Número do Inner** único. Este número, que pode variar de **1 a 99**, é utilizado pela EasyInner.dll para identificar qual equipamento está se comunicando ou para qual equipamento um comando específico deve ser enviado.

É importante ressaltar que o Número do Inner é **independente da configuração da porta TCP/IP**. Ou seja, você pode ter várias catracas conectadas à mesma porta do seu servidor (ex: 3570), e cada uma será identificada pelo seu Número do Inner exclusivo.

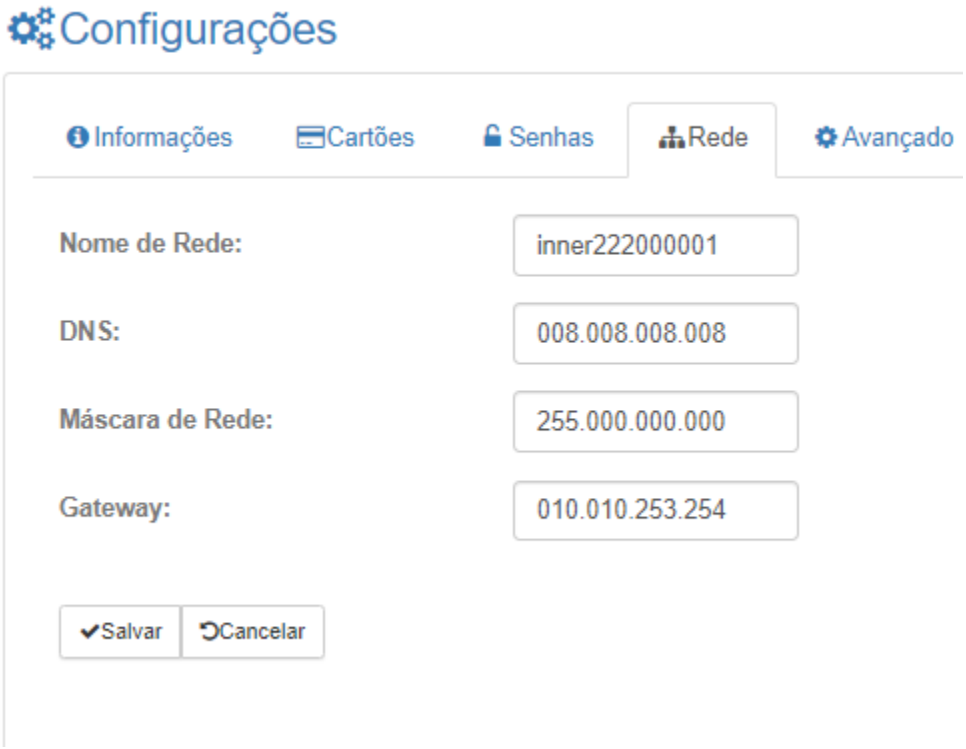
3.1.4 Como Configurar (Menu da Catraca ou WebServer)

Os parâmetros de rede (IP da catraca, Máscara, Gateway, IP do Servidor, Porta TCP, Número do Inner) são configurados de duas maneiras principais:

- **Diretamente pelo Menu da Catraca:** Todos os modelos de catraca Topdata possuem um menu de configurações acessível pelo display e teclado. Navegue pelas opções de rede para inserir os dados.



- **Via Interface Web (WebServer - Somente Catracas Linha 4):** As catracas da Linha 4 (modelos com placa "Controle Catraca", como Fit 4, Revolution 4, Box 4) possuem um WebServer embarcado. Isso permite acessar e configurar os parâmetros de rede e outras funcionalidades diretamente de um navegador web.
    - Para acessar: Digite http://[IP_DA_CATRACA] na barra de endereços do seu navegador.



3.2 Configuração de Rede do Servidor (Software)

Além das catracas, o ambiente onde seu Software de Integração está rodando também precisa de configurações de rede adequadas.

3.2.1 Liberação de Portas no Firewall

É fundamental que a porta TCP que sua aplicação estará escutando (padrão 3570) esteja liberada para **conexões de entrada** em qualquer firewall que possa existir entre as catracas e o seu servidor. Isso inclui:

- **Firewall do Sistema Operacional (Windows Firewall):** No servidor onde sua aplicação está rodando, crie uma regra de entrada para permitir o tráfego na porta 3570 (ou a porta que você escolheu).
- **Firewall de Rede (Hardware):** Se houver um firewall físico ou um roteador com firewall ativo na sua rede, certifique-se de que a porta 3570 esteja liberada e, se necessário, que haja um redirecionamento de porta (port forwarding) configurado para o IP interno do seu servidor.

- **Firewall da Rede da Catraca:** O firewall da rede onde o leitor facial (se houver) está localizado também deve permitir a saída de tráfego para o IP público do servidor nessa mesma porta.

3.3 Estabelecendo a Conexão via SDK Inner Acesso

Com as catracas configuradas na rede e o ambiente do servidor preparado, o próximo passo é iniciar a comunicação a partir do seu Software de Integração utilizando a EasyInner.dll.

3.3.1 EasyInner.DefinirTipoConexao()

Esta função informa à EasyInner.dll qual o meio de comunicação será utilizado para interagir com os equipamentos. Para a comunicação em rede (TCP/IP), o valor a ser passado é 2 (TCP/IP com Porta Fixa, onde a catraca se conecta ao servidor).

Sintaxe (C#):

```
// Define o tipo de conexão como TCP/IP com Porta Fixa

// (onde a catraca se conecta ao servidor)
int ret = EasyInner.DefinirTipoConexao(2);

if (ret != (int)Enumeradores.Retorno.RET_COMANDO_OK)
{
  // Tratar erro: não foi possível definir o tipo de conexão
  Console.WriteLine("Erro ao definir o tipo de conexão: " + ret);
  // Sua máquina de estados deve lidar com este erro
}
```

3.3.2 EasyInner.AbrirPortaComunicacao()

Após definir o tipo de conexão, esta função instrui a EasyInner.dll a começar a "escutar" por tentativas de conexão das catracas na porta TCP especificada. Esta função deve ser chamada **apenas uma vez** no início da sua aplicação, para abrir a porta do servidor.

- **Propósito:** Abrir a porta de comunicação desejada (TCP/IP, serial, etc.) para que a EasyInner.dll possa enviar e receber dados dos equipamentos.
- **Parâmetros (C#):**

```
EasyInner.AbrirPortaComunicacao(int Porta); // [Entrada] Número da porta serial ou TCP/IP (padrão 3570 para TCP/IP)
```

- **Retornos:** Esta função pode retornar os seguintes códigos:

- 0 ou RET_COMANDO_OK: A porta de comunicação foi aberta com sucesso.
- 1 ou RET_ERRO: Erro genérico ao tentar abrir a porta.
- 2: A porta de comunicação não foi aberta (pode indicar um problema específico ao tentar acessar o recurso da porta).
- 3: A porta de comunicação já está aberta (indica que a função foi chamada novamente sem fechar a porta anteriormente, ou que outro processo já a está utilizando).
- 8: Erro GPF (General Protection Fault) dentro da DLL. Este erro é crítico e pode ser causado por:
    - EasyInner.dll não registrada corretamente no sistema (embora o instalador da SDK Inner Acesso geralmente faça isso automaticamente).
    - .NET Framework 3.5 (ou compatível) não instalado ou não habilitado corretamente no sistema operacional.
    - Incompatibilidade de versão entre a EasyInner.dll e outras DLLs de suporte da Topdata.
    - Problemas de compatibilidade de arquitetura (aplicação 64 bits tentando carregar DLL 32 bits - ver seção 1.3.3 e 6.6).

- **Exemplo (C#):**

```
// Define a porta TCP que o software irá escutar


int portaEscuta = 3570; // Porta padrão da EasyInner.dll para catracas

Console.WriteLine($"Tentando abrir a porta de comunicação {portaEscuta}...");
// Tenta abrir a porta de comunicação
int ret = EasyInner.AbrirPortaComunicacao(portaEscuta);

if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
{
   Console.WriteLine($"Porta {portaEscuta} aberta com sucesso.");
   // Máquina de estados: Próximo passo (ex: ESTADO_CONECTAR)
}
else
{
   Console.WriteLine($"Erro ao abrir a porta de comunicação {portaEscuta}. Código de retorno: {ret}");
   // Tratar erro. Verificar códigos 1, 2, 3, 8 e outros.
   if (ret == 8)
```

```
    {
        Console.WriteLine("Erro 8 (GPF) - Verificar Framework 3.5, registro da DLL, compatibilidade de
versões/arquitetura.");
    }
    // Sua máquina de estados deve lidar com este erro, talvez tentando novamente ou notificando o usuário
}
```

- *Nota: Se a porta já estiver em uso por outra aplicação ou se houver problemas de permissão, esta função retornará um erro.*

3.3.3 Sugestão de Código para Conexão Inicial (C# - Exemplo Combinado):

Este código combina as etapas iniciais de configuração do tipo de conexão, abertura da porta e tentativa de conexão para uma catraca específica, simulando o fluxo do estado ESTADO_CONECTAR.

```
using System;
using System.Threading; // Necessário para Thread.Sleep

// Suponha que esta classe representa uma catraca com suas informações e estado
public class Inner
{
    public int Numero { get; set; } // Número do Inner (1 a 99)
    public Enumeradores.EstadosInner EstadoAtual { get; set; } // Estado atual na Máquina de Estados
                                    // Adicione outras propriedades relevantes como IP, Porta, etc.

    public Inner(int numeroInner)
    {
        Numero = numeroInner;
        EstadoAtual = Enumeradores.EstadosInner.ESTADO_CONECTAR; // Começa no estado de conexão
    }
}

// Suponha que EasyInner e Enumeradores são classes/enums estáticos
// acessíveis em seu projeto, referenciando a EasyInner.dll

// Simula as funções da EasyInner.dll para este exemplo (Remova em seu código real)
public static class EasyInner
{
    public static int DefinirTipoConexao(byte tipo) { Console.WriteLine($"DLL: DefinirTipoConexao({tipo})");
return 0; }
    public static int AbrirPortaComunicacao(int porta) { Console.WriteLine($"DLL:
AbrirPortaComunicacao({porta})"); return 0; } // Simula sucesso ao abrir
    public static void FecharPortaComunicacao() { Console.WriteLine("DLL: FecharPortaComunicacao()"); }
```

```
   // Simula TestarConexaoInner: Suponha que a catraca 1 conecta após 3 tentativas, a catraca 2 nunca conecta
neste exemplo
   private static int _tentativasCatraca1 = 0;
   public static int TestarConexaoInner(int inner)
   {
      Console.WriteLine($"DLL: TestarConexaoInner({inner})");
      if (inner == 1)
      {
         _tentativasCatraca1++;
         if (_tentativasCatraca1 >= 3) return (int)Enumeradores.Retorno.RET_COMANDO_OK; // Conecta após 3
tentativas
         return (int)Enumeradores.Retorno.RET_ERRO; // Simula erro/timeout antes
      }
      if (inner == 2)
      {
         return (int)Enumeradores.Retorno.RET_ERRO; // Simula sempre erro para catraca 2
      }
      return (int)Enumeradores.Retorno.RET_ERRO; // Default para outros numbers
   }

   // Outras funções da DLL...
   public static int PingOnline(int inner) { Console.WriteLine($"DLL: PingOnline({inner})"); return 0; }
   // Adicionar outras funções Dummy se necessário para o exemplo compilar
   public static int EnviarConfiguracoes(int inner) { Console.WriteLine($"DLL: EnviarConfiguracoes({inner})");
return 0; }
}

// Simula enumeradores de exemplo (Remova em seu código real)
public static class Enumeradores
{
   public enum Retorno { RET_COMANDO_OK = 0, RET_ERRO = 1 }
   public enum EstadosInner { [*]= 0, ESTADO_CONECTAR, ESTADO_ENVIAR_CFG_ONLINE, ESTADO_POLLING,
ESTADO_RECONECTAR } // Simplificado
                                                          // Adicionar outros enums dummy se necessário
   public enum PadraoCartao { PADRAO_TOPDATA, PADRAO_LIVRE }
   public enum TipoLeitor { CODIGO_DE_BARRAS, MAGNETICO, PROXIMIDADE_ABATRACK2 }
   public enum FuncaoAcionamento { NAO_UTILIZADO, ACIONA_REGISTRO_ENTRADA_OU_SAIDA }
   public enum Operacao { DESABILITADO, SOMENTE_ENTRADA, SOMENTE_SAIDA, ENTRADA_SAIDA }
}



// --- Método principal que executa a lógica de conexão ---
public class GerenciadorDeCatracas
{
   private List<Inner> _catracasParaGerenciar;
   private bool _executando = true; // Flag para controlar o loop principal
```

```
public GerenciadorDeCatracas()
{
    _catracasParaGerenciar = new List<Inner>();
    _catracasParaGerenciar.Add(new Inner(1)); // Adiciona catraca 1
    _catracasParaGerenciar.Add(new Inner(2)); // Adiciona catraca 2
}

public void IniciarGerenciamento()
{
    Console.WriteLine("Iniciando gerenciamento de catracas...");

    // 1. Definir Tipo de Conexão (TCP/IP Porta Fixa)
    int retTipo = EasyInner.DefinirTipoConexao(2);
    if (retTipo != (int)Enumeradores.Retorno.RET_COMANDO_OK)
    {
        Console.WriteLine("Falha crítica ao definir tipo de conexão. Encerrando.");
        _executando = false; // Não pode continuar
    }

    // 2. Abrir Porta de Comunicação (Porta 3570)
    int portaEscuta = 3570;
    int retPorta = EasyInner.AbrirPortaComunicacao(portaEscuta);
    if (retPorta != (int)Enumeradores.Retorno.RET_COMANDO_OK)
    {
        Console.WriteLine($"Falha crítica ao abrir porta {portaEscuta}. Encerrando.");
        _executando = false; // Não pode continuar
    }


    // 3. Loop Principal da Máquina de Estados
    while (_executando)
    {
        // Processar cada catraca individualmente
        foreach (Inner catraca in _catracasParaGerenciar)
        {
            switch (catraca.EstadoAtual)
            {
                case Enumeradores.EstadosInner.ESTADO_CONECTAR:
                    Console.WriteLine($"Processando Catraca {catraca.Numero} no estado ESTADO_CONECTAR");
                    // Lógica para tentar conectar (chamar TestarConexaoInner)
                    int retConexao = EasyInner.TestarConexaoInner(catraca.Numero);

                    if (retConexao == (int)Enumeradores.Retorno.RET_COMANDO_OK)
                    {
                        Console.WriteLine($"Catraca {catraca.Numero} conectada com sucesso!");
```

```
                    // Transitar para o próximo estado (ex: enviar configurações)
                    catraca.EstadoAtual = Enumeradores.EstadosInner.ESTADO_ENVIAR_CFG_ONLINE; // Exemplo: Ir
para Configuração
                }
                else
                {
                    // Conexão ainda não estabelecida, permanecer neste estado ou ir para reconectar
                    // Para simplificar, fica tentando neste estado. Uma FSM real teria lógica de timeout e
ESTADO_RECONECTAR.
                    Console.WriteLine($"Catraca {catraca.Numero}: Aguardando conexão. Retorno
TestarConexaoInner: {retConexao}");
                }
                break;

            case Enumeradores.EstadosInner.ESTADO_ENVIAR_CFG_ONLINE:
                Console.WriteLine($"Processando Catraca {catraca.Numero} no estado
ESTADO_ENVIAR_CFG_ONLINE");
                // Aqui você chamaria a lógica para enviar configurações (ver seção 4.1)
                // E depois mudaria de estado
                // catraca.EstadoAtual = Enumeradores.EstadosInner.ESTADO_POLLING; // Ir para polling após
configurar
                Console.WriteLine("Simulando envio de configuração...");
                EasyInner.EnviarConfiguracoes(catraca.Numero); // Apenas para mostrar chamada
                catraca.EstadoAtual = Enumeradores.EstadosInner.ESTADO_POLLING; // Próximo estado simulado
                break;

            case Enumeradores.EstadosInner.ESTADO_POLLING:
                Console.WriteLine($"Processando Catraca {catraca.Numero} no estado ESTADO_POLLING");
                // Aqui seria o loop de ReceberDadosOnLine
                // Se receber evento, muda de estado (ex: ESTADO_VALIDAR_ACESSO)
                // Se não receber evento, envia PingOnline
                // Console.WriteLine("Simulando Polling...");
                // EasyInner.PingOnline(catraca.Numero); // Exemplo de Ping
                // Lógica de ReceberDadosOnLine (não simulada em detalhe aqui)
                // Se ReceberDadosOnLine tiver dados, processar e mudar de estado
                // Caso contrário, continuar no polling ou PingOnline
                break;

            // ... outros estados da máquina ...

            default:
                // Estado desconhecido ou não tratado
                Console.WriteLine($"Catraca {catraca.Numero} em estado desconhecido:
{catraca.EstadoAtual}");
                // Lidar com estado inválido, talvez retornar para ESTADO_CONECTAR ou logar erro
                break;
        }
```

```
        }

        // Pequena pausa para não sobrecarregar o processador
        Thread.Sleep(100); // Espera 100ms antes de verificar novamente as catracas
      }

      // Ao sair do loop principal (se _executando for false)
      Console.WriteLine("Gerenciamento de catracas encerrado.");
      EasyInner.FecharPortaComunicacao();
   }
}

/*
// --- Como chamar o gerenciador em seu Main() ---
public class Program
{
  public static void Main(string[] args)
  {
    GerenciadorDeCatracas gerenciador = new GerenciadorDeCatracas();
    gerenciador.IniciarGerenciamento();

    Console.WriteLine("Aplicação finalizada.");
    // A aplicação de console pode precisar de um Console.ReadKey() para não fechar imediatamente
    // se o gerenciamento rodar em thread separada.
  }
}
*/
```

Explicação do Código:

1. **Estrutura:** O código demonstra a estrutura básica de um GerenciadorDeCatracas que mantém uma lista de objetos Inner, cada um representando uma catraca específica com seu número e estado atual.
2. **Inicialização:** O construtor do GerenciadorDeCatracas adiciona as catracas a serem gerenciadas e as inicializa no estado ESTADO_CONECTAR. Em um sistema real, a lista de catracas viria de uma configuração ou banco de dados.
3. **IniciarGerenciamento():** Este método contém o loop principal da aplicação. Ele chama DefinirTipoConexao e AbrirPortaComunicacao uma vez. Em seguida, entra em um loop while que continua enquanto _executando for true.

4. **Loop da Máquina de Estados:** Dentro do loop principal, ele itera sobre cada Inner na lista. Para cada catraca, um switch verifica o EstadoAtual e executa a lógica correspondente (simulada de forma básica neste exemplo).
5. **Estado ESTADO_CONECTAR:** Neste estado, ele chama EasyInner.TestarConexaoInner() para a catraca atual.
    - Se o retorno for RET_COMANDO_OK, a conexão foi estabelecida, e o estado da catraca muda para o próximo passo (ESTADO_ENVIAR_CFG_ONLINE neste exemplo).
    - Se o retorno não for RET_COMANDO_OK, a catraca permanece no estado ESTADO_CONECTAR, e a tentativa de conexão será repetida na próxima iteração do loop.
6. **Outros Estados (Simulados):** O código mostra placeholders para outros estados (como ESTADO_ENVIAR_CFG_ONLINE, ESTADO_POLLING), indicando onde você adicionaria a lógica correspondente.
7. **Pausa:** Thread.Sleep(2) é usada para evitar que o loop consuma 100% da CPU. Em uma aplicação real, o tempo de espera pode ser ajustado ou controlado de outras formas.
8. **Encerrar:** A porta é fechada com EasyInner.FecharPortaComunicacao() ao sair do loop principal.

Este exemplo ilustra a estrutura da máquina de estados e o processo inicial de conexão para múltiplas catracas. Ele serve como uma base para construir sua aplicação real, que precisará implementar a lógica completa de cada estado e transição.

4 Comandos Essenciais e Gerenciamento de Acesso

Este capítulo aprofunda-se nos comandos mais importantes da EasyInner.dll que sua aplicação utilizará para configurar as catracas e gerenciar o controle de acesso. É aqui que a lógica da sua máquina de estados (Capítulo 2) se conecta diretamente às funcionalidades da DLL.

4.1 Enviando Configurações para a Catraca

Após estabelecer a conexão com a catraca (conforme o ESTADO_CONECTAR do Capítulo 3), é fundamental enviar as configurações operacionais para que ela se comporte conforme o esperado pela sua aplicação. Essas configurações são montadas em um "buffer" interno da EasyInner.dll e, em seguida, enviadas em um único pacote para a catraca. Este processo geralmente ocorre nos estados ESTADO_ENVIAR_CFG_OFFLINE e ESTADO_ENVIAR_CFG_ONLINE.

4.1.1 O Processo de Montagem da Configuração

A montagem da configuração para o Inner é um dos pontos mais sensíveis da integração com a EasyInner.dll. Ela envolve chamar uma série de funções que preenchem um buffer interno na DLL com as definições desejadas para o equipamento. **É importante que todas as configurações relevantes para o modo de operação (online ou offline) e para as funcionalidades específicas da catraca sejam definidas neste buffer antes de serem enviadas.**

A ordem em que as funções de configuração são chamadas geralmente não importa, pois elas apenas adicionam informações ao buffer. O que importa é que todas as definições necessárias estejam presentes no buffer antes de chamar a função de envio (EasyInner.EnviarConfiguracoes()).

A lógica para montar a configuração geralmente reside em um método ou função dedicada em seu código (como o MontaConfiguracaoInner no exemplo SDK). Este método receberá as informações sobre a catraca atual e o modo de comunicação desejado, e chamará as funções da DLL para preencher o buffer.

Funções Comuns Utilizadas na Montagem da Configuração:

Abaixo estão exemplos de funções que são frequentemente utilizadas para montar o buffer de configuração. A escolha exata das funções e seus parâmetros dependerá do modelo da

catraca, dos leitores instalados e do comportamento desejado (modo online/offline, tipo de leitor, acionamentos, etc.).

- **Definição do Padrão e Quantidade de Dígitos do Cartão:**
  - EasyInner.DefinirPadraoCartao(): Define se o padrão é Topdata ou Livre.
  - EasyInner.DefinirQuantidadeDigitosCartao(): Define a quantidade fixa de dígitos (se aplicável).
  - EasyInner.InserirQuantidadeDigitovariavel(): Adiciona quantidades variáveis de dígitos (para leitores compatíveis).
- **Definição do Modo de Operação:**
  - EasyInner.ConfigurarInnerOffLine(): Prepara para modo offline.
  - EasyInner.ConfigurarInnerOnLine(): Prepara para modo online.
- **Configuração de Acionamentos (Relés):**
  - EasyInner.ConfigurarAcionamento1() / EasyInner.ConfigurarAcionamento2(): Definem a função e o tempo dos relés.
- **Configuração de Leitores Físicos:**
  - EasyInner.ConfigurarTipoLeitor(): Define a tecnologia do leitor (Wiegand, Abatrack, QR Code, etc.).
  - EasyInner.ConfigurarLeitor1() / EasyInner.ConfigurarLeitor2(): Definem a operação de cada leitor (entrada, saída, desabilitado).
- **Outras Configurações:**
  - EasyInner.HabilitarTeclado(): Habilita ou desabilita o teclado.
  - EasyInner.RegistrarAcessoNegado(): Configura o registro de acessos negados.
  - EasyInner.DefinirFuncaoDefaultLeitoresProximidade(): Define o tipo de registro padrão para leitores de proximidade.
  - EasyInner.ReceberDataHoraDadosOnLine(): Configura o envio de data/hora nos bilhetes online.
  - EasyInner.DefinirTipoListaAcesso(): Define o tipo de lista de acesso (branca/negra) para modo offline.

- EasyInner.ConfigurarWiegandDoisLeitores(): Configura leitores Wiegand duplos.
- EasyInner.SetarBioVariavel() / EasyInner.ConfigurarBioVariavel(): Configurações para biometria variável (se aplicável).
- Configurações específicas para Inner Acesso 2 (WebServer, Sensor Porta Offline, etc.).

Exemplo Conceitual de Montagem (Pseudocódigo):

```
// Método para montar a configuração para uma catraca específica
Funcao MontaConfiguracaoInner(catracaAtual, modoDesejado):
  // Definir Padrão do Cartão
  Se catracaAtual.PadraoCartao == PADRAO_TOPDATA:
    Chamar EasyInner.DefinirPadraoCartao(PADRAO_TOPDATA)
  Senão:
    Chamar EasyInner.DefinirPadraoCartao(PADRAO_LIVRE)

  // Definir Modo de Comunicação
  Se modoDesejado == MODO_OFF_LINE:
    Chamar EasyInner.ConfigurarInnerOffLine()
  Senão:
    Chamar EasyInner.ConfigurarInnerOnLine()

  // Configurar Acionamentos (Relés) - Exemplo para Coletor
  Se catracaAtual.TipoEquipamento == COLETOR:
    Chamar EasyInner.ConfigurarAcionamento1(ACIONA_REGISTRO_ENTRADA_OU_SAIDA, 5)
    Chamar EasyInner.ConfigurarAcionamento2(ACIONA_REGISTRO_ENTRADA_OU_SAIDA, 5)
    // Configurar Leitores para Coletor
    Se catracaAtual.DoisLeitores:
      Chamar EasyInner.ConfigurarLeitor1(SOMENTE_ENTRADA)
      Chamar EasyInner.ConfigurarLeitor2(SOMENTE_SAIDA)
    Senão:
      Chamar EasyInner.ConfigurarLeitor1(SOMENTE_ENTRADA)
      Chamar EasyInner.ConfigurarLeitor2(DESATIVADO)
  // ... Outros casos para Catraca, Urna, etc.

  // Configurar Tipo de Leitor
  Chamar ConfigurarTpLeitor(catracaAtual) // Método auxiliar para configurar tipo de leitor e dígitos
variáveis

  // Enviar Quantidade de Dígitos (se não for tratado em ConfigurarTpLeitor)
  Chamar EasyInner.DefinirQuantidadeDigitosCartao(catracaAtual.QtdDigitos)

  // Configurar Dígitos Variáveis (se aplicável e não tratado em ConfigurarTpLeitor)
  // Se catracaAtual.TipoLeitor suporta dígitos variáveis:
  //     Para cada digito em catracaAtual.DigitosVariaveis:
```

```
  //         Chamar EasyInner.InserirQuantidadeDigitoVariavel(digito)

  // Configurar Cartão Master, Teclado, Registro Negado, etc.
  Se catracaAtual.CartMaster:
    Chamar EasyInner.DefinirNumeroCartaoMaster(catracaAtual.Master)
  Chamar EasyInner.HabilitarTeclado(catracaAtual.TecladoHabilitado, EcoDisplay)
  Chamar EasyInner.RegistrarAcessoNegado(TipoRegistro)

  // Configurar Funções Default (como interpretar leitura sem botão de função)
  Chamar EasyInner.DefinirFuncaoDefaultLeitoresProximidade(Funcao)
  // ... Outras configurações ...

  // O buffer da DLL agora está preenchido com todas essas definições.
  // O próximo passo é chamar EasyInner.EnviarConfiguracoes(catracaAtual.Numero)
  // para enviar tudo para a catraca.

Fim Funcao
```

Pontos Críticos na Montagem da Configuração:

- **Completude:** Certifique-se de que todas as configurações relevantes para o funcionamento desejado da catraca estejam incluídas na montagem. Configurações ausentes podem levar a comportamentos inesperados.
- **Ordem de Chamada (Exceções):** Embora a ordem da maioria das funções de configuração não importe, DefinirPadraoCartao e DefinirQuantidadeDigitosCartao devem ser chamadas **antes** de InserirUsuarioListaAcesso. Outras funções podem ter pré-requisitos similares; consulte os exemplos SDK.
- **Funções Específicas:** Utilize as funções de configuração específicas para o modelo da sua catraca.
- **Consulte os Exemplos SDK:** Os exemplos de código fornecidos pela Topdata são a melhor referência para ver como a montagem da configuração é feita para diferentes modelos e funcionalidades. Analise o código da função que monta a configuração (geralmente chamada antes de EnviarConfiguracoes) no exemplo que mais se aproxima do seu cenário.

A montagem correta da configuração é um passo fundamental para garantir que a catraca opere de forma confiável e alinhada com a lógica do seu Software de Integração.

4.1.2 EasyInner.DefinirQuantidadeDigitosCartao()

Configura a quantidade de dígitos que a catraca espera ler nos cartões **quando utilizando um padrão de cartão que exige quantidade fixa** (como o padrão Livre com quantidade fixa).

- **Parâmetros:**
  - Quantidade (byte): Número de dígitos (de 4 a 16).
- **Exemplo (C#):**

```
// Define que a catraca lerá cartões com 10 dígitos
EasyInner.DefinirQuantidadeDigitosCartao(10);
```

- **Dígitos Variáveis:** Para leitores que podem ler cartões com **quantidades variáveis de dígitos** (como alguns leitores Abatrack II ou QR Code/Barras Serial), em vez de usar DefinirQuantidadeDigitosCartao com um valor fixo, você deve chamar EasyInner.InserirQuantidadeDigitovariavel() para *cada* quantidade de dígitos que a catraca deve aceitar (ex: chamar para 6, depois para 8, etc.). Esta função será detalhada na seção 5.1.3.

4.1.3 EasyInner.ConfigurarTipoLeitor()

Define o tipo de tecnologia do leitor físico que a catraca irá utilizar para a leitura de cartões. Esta função configura o leitor, seja ele código de barras, magnético ou proximidade.

- **Parâmetros:**
  - Tipo (byte):
    - 0 ou TpLeitor_CODIGO_DE_BARRAS: Leitor de código de barras (padrão).
    - 1 ou TpLeitor_MAGNETICO: Leitor magnético.
    - 2 ou TpLeitor_PROXIMIDADE_ABATRACK2: Leitor de proximidade Abatrack II.
    - 3 ou TpLeitor_WIEGAND: Leitor Wiegand genérico.
    - 4 ou TpLeitor_PROXIMIDADE_SMART_CARD_SERIAL: Leitor de Smart Card Serial.
    - 5 ou TpLeitor_CODIGO_BARRAS_SERIAL: Leitor de código de barras serial (usado também para QR Code via interface serial).

- 6 ou TpLeitor_WIEGAND_FC_SEM_SEPARADOR: Leitor Wiegand com Facility Code sem separador.
- 7 ou TpLeitor_TTL_SERIAL_ASCII: Leitores TTL/Serial com leitura de cartão em ASCII (mais compatível com leitores variados, incluindo QR Code serial).

- **Exemplo (C#):**

```
// Configura o leitor para um leitor QR Code serial
EasyInner.ConfigurarTipoLeitor((byte)Enumeradores.TipoLeitor.TTL_SERIAL_ASCII);
```

4.1.4 EasyInner.ConfigurarInnerOnline() / EasyInner.ConfigurarInnerOffLine()

Essas funções definem o modo principal de operação da catraca (online ou offline), conforme explicado na seção 2.2. Essas funções preparam o Inner para trabalhar no respectivo modo, mas a informação só é enviada para o equipamento na chamada de EnviarConfiguracoes.

- **EasyInner.ConfigurarInnerOnline():** Prepara a catraca para operar em modo online (decisões de acesso pelo software).
- **EasyInner.ConfigurarInnerOffLine():** Prepara a catraca para operar em modo offline (decisões de acesso pela própria catraca).
- **Exemplo (C#):**

```
// Configura a catraca para operar em modo online
EasyInner.ConfigurarInnerOnline();
```

4.1.5 EasyInner.ConfigurarAcionamento1() / EasyInner.ConfigurarAcionamento2() (Relés)

Configuram o comportamento dos relés da catraca (acionamento 1 e 2) e por quanto tempo eles permanecerão acionados após um evento. Essas funções definem como os relés (acionamentos) 1 e 2 do Inner irão funcionar. Estes comandos não devem ser utilizados em catracas se a intenção for acionar o mecanismo de giro. Para a giro de catracas, utilize os comandos EasyInner.LiberarCatraca...().

- **Parâmetros:**
  - Funcao (byte): Define a ação do relé.

- Tempo (byte): Duração do acionamento em segundos (de 0 a 50). 0 é o valor padrão.

- **Funções Comuns (Funcao):**
  - 0 ou FncAcionamento_NAO_UTILIZADO: Relé não utilizado (padrão).
  - 1 ou FncAcionamento_ACIONA_REGISTRO_ENTRADA_OU_SAIDA: Aciona ao registrar entrada ou saída.
  - 2 ou FncAcionamento_ACIONA_REGISTRO_ENTRADA: Aciona ao registrar entrada.
  - 3 ou FncAcionamento_ACIONA_REGISTRO_SAIDA: Aciona ao registrar saída.
  - 4 ou FncAcionamento_SIRENE: Conectado a uma sirene (ver funções de sirene).
  - 5 ou FncAcionamento_REVISTA: Utilizado para a revista de usuários (ver DefinirPorcentagemRevista).

- **Exemplo (C#):**

```
// Configura o acionamento 1 para acionar ao registrar uma entrada por 2 segundos
EasyInner.ConfigurarAcionamento1((byte)Enumeradores.FuncaoAcionamento.ACIONA_REGISTRO_ENTRADA, 2);
// Configura o acionamento 2 para não ser utilizado
EasyInner.ConfigurarAcionamento2((byte)Enumeradores.FuncaoAcionamento.NAO_UTILIZADO, 0);
```

4.1.6 EasyInner.ConfigurarLeitor1() / EasyInner.ConfigurarLeitor2()

Definem a operação de cada leitor físico da catraca (Leitor 1 e Leitor 2). Estas funções configuram as operações que o leitor irá executar.

- **Parâmetros:**
  - Operacao (byte):
    - 0 ou EntradasOff_LEITOR_DESABILITADO: Leitor desabilitado (padrão para Leitor 2).
    - 1 ou EntradasOff_LEITOR_SOMENTE_ENTRADA: Leitor para somente entrada.
    - 2 ou EntradasOff_LEITOR_SOMENTE_SAIDA: Leitor para somente saída.
    - 3 ou EntradasOff_LEITOR_ENTRADA_SAIDA: Leitor para entrada e saída (padrão para Leitor 1).
    - 4 ou EntradasOff_LEITOR_ENTRADA_SAIDA_INVERTIDA: Leitor para entrada e saída invertidas.

- **Exemplo (C#):**

```
// Configura o Leitor 1 para entrada e saída
EasyInner.ConfigurarLeitor1((byte)Enumeradores.Operacao.ENTRADA_SAIDA);
// Configura o Leitor 2 para somente saída
EasyInner.ConfigurarLeitor2((byte)Enumeradores.Operacao.SOMENTE_SAIDA);
```

4.1.7 EasyInner.EnviarConfiguracoes(): Aplicando as Definições

Esta é a função mais importante da etapa de configuração. Após chamar todas as funções EasyInner.Definir...() ou EasyInner.Configurar...() para montar o buffer de configurações na DLL, você deve chamar EasyInner.EnviarConfiguracoes() para enviar esse pacote completo para a catraca. Após o envio, esse buffer interno é limpo, sendo necessário chamar novamente as funções de configuração para reconfigurá-lo. **As configurações só terão efeito no equipamento após a execução desta função.**

- **Parâmetros:**
  - NumeroInner (int): O número de identificação da catraca (1 a 99) para a qual as configurações serão enviadas.
- **Retorno:** 0 (sucesso) ou código de erro.
- **Exemplo (C#):**

```
int numeroCatraca = 1; // Exemplo: Catraca com Inner #1
int ret = EasyInner.EnviarConfiguracoes(numeroCatraca);

if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
{
   Console.WriteLine($"Configurações enviadas com sucesso para a catraca {numeroCatraca}.");
   // Máquina de estados: Transitar para o próximo estado (ex: ESTADO_POLLING)
   // Sua máquina de estados deve gerenciar essa transição
}
else
{
   Console.WriteLine($"Erro ao enviar configurações para a catraca {numeroCatraca}: {ret}");
   // Máquina de estados: Tratar erro, talvez transitar para ESTADO_RECONECTAR
   // Sua máquina de estados deve gerenciar essa transição
}
```

4.1.8 Configuração de Mudança Automática Online/Offline

Em ambientes onde a estabilidade da conexão de rede não é garantida, configurar a mudança automática entre os modos online e offline é essencial para a resiliência da sua

aplicação. Isso garante que a catraca continue operando (no modo offline) se a comunicação com o software for perdida, e retorne automaticamente ao modo online quando a conexão for restabelecida.

A configuração é feita através da função EasyInner.HabilitarMudancaOnLineOffLine() e enviada com EasyInner.EnviarConfiguracoesMudancaAutomaticaOnLineOffLine().

- **EasyInner.HabilitarMudancaOnLineOffLine():** Habilita ou desabilita a mudança automática e define o tempo limite para detecção de falha na comunicação.
  - **Parâmetros:**
    - Habilita (byte):
      - 0: Desabilita a mudança automática (padrão).
      - 1: Habilita a mudança automática.
      - 2: Habilita a mudança automática no modo Online TCP com Ping (a catraca espera receber PingOnline do software).
    - Tempo (byte): Tempo em segundos (de 1 a 50) após a perda da comunicação (ou falta de PingOnline no modo 2) para que a catraca mude para o modo offline por timeout.
  - **Exemplo (C#):**

```
// Habilita a mudança automática para offline após 15 segundos sem comunicação
// Utilizando o modo que espera PingOnline
EasyInner.HabilitarMudancaOnLineOffLine(2, 15);
```

- **EasyInner.EnviarConfiguracoesMudancaAutomaticaOnLineOffLine():** Envia a configuração de mudança automática para a catraca. Esta função envia um buffer separado das configurações gerais (enviadas por EasyInner.EnviarConfiguracoes()).
  - **Parâmetros:**
    - Inner (int): O número de identificação da catraca (1 a 99).
  - **Retorno:** 0 (sucesso) ou código de erro.
  - **Exemplo (C#):**

```
int numeroCatraca = 1;
int ret = EasyInner.EnviarConfiguracoesMudancaAutomaticaOnLineOffLine(numeroCatraca);
```

```
if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
{
   Console.WriteLine($"Configuração de mudança automática enviada com sucesso para a catraca
{numeroCatraca}.");
   // Próximo passo na máquina de estados
}
else
{
   Console.WriteLine($"Erro ao enviar configuração de mudança automática para a catraca {numeroCatraca}:
{ret}");
   // Tratar erro
}
```

- **Onde Usar:** A função HabilitarMudancaOnLineOffLine() é chamada durante a fase de configuração (antes de EnviarConfiguracoesMudancaAutomaticaOnLineOffLine()). O estado PASSO_ESTADO_ENVIAR_CONFIGMUD_ONLINE_OFFLINE na máquina de estados (Capítulo 2.1.2) é responsável por chamar EnviarConfiguracoesMudancaAutomaticaOnLineOffLine() após montar a configuração.

Importância do PingOnline(): Se você configurar a mudança automática no modo 2 (HabilitarMudancaOnLineOffLine(2, ...)), sua aplicação no estado ESTADO_POLLING (ou um estado similar para manter o Inner online) deve enviar o comando EasyInner.PingOnline(NumeroInner) regularmente (em um intervalo menor que o Tempo configurado na mudança automática). Isso sinaliza para a catraca que o software está ativo e a conexão está sendo mantida. A falta do PingOnline fará com que a catraca mude para o modo offline.

Exemplo de Código para Configuração de Mudança Automática (C# - Pseudocódigo):

```
// Supondo que 'InnerAtual' é um objeto que representa a catraca atual

public void ConfigurarMudancaAutomatica(Inner InnerAtual)
{
   // Habilita a mudança automática para offline após 20 segundos sem PingOnline
   EasyInner.HabilitarMudancaOnLineOffLine(2, 20);

   // ... (Outras configurações de mudança automática, se necessário, como DefinirEntradasMudancaOffLine)

   // Envia a configuração de mudança automática para a catraca
   int ret = EasyInner.EnviarConfiguracoesMudancaAutomaticaOnLineOffLine(InnerAtual.Numero);

   if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
   {
```

```
        Console.WriteLine($"Configuração de mudança automática enviada com sucesso para a catraca
{InnerAtual.Numero}.");
        // Máquina de estados: Próximo passo (pode ser ESTADO_POLLING se já enviou configurações gerais)
    }
    else
    {
        Console.WriteLine($"Erro ao enviar configuração de mudança automática para a catraca
{InnerAtual.Numero}: {ret}");
        // Máquina de estados: Tratar erro
    }
}
```

A configuração da mudança automática é uma funcionalidade crucial para garantir a operação contínua da catraca, mesmo em face de interrupções na comunicação de rede.

4.1.9 Tabela de Comandos para Montar a Configuração

Comando (C#)	**Parâmetros (Tipos)**	**Propósito / Breve Explicação**	**Retornos (Comuns)**
EasyInner.DefinirPadraoCartao(Tipo)	byte Tipo (0: Topdata, 1: Livre)	Define o padrão de leitura dos cartões (Topdata ou Livre). Afeta como os dígitos são interpretados.	0 (Sucesso), 128 (Padrão inválido)
EasyInner.DefinirQuantidadeDigitosCartao(Quantidade)	byte Quantidade (1 a 16)	Configura a quantidade fixa de dígitos esperada nos cartões (para padrões que exigem quantidade fixa).	0 (Sucesso), 128 (Quantidade inválida)
EasyInner.InserirQuantidadeDigitovariavel(Digito)	byte Digito (1 a 16)	Adiciona uma quantidade de dígitos à lista de quantidades variáveis que a catraca deve aceitar (para leitores compatíveis com dígitos variáveis).	0 (Sucesso)
EasyInner.ConfigurarInnerOffLine()	(void)	Prepara a catraca para operar no modo offline (decisões locais).	0 (Sucesso)
EasyInner.ConfigurarInnerOnLine()	(void)	Prepara a catraca para operar no modo online (decisões pelo software).	0 (Sucesso)
EasyInner.ConfigurarAcionamento1(Funcao, Tempo)	byte Funcao, byte Tempo (0 a 50)	Configura a função e o tempo de acionamento do Relé 1.	0 (Sucesso), 128 (Função

			inválida), 129 (Tempo inválido)
EasyInner.ConfigurarAcionamento2(Funcao, Tempo)	byte Funcao, byte Tempo (0 a 50)	Configura a função e o tempo de acionamento do Relé 2.	0 (Sucesso), 128 (Função inválida), 129 (Tempo inválido)
EasyInner.ConfigurarTipoLeitor(Tipo)	byte Tipo (0 a 7)	Define a tecnologia do leitor físico (Barras, Magnético, Proximidade, etc.).	0 (Sucesso), 128 (Tipo de leitor inválido)
EasyInner.ConfigurarLeitor1(Operacao)	byte Operacao (0 a 4)	Define a operação do Leitor 1 (Desabilitado, Somente Entrada, Somente Saída, Entrada/Saída, Entrada/Saída Invertida).	0 (Sucesso), 128 (Operação inválida)
EasyInner.ConfigurarLeitor2(Operacao)	byte Operacao (0 a 4)	Define a operação do Leitor 2 (Desabilitado, Somente Entrada, Somente Saída, Entrada/Saída, Entrada/Saída Invertida).	0 (Sucesso), 129 (Operação inválida)
EasyInner.HabilitarTeclado(Habilita, Ecoar)	byte Habilita (0/1), byte Ecoar (0/1/2)	Habilita/desabilita o teclado e configura o eco no display.	0 (Sucesso), 128 (Habilita inválido), 129 (Ecoar inválido)
EasyInner.RegistrarAcessoNegado(TipoRegistro)	byte TipoRegistro (0 a 3)	Configura o registro de tentativas de acesso negado no modo offline.	0 (Sucesso), 128 (Tipo de registro inválido)
EasyInner.DefinirFuncaoDefaultLeitoresProximidade(Funcao)	byte Funcao (0 a 12)	Define o tipo de registro padrão para leitores de proximidade sem pressionar tecla de função.	0 (Sucesso), 128 (Função do leitor inválida)
EasyInner.DefinirNumeroCartaoMaster(Master)	string Master (até 14 dígitos)	Configura o número do cartão master (para liberar acesso a cartões bloqueados pela lista offline). Válido para padrão Livre.	0 (Sucesso), 128 (Número do master inválido)
EasyInner.ConfigurarWiegandDoisLeitores(	byte Habilita (0/1),	Configura leitores Wiegand	0 (Sucesso)

Habilita, ExibirMensagem)	byte ExibirMensagem (0/1)	duplos (habilita o segundo leitor Wiegand e se exibe mensagens no segundo leitor).	
EasyInner.SetarBioVariavel(Variavel)	byte Variavel (0/1)	Define se o Inner Bio utilizado é o modelo Variável (até 16 dígitos) ou 1000/4000.	0 (Sucesso)
EasyInner.ConfigurarBioVariavel(Variavel)	byte Variavel (0/1)	Configura a biometria variável (habilita/desabilita).	0 (Sucesso)
EasyInner.ReceberDataHoraDadosOnLine(Recebe)	byte Recebe (0/1)	Configura se a catraca envia data/hora nos bilhetes online.	0 (Sucesso)
EasyInner.DefinirTipoListaAcesso(Tipo)	byte Tipo (0 a 2)	Define o tipo de lista de acesso (Não usar, Branca, Negra) para modo offline.	0 (Sucesso), 128 (Tipo da lista inválido)
EasyInner.HabilitarMudancaOnLineOffLine(Habilita, Tempo)	byte Habilita (0/1/2), byte Tempo (1 a 50)	Habilita/desabilita a mudança automática online/offline e define o tempo limite.	0 (Sucesso), 128 (Habilita inválido), 129 (Tempo inválido)
EasyInner.EnviarConfiguracoesMudancaAutomaticaOnLineOffLine(Inner)	int Inner	Envia a configuração de mudança automática online/offline para a catraca.	0 (Sucesso), 1 (Erro)
EasyInner.EnviarConfiguracoes(Inner)	int Inner	**Envia todas as configurações montadas no buffer da DLL para a catraca.** Limpa o buffer após o envio.	0 (Sucesso), 1 (Erro)

Observações:

- Esta tabela lista os comandos mais comuns para montar a configuração. Outras funções de configuração podem existir dependendo da versão da DLL e do modelo da catraca.
- Os retornos listados são os mais comuns para sucesso e falha. Outros códigos de erro específicos podem ser retornados; consulte os exemplos SDK e o futuro documento de Erros Detalhados.
- As funções que montam o buffer (Definir..., Configurar..., Inserir...) geralmente retornam 0 para sucesso na adição ao buffer, mas o sucesso real da configuração no equipamento só é confirmado pelo retorno de EnviarConfiguracoes() ou EnviarConfiguracoesMudancaAutomaticaOnLineOffLine().

Esta tabela consolida as informações sobre os comandos de configuração, servindo como uma referência rápida durante o desenvolvimento.

4.2 Gerenciamento de Usuários (Modo Offline)

No modo offline, a catraca armazena uma lista de usuários permitidos (ou bloqueados) diretamente em sua memória. Isso permite que ela tome decisões de acesso de forma autônoma, sem depender de uma conexão contínua com o software. Sua aplicação será responsável por montar e enviar essa lista para a catraca.

4.2.1 EasyInner.DefinirTipoListaAcesso() (Lista Branca/Negra)

Esta função define como a catraca interpretará a lista de usuários que será enviada. De acordo com as informações da DLL, ela define qual tipo de lista(controle) de acesso o Inner vai utilizar. Após habilitar a lista de acesso é necessário preencher a lista e os horários de acesso.

- **Parâmetros:**
  - Tipo (byte):
    - 0: Não utilizar a lista de acesso (todos liberados ou todos bloqueados, dependendo de outras configurações).
    - 1: Utilizar **lista branca** (somente cartões presentes na lista terão acesso permitido; cartões fora da lista terão acesso negado).
    - 2: Utilizar **lista negra** (cartões presentes na lista terão acesso negado; cartões fora da lista terão acesso permitido).
- **Exemplo (C#):**

```
// Configura a catraca para usar uma lista branca
EasyInner.DefinirTipoListaAcesso(1);
```

4.2.2 EasyInner.InserirUsuarioListaAcesso() (Cartão, Horário)

Esta função adiciona um usuário (identificado pelo número do cartão) e associa-o a um horário de acesso específico no buffer da EasyInner.dll. Você chamará esta função para cada usuário que deseja incluir na lista. Os horários já deverão ter sido cadastrados através das funções InserirHorarioAcesso e enviados através da função EnviarHorariosAcesso para a lista ter o efeito correto.

- **Parâmetros:**

- Cartao (string): O número do cartão do usuário. O formato e a quantidade de dígitos devem estar de acordo com o que foi definido em EasyInner.DefinirPadraoCartao() e EasyInner.DefinirQuantidadeDigitosCartao() (ou InserirQuantidadeDigitovariavel()).
- Horario (byte): O número do horário de acesso associado a este usuário.
    - 1 a 100: Corresponde a um número de horário previamente cadastrado na catraca.
    - 101: Acesso sempre liberado para este usuário, independentemente do horário.
    - 102: Acesso sempre negado para este usuário, independentemente do horário.

- **Exemplo (C#):**

```
// Insere usuários na lista de acesso com diferentes horários/permissões
EasyInner.InserirUsuarioListaAcesso("12345", 101); // Cartão 12345, acesso sempre liberado
EasyInner.InserirUsuarioListaAcesso("67890", 1);   // Cartão 67890, associado ao Horário #1
EasyInner.InserirUsuarioListaAcesso("11223", 102); // Cartão 11223, acesso sempre negado
```

4.2.3 EasyInner.EnviarListaAcesso()

Após adicionar todos os usuários desejados ao buffer da DLL com EasyInner.InserirUsuarioListaAcesso(), esta função envia a lista completa para a catraca. Após executar esta função, o buffer com os usuários da lista é limpo automaticamente pela DLL.

- **Parâmetros:**
    - NumeroInner (int): O número de identificação da catraca (1 a 99) para a qual a lista será enviada.
- **Retorno:** 0 (sucesso) ou código de erro.
- **Exemplo (C#):**

```
int numeroCatraca = 1;
int ret = EasyInner.EnviarListaAcesso(numeroCatraca);

if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
{
  Console.WriteLine($"Lista de acesso enviada com sucesso para a catraca {numeroCatraca}.");
  // Transitar para o próximo estado
}
```

```
else
{
  Console.WriteLine($"Erro ao enviar lista de acesso para a catraca {numeroCatraca}: {ret}");
  // Tratar erro
}
```

- **Atualização da Lista:** Para alterar ou excluir um usuário no modo offline, você deve **reenviar a lista completa** de usuários, com as alterações aplicadas (o usuário alterado com o novo horário, ou o usuário excluído simplesmente não sendo incluído na nova lista). A catraca sobrescreverá a lista existente com a nova lista enviada.

4.2.4 EasyInner.ApagarListaAcesso()

Esta função limpa o buffer da lista de acesso na EasyInner.dll. De acordo com a descrição da DLL, ela limpa o buffer com a lista de usuários cadastrados e envia automaticamente para o Inner (embora a descrição de "envia automaticamente" deva ser validada com o comportamento real, o propósito principal é limpar o buffer na DLL). É uma boa prática chamá-la antes de começar a montar uma nova lista de usuários para garantir que não haja dados residuais de envios anteriores.

- **Parâmetros:**
  - Inner (int): O número de identificação da catraca (1 a 99). (Verificar nos exemplos se este parâmetro é realmente usado ou se limpa o buffer global).
- **Exemplo (C#):**

```
// Limpa o buffer da lista de acesso na DLL (verificar se o parâmetro Inner é necessário)
EasyInner.ApagarListaAcesso(); // Ou EasyInner.ApagarListaAcesso(numeroCatraca);
```

Exemplo de Código para Gerenciamento de Lista Offline (C# - Pseudocódigo):

```
// Supondo que 'InnerAtual' é um objeto que representa a catraca atual
// e que 'listaDeUsuariosDoSeuSistema' é uma coleção de usuários a serem enviados

public void EnviarListaDeAcessoOffline(Inner InnerAtual, List<Usuario> listaDeUsuariosDoSeuSistema)
{
  // 1. Limpar o buffer da lista na DLL antes de montar uma nova
  EasyInner.ApagarListaAcesso(); // Verificar se precisa passar InnerAtual.Numero

  // 2. Definir o tipo de lista (ex: lista branca)
  EasyInner.DefinirTipoListaAcesso(1); // 1 = Lista Branca
```

```
  // 3. Definir padrão e quantidade de dígitos do cartão (se ainda não configurado)
  // Essas funções devem ser chamadas ANTES de InserirUsuarioListaAcesso,
  // conforme a descrição da DLL (página 90).
  EasyInner.DefinirPadraoCartao((byte)Enumeradores.PadraoCartao.PADRAO_LIVRE);
  EasyInner.DefinirQuantidadeDigitosCartao(14); // Exemplo: 14 dígitos fixos
  // OU usar InserirQuantidadeDigitovariavel(digito) para padrões variáveis

  // 4. Inserir cada usuário na lista da DLL
  // Os horários (1-100) devem ser previamente cadastrados e enviados com EnviarHorariosAcesso (página 89).
  foreach (Usuario user in listaDeUsuariosDoSeuSistema)
  {
    // user.CartaoId deve ser o número do cartão (string)
    // user.HorarioAcesso deve ser 1-100, 101 (liberado) ou 102 (negado)
    EasyInner.InserirUsuarioListaAcesso(user.CartaoId, (byte)user.HorarioAcesso);
  }

  // 5. Enviar a lista completa para a catraca
  int ret = EasyInner.EnviarListaAcesso(InnerAtual.Numero);

  if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
  {
    Console.WriteLine($"Lista de acesso enviada com sucesso para a catraca {InnerAtual.Numero}.");
    // Máquina de estados: Mudar o estado ou registrar sucesso
  }
  else
  {
    Console.WriteLine($"Erro ao enviar lista de acesso para a catraca {InnerAtual.Numero}: {ret}");
    // Máquina de estados: Tratar erro
  }
}
```

Observação sobre Horários de Acesso: Para utilizar os horários de 1 a 100 associados aos usuários, é necessário configurar esses horários previamente na catraca. Isso é feito através das funções EasyInner.InserirHorarioAcesso() (insere um horário no buffer da DLL) e EasyInner.EnviarHorariosAcesso() (envia os horários para a catraca). Essas funções são semelhantes ao processo de envio da lista de usuários.

Importante: O gerenciamento de biometrias digitais ou faciais no modo offline é feito através de comandos específicos que interagem com os módulos biométricos da catraca. Para biometria digital, a EasyInner.dll possui funções como EasyInner.EnviarDigitalUsuarioBio(). Para biometria facial, uma SDK separada é utilizada (veja seção 5.4).

4.3 Recebendo Eventos em Tempo Real (Modo Online)

No modo online, a catraca atua como um terminal que envia eventos para o seu Software de Integração. Para que sua aplicação possa reagir a esses eventos (como a leitura de um cartão, um giro da catraca, ou um acesso negado), ela precisa estar continuamente "escutando" e processando os dados que vêm do equipamento. Isso é feito principalmente através da função EasyInner.ReceberDadosOnLine().

4.3.1 EasyInner.ReceberDadosOnLine(): Capturando Acessos e Eventos

Esta é a função central para a comunicação em tempo real no modo online. **É uma função bloqueante.** Sua aplicação deve chamá-la repetidamente em um loop (geralmente dentro do ESTADO_POLLING da sua máquina de estados, em uma thread dedicada à comunicação) para verificar se há novos eventos vindos da catraca específica.

- **Propósito:** Receber dados de eventos gerados pela catraca, como a leitura de um cartão, o acionamento de um teclado, o giro da catraca, ou mensagens de status.
- **Comportamento:** A função EasyInner.ReceberDadosOnLine() **pausa a execução da thread que a chama** até que:
    - Um evento seja recebido da catraca (neste caso, retorna 0 - sucesso - e preenche os parâmetros de saída).
    - Ocorra um timeout na espera por um evento (retorna um código indicando "sem eventos").
    - Ocorra um erro na comunicação (retorna um código de erro).
- **Importância da Thread Dedicada:** Devido a ser bloqueante, **esta função NUNCA deve ser chamada na thread principal da interface gráfica (UI)**, pois isso travaria a aplicação. Ela deve ser executada em uma thread separada, gerenciada pela lógica da sua máquina de estados.
- **Parâmetros (C#):**

```
// Exemplo de parâmetros para a função ReceberDadosOnLine
// Note que muitos parâmetros são passados por referência (ref) para serem preenchidos pela DLL
EasyInner.ReceberDadosOnLine(
  int NumeroInner,        // [Entrada] Número da catraca (Inner) que está sendo monitorada
  ref byte Origem,        // [Saída] Tipo de evento ocorrido (ex: leitor 1, teclado, biometria)
  ref byte Complemento,   // [Saída] Informação adicional sobre a origem (ex: sentido do giro)
  StringBuilder Cartao,   // [Saída] Número do cartão lido (ou QR Code, senha, etc.)
```

```
  ref byte Dia,           // [Saída] Dia do evento
  ref byte Mes,           // [Saída] Mês do evento
  ref byte Ano,           // [Saída] Ano do evento
  ref byte Hora,          // [Saída] Hora do evento
  ref byte Minuto,        // [Saída] Minuto do evento
  ref byte Segundo        // [Saída] Segundo do evento
);
```

- **NumeroInner:** O número de identificação da catraca que você está monitorando.
- **Origem:** Este é um parâmetro crucial que indica o tipo de evento que ocorreu (detalhado na seção 4.3.2).
- **Complemento:** Fornece informações adicionais sobre a Origem. Por exemplo, para eventos de giro, pode indicar o sentido.
- **Cartao:** Uma StringBuilder (ou tipo equivalente na sua linguagem) que será preenchida com o número do cartão, QR Code, senha digitada, ou ID biométrico lido. É importante inicializar este StringBuilder com um tamanho suficiente para o maior número de cartão esperado (ex: 16 ou 20 caracteres).
- **Dia, Mes, Ano, Hora, Minuto, Segundo:** Data e hora exatas em que o evento ocorreu na catraca.

- **Exemplo de Uso (C# - dentro do ESTADO_POLLING na thread de comunicação):**

```
// Este código é parte do loop da Máquina de Estados que roda em UMA thread dedicada.
// Ele itera por cada catraca gerenciada.

// Supondo que 'catraca' é o objeto Inner atual no loop
// Supondo que 'sbCartao' é um StringBuilder inicializado antes do loop principal da FSM

if (catraca.EstadoAtual == Enumeradores.EstadosInner.ESTADO_POLLING)
{
   byte origem = 0;
   byte complemento = 0;
   byte dia = 0, mes = 0, ano = 0, hora = 0, minuto = 0, segundo = 0;
   sbCartao.Clear(); // Limpar o StringBuilder antes de cada chamada

   // Tenta receber dados da catraca (esta chamada é BLOQUEANTE)
   int ret = EasyInner.ReceberDadosOnLine(
     catraca.Numero,
     ref origem,
     ref complemento,
     sbCartao,
```

```
      ref dia, ref mes, ref ano, ref hora, ref minuto, ref segundo
   );

   if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
   {
      // Evento recebido com sucesso! A thread foi desbloqueada por um evento.
      Console.WriteLine($"Evento da catraca {catraca.Numero}: Origem={origem}, Cartão={sbCartao.ToString()},
Hora={hora}:{minuto}");

      // Mudar o estado desta catraca para processar o evento
      // (ex: ESTADO_VALIDAR_ACESSO).
      catraca.EstadoAtual = Enumeradores.EstadosInner.ESTADO_VALIDAR_ACESSO;
      catraca.UltimoEvento = new EventoCatraca(origem, complemento, sbCartao.ToString(), dia, mes, ano, hora,
minuto, segundo);

      // A thread continuará no loop principal e passará a verificar a próxima catraca.
   }
   // Verificar outros retornos possíveis de ReceberDadosOnLine (consultar exemplos/documentação)
   else if (ret == (int)Enumeradores.Retorno.RET_SEM_EVENTOS) // Exemplo: Se DLL retornar um código para "sem
eventos"
   {
      // NENHUM evento ocorreu para esta catraca neste ciclo de polling.
      // A thread continua para a próxima catraca.
      // Pode ser necessário enviar um PingOnline periodicamente aqui para manter a catraca online (se
configurado).
      EasyInner.PingOnline(catraca.Numero);
   }
   else
   {
      // Erro na comunicação com esta catraca.
      Console.WriteLine($"Erro ao receber dados da catraca {catraca.Numero}: {ret}");
      // Mudar o estado desta catraca para ESTADO_RECONECTAR
      catraca.EstadoAtual = Enumeradores.EstadosInner.ESTADO_RECONECTAR;

      // A thread continuará no loop principal e passará a verificar a próxima catraca.
   }
} // Fim do 'if' que verifica o estado ESTADO_POLLING
```

A função EasyInner.ReceberDadosOnLine() é a base para a inteligência do seu sistema no modo online. Sua natureza bloqueante requer execução em thread separada, e a Máquina de Estados gerencia a alternância entre as catracas nessa thread.

4.3.2 Tabela de Origens de Eventos

O parâmetro Origem retornado pela função EasyInner.ReceberDadosOnLine() (ou similar na sua linguagem) é fundamental para identificar o tipo de evento que ocorreu na catraca e determinar a ação que sua aplicação deve tomar. Abaixo estão as principais origens e seus significados, conforme informações da DLL e histórico de chat:

Valor da Origem	Nome (Comum nos Exemplos/SDK)	Significado	Notas
1	ORIGEM_TECLADO	Dados digitados no teclado da catraca. O parâmetro Cartao conterá a sequência numérica digitada.	Pode ser usado para acesso via senha ou para menus no teclado.
2	ORIGEM_LEITOR1	Leitura realizada pelo **Leitor 1** da catraca. O parâmetro Cartao conterá o número do cartão (RFID, código de barras, etc.).	O tipo de leitor físico associado ao Leitor 1 é configurado via EasyInner.ConfigurarLeitor1().
3	ORIGEM_LEITOR2	Leitura realizada pelo **Leitor 2** da catraca. O parâmetro Cartao conterá o número do cartão.	O tipo de leitor físico associado ao Leitor 2 é configurado via EasyInner.ConfigurarLeitor2().
4	ORIGEM_SENSOR_CATRACA	Evento do sensor de giro da catraca (obsoleto em modelos mais recentes, o giro é geralmente indicado pela Origem 6).	Raramente usado em implementações modernas.
5	ORIGEM_FIM_TEMPO_ACIONAMENTO	O tempo configurado para um acionamento (relé) expirou.	Usado para saber se um giro não ocorreu dentro do tempo liberado ou se um relé desligou.
6	ORIGEM_GIRO_CATRACA_TOPDATA	Confirmação de que o giro da catraca foi concluído. Este evento é gerado pelo sensor óptico após uma liberação bem-sucedida.	Crucial para registrar acessos efetivos após uma liberação.
7	ORIGEM_CARTAO_RECOLHIDO_URNA	Um cartão foi detectado e recolhido pela urna coletora.	Específico para catracas com urna. Usado no fluxo de saída com coleta de cartão.
8	ORIGEM_EVENTO_SENSOR1	Um evento ocorreu no Sensor 1 (geralmente associado a	A configuração do Sensor 1 é feita com comandos específicos

		botões ou sensores externos configurados).	(ex: DefinirEventoSensor).
9	ORIGEM_EVENTO_SENSOR2	Um evento ocorreu no Sensor 2 (geralmente associado a botões ou sensores externos configurados).	A configuração do Sensor 2 é feita com comandos específicos (ex: DefinirEventoSensor).
10	ORIGEM_EVENTO_SENSOR3	Um evento ocorreu no Sensor 3 (geralmente associado a botões ou sensores externos configurados).	A configuração do Sensor 3 é feita com comandos específicos (ex: DefinirEventoSensor).
12	ORIGEM_SENSOR_BIOMETRICO	Leitura realizada pelo sensor biométrico (impressão digital, identificação ou verificação). O parâmetro Cartao pode conter o ID do usuário associado à biometria.	Específico para catracas com leitor biométrico digital.
13	ORIGEM_RESPOSTA_REQUISICAO_BIO	Resposta a uma requisição interna da catraca ao módulo biométrico. Não é uma resposta direta ao envio de biometria pelo software.	Geralmente gerado pela catraca para sua própria lógica interna com a biometria.
18	ORIGEM_TEMPLATE	Recebimento ou envio de dados de template biométrico (impressão digital).	Usado em processos de cadastro ou transferência de digitais.
20	ORIGEM_URNA_CHEIA	A urna coletora de cartões está cheia.	Específico para catracas com urna.
21	ORIGEM_QRCODE	Leitura realizada por um leitor de QR Code. O parâmetro Cartao conterá a sequência lida do QR Code.	Específico para catracas com leitor de QR Code.

Como Usar as Origens na sua Aplicação:

Dentro do loop que chama EasyInner.ReceberDadosOnLine(), sua máquina de estados (ou a lógica do estado ESTADO_POLLING) deve ter um switch ou estrutura condicional para verificar o valor do parâmetro Origem e executar a ação apropriada.

Exemplo (C# - Dentro do Estado ESTADO_POLLING):

```
// ... (código para chamar EasyInner.ReceberDadosOnLine, como na seção 4.3.1)
```

```
if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
{
  // Evento recebido!
  switch (origem)
  {
    case (byte)Enumeradores.Origem.ORIGEM_LEITOR1:
      // Evento de leitura pelo Leitor 1
      Console.WriteLine($"Cartão {sbCartao.ToString()} lido no Leitor 1 da catraca {InnerAtual.Numero}.");
      // Mudar o estado para validar o acesso (ESTADO_VALIDAR_ACESSO)
      InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_VALIDAR_ACESSO;
      InnerAtual.UltimoEvento = new EventoCatraca(origem, complemento, sbCartao.ToString(), dia, mes, ano,
hora, minuto, segundo);
      break;

    case (byte)Enumeradores.Origem.ORIGEM_TECLADO:
      // Evento de digitação no teclado
      Console.WriteLine($"Senha/Código {sbCartao.ToString()} digitado na catraca {InnerAtual.Numero}.");
      // Mudar o estado para validar a senha
      InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_VALIDAR_ACESSO;
      InnerAtual.UltimoEvento = new EventoCatraca(origem, complemento, sbCartao.ToString(), dia, mes, ano,
hora, minuto, segundo);
      break;

    case (byte)Enumeradores.Origem.ORIGEM_GIRO_CATRACA_TOPDATA:
      // Evento de giro da catraca
      Console.WriteLine($"Giro confirmado na catraca {InnerAtual.Numero}.");
      // Registrar o acesso efetivo
      // Mudar o estado de volta para POLLING ou outro estado apropriado
      InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_POLLING;
      InnerAtual.UltimoEvento = new EventoCatraca(origem, complemento, sbCartao.ToString(), dia, mes, ano,
hora, minuto, segundo);
      break;

    case (byte)Enumeradores.Origem.ORIGEM_CARTAO_RECOLHIDO_URNA:
      // Evento de cartão recolhido pela urna
      Console.WriteLine($"Cartão {sbCartao.ToString()} recolhido pela urna da catraca {InnerAtual.Numero}.");
      // Mudar o estado para processar o cartão recolhido
      InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_VALIDAR_ACESSO_URNA; // Exemplo de novo estado
      InnerAtual.UltimoEvento = new EventoCatraca(origem, complemento, sbCartao.ToString(), dia, mes, ano,
hora, minuto, segundo);
      break;

    case (byte)Enumeradores.Origem.ORIGEM_URNA_CHEIA:
      // Evento de urna cheia
      Console.WriteLine($"Urna da catraca {InnerAtual.Numero} está cheia.");
      // Notificar o operador, impedir novos depósitos, etc.
```

```
      // Permanecer no estado POLLING
      InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_POLLING;
      InnerAtual.UltimoEvento = new EventoCatraca(origem, complemento, sbCartao.ToString(), dia, mes, ano,
hora, minuto, segundo);
      break;

    case (byte)Enumeradores.Origem.ORIGEM_QRCODE:
      // Evento de leitura QR Code
      Console.WriteLine($"QR Code {sbCartao.ToString()} lido na catraca {InnerAtual.Numero}.");
      // Mudar o estado para validar o QR Code
      InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_VALIDAR_ACESSO; // Ou um estado específico
para QR Code
      InnerAtual.UltimoEvento = new EventoCatraca(origem, complemento, sbCartao.ToString(), dia, mes, ano,
hora, minuto, segundo);
      break;

    // ... outros casos de origem ...

    default:
      // Outras origens ou origens não tratadas
      Console.WriteLine($"Evento desconhecido da catraca {InnerAtual.Numero}. Origem: {origem}, Complemento:
{complemento}");
      // Permanecer no estado POLLING ou tratar conforme a lógica
      InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_POLLING;
      InnerAtual.UltimoEvento = new EventoCatraca(origem, complemento, sbCartao.ToString(), dia, mes, ano,
hora, minuto, segundo);
      break;
  }
}
// ... (restante do loop de polling para sem eventos, erros, etc.)
```

Entender as origens de evento permite que sua aplicação no modo online saiba exatamente o que está acontecendo na catraca e reaja a cada interação do usuário ou do equipamento.

Você está certíssimo! A coleta de bilhetes (EasyInner.ColetarBilhete()) é uma funcionalidade crucial não apenas para a Catraca Expedidora, mas para **qualquer catraca ou coletor que opere em modo offline** em algum momento. Essa coleta é o meio pelo qual o Software de Integração recupera o histórico de acessos registrados localmente no equipamento durante períodos de desconexão.

4.4 Coleta de Bilhetes Offline

Qualquer catraca ou coletor configurado para operar no modo offline (totalmente ou em momentos de desconexão automática) registra os eventos de acesso em sua memória

interna como **bilhetes**. Esses bilhetes representam o histórico de quem acessou, quando e como. Sua aplicação deve coletar esses bilhetes para manter seu banco de dados centralizado atualizado. Este processo é geralmente executado após a catraca estabelecer ou restabelecer a conexão online.

4.4.1 EasyInner.ColetarBilhete(): Recuperando Registros Offline

Esta função é utilizada para coletar os bilhetes armazenados na memória de um equipamento que operou em modo offline. Ela coleta um bilhete Offline que está armazenado na memória do Inner.

- **Propósito:** Recuperar um registro de evento de acesso armazenado localmente no equipamento.
- **Comportamento:** Sua aplicação deve chamar EasyInner.ColetarBilhete() repetidamente para uma catraca específica (no estado ESTADO_COLETAR_BILHETES ou similar). A cada chamada bem-sucedida que retorna 0 (sucesso), a função preenche os parâmetros de saída com os dados de um bilhete e o remove da memória do equipamento. Se não houver mais bilhetes pendentes, ela retornará um código indicando "sem bilhetes".
- **Parâmetros (C#):**

```
// Exemplo de parâmetros para a função ColetarBilhete
// Os dados do bilhete são retornados por referência ou via PChar/StringBuilder
EasyInner.ColetarBilhete(
  int NumeroInner,      // [Entrada] Número da catraca
  ref byte Tipo,        // [Saída] Tipo do bilhete (acesso, negação, etc.)
  ref byte Dia,         // [Saída] Dia do evento
  ref byte Mes,         // [Saída] Mês do evento
  ref byte Ano,         // [Saída] Ano do evento
  ref byte Hora,        // [Saída] Hora do evento
  ref byte Minuto,      // [Saída] Minuto do evento
  StringBuilder Cartao  // [Saída] Número do cartão do usuário (ou informação relevante)
);
```

  - **NumeroInner:** O número de identificação da catraca que está coletando os bilhetes.
  - **Tipo:** Indica o tipo específico do bilhete coletado (semelhante às origens, mas para registros offline). Alguns tipos comuns: 10 (Entrada pelo cartão), 11 (Saída pelo cartão), 12 (Tentativa de entrada negada), 13 (Tentativa de saída negada),

110 (Entrada pelo teclado), etc. O valor 128 indica bilhete repetido recebido do Inner (página 102).

- **Dia, Mes, Ano, Hora, Minuto:** Data e hora do evento registrado.
- **Cartao:** Contém o número do cartão do usuário associado ao bilhete.

- **Exemplo de Uso (C# - dentro de um estado ESTADO_COLETAR_BILHETES):**

```
// Supondo que 'InnerAtual' representa a catraca que operou offline
// e que 'sbCartao' é um StringBuilder inicializado

byte tipoBilhete = 0;
byte dia = 0, mes = 0, ano = 0, hora = 0, minuto = 0;
StringBuilder sbCartao = new StringBuilder(20);

Console.WriteLine($"Iniciando coleta de bilhetes offline da catraca {InnerAtual.Numero}...");

// Loop para coletar todos os bilhetes pendentes
while (true)
{
   // Tenta coletar um bilhete
   int ret = EasyInner.ColetarBilhete(
     InnerAtual.Numero,
     ref tipoBilhete,
     ref dia, ref mes, ref ano, ref hora, ref minuto,
     sbCartao
   );

   if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
   {
      // Bilhete coletado com sucesso!
      Console.WriteLine($"Bilhete coletado: Tipo={tipoBilhete}, Cartão={sbCartao.ToString()},
Data/Hora={dia}/{mes}/{ano} {hora}:{minuto}");

      // Processar o tipo de bilhete (ver tipos comuns) e registrar no seu sistema central
      // O loop continua para coletar o próximo bilhete
   }
   else if (ret == (int)Enumeradores.Retorno.RET_SEM_BILHETES) // Exemplo de retorno para "sem bilhetes"
   {
      // Não há mais bilhetes pendentes nesta catraca
      Console.WriteLine($"Catraca {InnerAtual.Numero}: Coleta de bilhetes offline finalizada.");
      // Máquina de estados: Transitar para o próximo estado (ex: ESTADO_POLLING)
      InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_POLLING;
      break; // Sai do loop de coleta
   }
   else
```

```
    {
        // Erro na comunicação durante a coleta
        Console.WriteLine($"Erro ao coletar bilhete da catraca {InnerAtual.Numero}: {ret}");
        // Máquina de estados: Tratar erro, talvez transitar para ESTADO_RECONECTAR
        InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_RECONECTAR;
        break; // Sai do loop de coleta
    }
}
```

- **Fluxo na Máquina de Estados:** O estado ESTADO_COLETAR_BILHETES (ou um estado similar no seu design da FSM) deve ser alcançado sempre que uma catraca que pode operar offline se conecta ou reconecta. Dentro desse estado, o loop de ColetarBilhete deve ser executado até que todos os bilhetes sejam recuperados (retorno indicando sem bilhetes). Em seguida, a catraca pode transitar para o modo online (ESTADO_POLLING) se essa for a configuração padrão, ou permanecer offline, dependendo do seu sistema.

Esta funcionalidade é vital para sistemas que não exigem controle online 100% do tempo, garantindo que nenhum registro de acesso seja perdido.

4.5 Liberando o Acesso da Catraca

No modo online, após seu Software de Integração receber um evento de acesso (como a leitura de um cartão, Origem 2 ou 3) e validar que o acesso é permitido (no estado ESTADO_VALIDAR_ACESSO), o próximo passo é enviar um comando para a catraca liberar o mecanismo de giro. As funções EasyInner.LiberarCatraca...() são utilizadas para este propósito.

- **Propósito:** Enviar um comando para a catraca destravar o mecanismo de giro, permitindo que o usuário passe.
- **Comportamento:** Sua aplicação deve chamar a função apropriada (dependendo do sentido desejado) após validar o acesso. A catraca receberá o comando e acionará o relé correspondente pelo tempo configurado nos acionamentos (ConfigurarAcionamento1/2) ou pelo tempo padrão de liberação.
- **Onde Usar:** Estas funções são tipicamente chamadas no estado ESTADO_LIBERAR_CATRACA da sua máquina de estados, após o ESTADO_VALIDAR_ACESSO confirmar que o acesso é permitido.

Aqui estão as principais funções de liberação:

- **EasyInner.LiberarCatracaEntrada():**
  - **Descrição:** Libera o giro da catraca especificamente para o sentido de **entrada**.
  - **Parâmetros:**
    - Inner (int): O número de identificação da catraca (1 a 99).
  - **Exemplo (C#):**

```
// Libera a catraca para a entrada
EasyInner.LiberarCatracaEntrada(InnerAtual.Numero);
```

- **EasyInner.LiberarCatracaSaida():**
  - **Descrição:** Libera o giro da catraca especificamente para o sentido de **saída**.
  - **Parâmetros:**
    - Inner (int): O número de identificação da catraca (1 a 99).
  - **Exemplo (C#):**

```
// Libera a catraca para a saída
EasyInner.LiberarCatracaSaida(InnerAtual.Numero);
```

- **EasyInner.LiberarCatracaDoisSentidos():**
  - **Descrição:** Libera o giro da catraca em **ambos os sentidos** (entrada e saída).
  - **Parâmetros:**
    - Inner (int): O número de identificação da catraca (1 a 99).
  - **Exemplo (C#):**

```
// Libera a catraca em ambos os sentidos
EasyInner.LiberarCatracaDoisSentidos(InnerAtual.Numero);
```

- **Funções Invertidas (EasyInner.LiberarCatracaEntradaInvertida(), EasyInner.LiberarCatracaSaidaInvertida()):** Algumas instalações podem exigir que o sentido de liberação seja invertido em relação ao padrão. As funções LiberarCatracaEntradaInvertida e LiberarCatracaSaidaInvertida existem para esses casos. Consulte a documentação e os exemplos SDK para determinar a necessidade de uso baseado na orientação física da catraca.

Tempo de Liberação: O tempo que a catraca permanece destravada após um comando de liberação é definido nas configurações de acionamento (EasyInner.ConfigurarAcionamento1() / ConfigurarAcionamento2(), seção 4.1.5) ou por um tempo padrão interno do firmware. O "tempo de giro" não é uma configuração separada, mas sim a duração em que o relé de liberação permanece acionado, permitindo o giro.

Após a Liberação: Depois de enviar o comando de liberação, sua máquina de estados deve geralmente transitar para um estado para **monitorar se o giro realmente ocorreu** (como ESTADO_MONITORA_GIRO_CATRACA). Isso é feito continuando a chamar EasyInner.ReceberDadosOnLine() e esperando um evento de Origem 6 (ORIGEM_GIRO_CATRACA_TOPDATA), que confirma o giro bem-sucedido. Se um evento Origem 5 (ORIGEM_FIM_TEMPO_ACIONAMENTO) for recebido antes da Origem 6, significa que o tempo de liberação expirou e o giro não ocorreu.

Exemplo de Código para Liberação (C# - Dentro do Estado ESTADO_VALIDAR_ACESSO ou ESTADO_LIBERAR_CATRACA):

```
// Supondo que 'InnerAtual' representa a catraca
// Supondo que a lógica de validação no ESTADO_VALIDAR_ACESSO definiu que o acesso é permitido e qual o
sentido

public void ProcessarLiberacao(Inner InnerAtual)
{
   // Exemplo: Lógica para liberar baseado no sentido validado anteriormente
   int ret = -1;

   // Supondo que InnerAtual.SentidoValidado seja 'Entrada' ou 'Saida'
   if (InnerAtual.SentidoValidado == "Entrada")
   {
      Console.WriteLine($"Acesso permitido para entrada na catraca {InnerAtual.Numero}. Liberando giro...");
      ret = EasyInner.LiberarCatracaEntrada(InnerAtual.Numero); // Ou LiberarCatracaEntradaInvertida
   }
   else if (InnerAtual.SentidoValidado == "Saida")
   {
      Console.WriteLine($"Acesso permitido para saída na catraca {InnerAtual.Numero}. Liberando giro...");
      ret = EasyInner.LiberarCatracaSaida(InnerAtual.Numero); // Ou LiberarCatracaSaidaInvertida
   }
   else // Ex: Acesso liberado para ambos
   {
      Console.WriteLine($"Acesso permitido para ambos os sentidos na catraca {InnerAtual.Numero}. Liberando
giro...");
      ret = EasyInner.LiberarCatracaDoisSentidos(InnerAtual.Numero);
   }
```

```
    if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
    {
        Console.WriteLine($"Comando de liberação enviado com sucesso para catraca {InnerAtual.Numero}.");
        // Máquina de estados: Mudar para monitorar o giro
        InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_MONITORA_GIRO_CATRACA;
    }
    else
    {
        Console.WriteLine($"Erro ao enviar comando de liberação para catraca {InnerAtual.Numero}: {ret}");
        // Máquina de estados: Tratar erro, talvez transitar para ESTADO_POLLING ou ESTADO_RECONECTAR
        InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_POLLING; // Exemplo
    }
}
```

Liberar o acesso da catraca é um dos comandos mais frequentes no modo online, diretamente vinculado à lógica de validação do seu sistema.

4.6 Outros Comandos Úteis

Além dos comandos essenciais para configuração, gerenciamento de usuários offline, recebimento de eventos e liberação de acesso, a EasyInner.dll oferece outras funções úteis para interagir com as catracas.

4.6.1 Comandos de Data/Hora

É fundamental manter a data e hora das catracas sincronizadas para garantir a precisão dos registros de acesso.

- **EasyInner.EnviarRelogio():** Configura a data e hora da catraca.
    - **Parâmetros (C#):** int Inner, byte Dia, byte Mes, byte Ano, byte Hora, byte Minuto, byte Segundo.
    - **Exemplo:** Enviar a data/hora atual do servidor para a catraca.
- **EasyInner.ReceberRelogio():** Solicita a data e hora atualmente configuradas na catraca.
    - **Parâmetros (C#):** int Inner, ref byte Dia, ref byte Mes, ref byte Ano, ref byte Hora, ref byte Minuto, ref byte Segundo.
    - **Exemplo:** Verificar a data/hora da catraca.

- **EasyInner.EnviarHorarioVerao():** Configura o período de horário de verão na catraca.

4.6.2 Comandos de Sinalização (Bips e LEDs)

Estes comandos permitem interagir com os elementos de sinalização visual e sonora da catraca para fornecer feedback ao usuário.

- **EasyInner.AcionarBipCurto() / EasyInner.AcionarBipLongo():** Fazem a catraca emitir um bip curto ou longo.
    - **Parâmetros (C#):** int Inner.
- **EasyInner.LigarLedVerde() / EasyInner.DesligarLedVerde():** Controlam o LED verde (geralmente indica acesso liberado).
    - **Parâmetros (C#):** int Inner. Função somente para Inner Acesso.
- **EasyInner.LigarLedVermelho() / EasyInner.DesligarLedVermelho():** Controlam o LED vermelho (geralmente indica acesso negado).
    - **Parâmetros (C#):** int Inner. Função somente para Inner Acesso.
- **EasyInner.LigarBackLite() / EasyInner.DesligarBackLite():** Controlam a luz de fundo do display.
    - **Parâmetros (C#):** int Inner.

4.6.3 Comandos de Mensagens no Display

Permitem exibir mensagens personalizadas no display da catraca (no modo online).

- **EasyInner.EnviarMensagemPadraoOnLine():** Envia uma mensagem fixa para ser exibida no display enquanto a catraca está ociosa no modo online.
    - **Parâmetros (C#):** int Inner, byte ExibirData, string Mensagem. ExibirData controla se a data/hora é exibida na linha superior. O tamanho da mensagem é limitado (32 caracteres ou 16 se exibir data/hora).
- **EasyInner.EnviarMensagemTemporariaOnLine():** Envia uma mensagem que será exibida por um tempo determinado.

4.6.4 Outros Comandos

- **EasyInner.PingOnline():** Essencial para manter a catraca em modo online quando configurada para a mudança automática modo 2 (HabilitarMudancaOnLineOffLine(2, ...)). Deve ser enviado periodicamente (intervalo menor que o tempo configurado na mudança automática).
    - **Parâmetros (C#):** int Inner.
- **EasyInner.FecharPortaComunicacao():** Fecha a porta de comunicação aberta por EasyInner.AbrirPortaComunicacao(). Essencial ao finalizar a aplicação.

Tabela 6: Resumo de Outros Comandos Úteis

Comando (C#)	Parâmetros (Tipos)	Propósito / Descrição	Notas
EasyInner.EnviarRelogio(Inner, Dia, Mes, Ano, Hora, Minuto, Segundo)	int, byte, byte, byte, byte, byte, byte	Configura a data e hora da catraca.	Essencial para sincronização.
EasyInner.ReceberRelogio(Inner, ref Dia, ref Mes, ref Ano, ref Hora, ref Minuto, ref Segundo)	int, ref byte, ref byte, ref byte, ref byte, ref byte, ref byte	Solicita e recebe a data e hora atual da catraca.	Use para verificar sincronização.
EasyInner.EnviarHorarioVerao(Inner, DiaInicio, MesInicio, AnoInicio, HoraInicio, MinutoInicio, DiaFim, MesFim, AnoFim, HoraFim, MinutoFim)	int, byte, byte, byte, byte, byte, byte, byte, byte, byte, byte	Configura o período de horário de verão na catraca (início e fim completos).	Configuração sazonal.
EasyInner.AcionarBipCurto(Inner)	int	Faz a catraca emitir um bip curto.	Feedback sonoro (ex: acesso permitido/negado).
EasyInner.AcionarBipLongo(Inner)	int	Faz a catraca emitir um bip longo.	Feedback sonoro (ex: atenção, erro).
EasyInner.LigarLedVerde(Inner)	int	Acende o LED verde no display da catraca.	Sinalização visual (ex:

			acesso liberado). Somente Inner Acesso.
EasyInner.DesligarLedVerde(Inner)	int	Desliga o LED verde.	Desliga sinalização. Somente Inner Acesso.
EasyInner.LigarLedVermelho(Inner)	int	Acende o LED vermelho no display da catraca.	Sinalização visual (ex: acesso negado). Somente Inner Acesso.
EasyInner.DesligarLedVermelho(Inner)	int	Desliga o LED vermelho.	Desliga sinalização. Somente Inner Acesso.
EasyInner.LigarBackLite(Inner)	int	Acende a luz de fundo do display da catraca.	Melhora visibilidade do display.
EasyInner.DesligarBackLite(Inner)	int	Desliga a luz de fundo do display.	Desliga iluminação.
EasyInner.EnviarMensagemPadraoOnLine(Inner, ExibirData, Mensagem)	int, byte, string	Exibe uma mensagem fixa no display em modo online ocioso.	Útil para mensagens de boas-vindas. Tamanho limitado.
EasyInner.EnviarMensagemTemporariaOnLine(Inner, ExibirData, Mensagem, Tempo)	int, byte, string, byte	Exibe uma mensagem por um tempo determinado.	Para mensagens de status temporárias.
EasyInner.PingOnline(Inner) (Comando 26)	int	Testa a comunicação e mantém a catraca online (se configurada para mudança automática modo 2).	Essencial para estabilidade em modo online com mudança automática.

EasyInner.FecharPortaComunicacao()	(void)	Fecha a porta de comunicação aberta por AbrirPortaComunicacao().	Chamar ao encerrar a aplicação.

Esta tabela resume alguns dos comandos úteis que podem ser integrados à sua aplicação para melhorar a interação com a catraca. Para detalhes completos de parâmetros, retornos e comportamento, consulte os exemplos SDK.

Ao integrar esses comandos em sua máquina de estados, você pode criar uma experiência de usuário mais completa e informativa na catraca.

5 Integrações Específicas e Modelos de Catraca

Embora a **SDK Inner Acesso** e a EasyInner.dll forneçam uma base unificada para a comunicação com a linha Acesso da Topdata, diferentes modelos de catracas e funcionalidades (como expedidora, urna, biometria facial) podem exigir configurações ou fluxos de comunicação específicos. Este capítulo explora essas particularidades.

5.1 Catraca Linha 3 (Inner Acesso) vs. Catraca Linha 4 (Controle Catraca)

As catracas da **Linha 3** e **Linha 4** diferem significativamente em seu hardware interno, o que impacta suas capacidades e funcionalidades de integração. Compreender essas distinções é vital para configurar a comunicação corretamente.

5.1.1 Diferenças de Hardware e Firmware

A principal diferença entre as catracas da Linha 3 e Linha 4 reside na placa controladora utilizada:

- **Catraca Linha 3:** Inclui modelos como **Fit 3, Revolution 3, Box 3**, e utiliza a placa mais antiga, conhecida como **"Inner Acesso"** (ou PCI Inner Acesso). Esta placa não é mais fabricada.
- **Catraca Linha 4:** Inclui modelos como **Fit 4, Revolution 4, Box 4**, e utiliza a placa mais recente, conhecida como **"Controle Catraca"** (ou PCI Controle Catraca).

Essas diferenças de hardware implicam em diferentes versões de firmware e capacidades:

- **Firmware:** A versão do firmware está diretamente ligada ao modelo da placa. Firmwares para a placa Inner Acesso (Linha 3) são diferentes dos firmwares para os modelos com placa Controle Catraca (Linha 4). As versões mais recentes e funcionalidades avançadas estão geralmente disponíveis apenas para a placa Controle Catraca.
- **Funcionalidades:** Algumas funcionalidades, como o WebServer embarcado, suporte a protocolos de leitor mais avançados (como Wiegand 42 bits) ou certas configurações de biometria (como o módulo LC na placa Inner Acesso a partir da versão 6.13 do firmware), podem ser exclusivas de um tipo de placa ou de versões de firmware mais recentes.

Como Identificar o Modelo da Catraca (Linha 3 vs. Linha 4):

Existem diversas formas de identificar se a catraca é da Linha 3 ou Linha 4, tanto pela interface quanto pela SDK:

1. **Via Display (Inicialização):** Ao ligar a catraca, a versão da placa (e, portanto, a linha) é exibida brevemente no display.
    - Se aparecer **"Inner Acesso"**, é uma Catraca da Linha 3.
    - Se aparecer **"Controle Catraca"**, é uma Catraca da Linha 4.
2. **Via SDK (Configuração Offline):** Envie qualquer configuração no modo offline utilizando o SDK e observe o campo "Versão" no retorno da configuração.
    - Se a versão retornar como **"Inner Acesso"**, é uma Catraca da Linha 3.
    - Se a versão retornar como **"Catraca"** (ou "Controle Catraca" dependendo da versão), é uma Catraca da Linha 4.
    - [Sugestão de Imagem 8: Screenshot do retorno da SDK mostrando o campo "Versão" com "Inner Acesso" e "Catraca 4".]
3. **Via SDK (EasyInner.ReceberVersaoFirmware()):** Esta função permite consultar diretamente a versão do firmware e obter informações sobre a linha do produto. É uma forma programática e confiável de identificar a catraca.
    - **Propósito:** Solicita e recebe informações detalhadas sobre a versão do firmware da catraca, incluindo a linha do produto.
    - **Parâmetros (C#):**

```
EasyInner.ReceberVersaoFirmware(

  int Inner,          // [Entrada] Número da catraca
  ref byte Linha,     // [Saída] Linha do produto
  ref short Variacao, // [Saída] Variação do firmware
  ref byte VersaoAlta,// [Saída] Versão alta do firmware
  ref byte VersaoBaixa,// [Saída] Versão baixa do firmware
  ref byte VersaoSufixo,// [Saída] Versão sufixo do firmware
  ref byte InnerAcessoBio // [Saída] Indica se é Inner Acesso configurado com BIO
              // Note: Parâmetros e tipos podem variar dependendo da versão da DLL e da linguagem
);
```

- **Interpretação do parâmetro Linha:** O parâmetro Linha retorna um código que identifica a linha do produto. Abaixo estão alguns dos códigos de linha comuns para a linha Acesso:

Código da Linha	Descrição Comum
1	Inner Plus
2	Inner Disk
3	Inner Verid
6	Inner Bio
7	Inner net
14	Inner Acesso
16	Controle Catraca
18	Inner Acesso 2

```
// Supondo que 'InnerAtual' é um objeto que representa a catraca atual e contém seu Numero (1 a 99).
public void IdentificarLinhaCatraca(Inner InnerAtual)
{
  byte linha = 0;
  short variacao = 0;
  byte versaoAlta = 0, versaoBaixa = 0, versaoSufixo = 0;
  byte innerAcessoBio = 0;
  // Declare outras variáveis de saída se a assinatura da função for diferente

  Console.WriteLine($"Solicitando versão do firmware para catraca {InnerAtual.Numero}...");

  // Chama a função para receber as informações de versão
  // Verifique a assinatura correta da função e a ordem dos parâmetros na documentação/exemplos
  int ret = EasyInner.ReceberVersaoFirmware(
    InnerAtual.Numero,
    ref linha,
    ref variacao,
    ref versaoAlta,
    ref versaoBaixa,
    ref versaoSufixo,
    ref innerAcessoBio
  // Passe outras variáveis por referência conforme a assinatura da função
  );

  if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
  {
    Console.WriteLine($"Versão recebida com sucesso da catraca {InnerAtual.Numero}.");
```

```
    Console.WriteLine($"  Linha: {linha}, Variação: {variacao}, Versão: {versaoAlta}.{versaoBaixa}.
{versaoSufixo}");

    // Interpreta o código da Linha para identificar o modelo
    // Consulte a Tabela de Códigos de Linha na seção 5.1.1 e no documento de Erros Detalhados
    switch (linha)
    {
      case 14: // Exemplo de código para Inner Acesso
        Console.WriteLine($"  Identificada como: Catraca Linha 3 (Placa Inner Acesso)");
        InnerAtual.LinhaProduto = "Linha 3"; // Atualiza informação no seu objeto de catraca
        break;
      // Adicione outros casos de outros códigos de Linhas de equipamentos Topdata
      default:
        Console.WriteLine($"  Identificada como: Catraca Linha 4 (Placa Controle Catraca) ou Outro Modelo");
        InnerAtual.LinhaProduto = "Linha 4"; // Assume Linha 4 para códigos diferentes de Linha 3 conhecidos
        break;
    }
  }
  else
  {
    Console.WriteLine($"Erro ao receber versão do firmware da catraca {InnerAtual.Numero}: {ret}");
    // Tratar erro (timeout, comunicação, etc.)
  }
}

// Exemplo de estrutura para armazenar dados da catraca no seu software
/*
public class Inner
{
  public int Numero { get; set; } // Número do Inner (1-99)
  public Enumeradores.EstadosInner EstadoAtual { get; set; } // Estado atual da Máquina de Estados
  public string LinhaProduto { get; set; } // Para armazenar a linha identificada ("Linha 3" ou "Linha 4")
  // ... outras propriedades (IP, Porta, etc.)
}
*/
```

5.1.2 Opções de Leitura (Wiegand, Abatrack, QR Code)

A forma como a catraca lê os diferentes tipos de cartões e identificadores depende tanto do tipo de leitor físico instalado quanto da configuração de "Tipo de Leitor" enviada via EasyInner.ConfigurarTipoLeitor() (seção 4.1.3). A compatibilidade com diferentes protocolos de leitura pode variar entre as Linhas 3 e 4, especialmente quando se tenta usar múltiplos leitores ou leitores com formatos complexos.

- **Protocolos Comuns:**
    - **Wiegand:** Um padrão comum para leitores de proximidade (RFID). Pode incluir Facility Code (FC).
    - **Abatrack II:** Outro padrão para leitores de proximidade, comum em cartões magnéticos também.
    - **Código de Barras Serial / TTL Serial ASCII:** Utilizado para leitores de código de barras e leitores de QR Code conectados via interface serial.
- **Catraca Linha 3 (Inner Acesso):** Geralmente mais limitada na flexibilidade de leitura simultânea de protocolos diferentes ou na configuração avançada de formatos. Para usar múltiplos leitores (ex: facial + proximidade) na Linha 3, é crucial que ambos estejam configurados para o mesmo protocolo e quantidade de dígitos (ex: ambos Wiegand com 8 dígitos), já que ela não consegue distinguir protocolos diferentes automaticamente. As opções "WIEGAND_FC_SEM_SEPARADOR" ou "WIEGAND" com 6 ou 8 dígitos são configurações comuns.
- **Catraca Linha 4 (Controle Catraca):** Oferece maior flexibilidade. A placa Controle Catraca consegue, em certas configurações (como Abatrack II com dígitos variáveis, ou "Barras, Prox, QR Code com Letras"), identificar automaticamente qual protocolo está sendo recebido (Wiegand ou Abatrack/Serial). Isso permite o uso conjunto de leitores que utilizam protocolos diferentes sem que o software precise tratar essa distinção na leitura.

5.1.3 EasyInner.InserirQuantidadeDigitovariable()

Esta função é particularmente útil para catracas (especialmente a Linha 4 com placa Controle Catraca) configuradas para leitores com protocolos que suportam uma quantidade variável de dígitos, como Abatrack II ou Código de Barras Serial/QR Code.

- **Propósito:** Configurar a catraca para aceitar cartões com diferentes quantidades de dígitos dentro de uma faixa definida. Em vez de um número fixo de dígitos, você informa à catraca quais tamanhos de cartões ela deve esperar.
- **Uso:** Chame esta função para cada quantidade de dígitos que seus cartões podem ter. A função deve ser chamada para cada dígito que o Inner deverá ler.
- **Parâmetros:**

- Quantidade (byte): Uma quantidade de dígitos válida para seus cartões (de 1 a 16). O valor 0 desabilita a leitura de cartões com quantidades de dígitos diferentes.

- **Exemplo (C#):**

```
// Configurar catraca para ler cartões com 4, 6, 8, 10, 12 ou 14 dígitos (Abatrack II ou QR Code)
EasyInner.DefinirPadraoCartao((byte)Enumeradores.PadraoCartao.PADRAO_LIVRE); // Geralmente usado com padrão LIVRE
EasyInner.ConfigurarTipoLeitor((byte)Enumeradores.TipoLeitor.PROXIMIDADE_ABATRACK2); // Ou CODIGO_BARRAS_SERIAL / TTL_SERIAL_ASCII

EasyInner.InserirQuantidadeDigitovariavel(4);
EasyInner.InserirQuantidadeDigitovariavel(6);
EasyInner.InserirQuantidadeDigitovariavel(8);
EasyInner.InserirQuantidadeDigitovariavel(10);
EasyInner.InserirQuantidadeDigitovariavel(12);
EasyInner.InserirQuantidadeDigitovariavel(14);

// Enviar as configurações (conforme 4.1.7)
// EasyInner.EnviarConfiguracoes(numeroInner);
```

- **Onde Usar:** Esta função deve ser chamada durante a fase de envio de configurações (antes de EasyInner.EnviarConfiguracoes()), geralmente no estado ESTADO_ENVIAR_CFG_ONLINE ou ESTADO_ENVIAR_CFG_OFFLINE.

- **Observação:** No exemplo SDK, a lógica para usar InserirQuantidadeDigitovariavel() pode estar associada à seleção de tipos de leitor como Abatrack II ou QR Code, como mostrado no trecho de código:

```
// Exemplo simplificado em Delphi do código SDK
Procedure TInnerConfiguracao.ConfigurarTpLeitor(InnerAtual: TInner);
Begin
 CASE InnerAtual.TipoLeitor OF
  TpLeitor_PROXIMIDADE_ABATRACK2:
   begin
    EasyInnerDLL.ConfigurarTipoLeitor(TpLeitor_PROXIMIDADE_ABATRACK2);
    // Se Abatrack II, adicionar quantidades variáveis
    EasyInnerDLL.InserirQuantidadeDigitovariavel(4);
    EasyInnerDLL.InserirQuantidadeDigitovariavel(6);
    // ... outras quantidades
   end;
  TpLeitor_CODIGO_BARRAS_SERIAL:
   begin
```

```
    EasyInnerDLL.ConfigurarTipoLeitor(TpLeitor_CODIGO_BARRAS_SERIAL);
    // Se Código de Barras Serial, adicionar quantidades variáveis
    EasyInnerDLL.InserirQuantidadeDigitovariavel(4);
    EasyInnerDLL.InserirQuantidadeDigitovariavel(6);
    // ... outras quantidades
   end;
  // ... outros tipos de leitor
 end;
End;
```

5.1.4 Solução para Limitação de Dígitos (Troca de Placa)

Em cenários onde é essencial utilizar leitores que geram IDs com mais dígitos (ex: mais de 8 dígitos em alguns formatos Wiegand) em catracas da Linha 3 (placa Inner Acesso), e essa funcionalidade não é totalmente suportada pelo firmware ou hardware original, uma solução técnica viável é a **substituição da placa Inner Acesso pela placa Controle Catraca (da Linha 4)**.

- **Compatibilidade Física:** A placa Controle Catraca é compatível mecanicamente e eletricamente com a estrutura das catracas da Linha 3. Isso significa que a troca é fisicamente possível.
- **Funcionalidades Expandidas:** Ao substituir a placa, a catraca da Linha 3 essencialmente ganha as funcionalidades de uma Catraca da Linha 4, tendo acesso a recursos como suporte a Wiegand 42 bits (que permite até 12 dígitos em alguns formatos) e outras melhorias da placa Controle Catraca.
- **Procedimento:** A troca da placa deve ser realizada por pessoal técnico qualificado. O Suporte Técnico Topdata pode fornecer orientações sobre o processo e a aquisição da placa Controle Catraca para reposição.
- **Benefício:** Essa solução permite que clientes que já possuem catracas da Linha 3 atualizem suas funcionalidades de leitura sem a necessidade de adquirir uma catraca completamente nova.

5.2 Catraca Expedidora de Cartões

A **Catraca Expedidora de Cartões** é um equipamento especializado na dispensa controlada de cartões ou comandas. É frequentemente utilizada em sistemas de controle de acesso onde o usuário precisa retirar um bilhete na entrada para posterior validação ou coleta na saída (como em restaurantes por quilo, estacionamentos, etc.).

5.2.1 Fluxo de Operação e Coleta de Bilhetes (EasyInner.ColetarBilhete())

Diferente das catracas convencionais que operam principalmente em modo online para validação em tempo real, a Catraca Expedidora opera de forma autônoma (modo offline para a dispensa). Quando um usuário interage com ela (geralmente girando um braço após alguma validação simples, ou pressionando um botão se configurado), ela dispensa um cartão e registra esse evento internamente. Este comando funciona somente com o Inner no modo Off-Line.

O papel do seu Software de Integração, neste caso, não é controlar a dispensa em tempo real (a catraca decide dispensar ou não com base em sua configuração interna), mas sim **coletar os registros (bilhetes)** gerados pela catraca sobre as dispensas realizadas.

A função principal para interagir com a Catraca Expedidora é EasyInner.ColetarBilhete().

- **Propósito:** Coletar um bilhete (registro de evento) que foi gerado e armazenado na memória da Catraca Expedidora (ou outros equipamentos que operam em modo offline). Ela coleta um bilhete Offline que está armazenado na memória do Inner. Os dados do bilhete são retornados por referência nos parâmetros da função.
- **Comportamento:** Sua aplicação deve chamar EasyInner.ColetarBilhete() periodicamente (dentro da lógica da sua máquina de estados, talvez em um estado dedicado como ESTADO_COLETAR_BILHETES). A cada chamada bem-sucedida que retorna 0 (sucesso), a função preenche os parâmetros de saída com os dados de um bilhete e remove esse bilhete da memória do equipamento. Se não houver mais bilhetes pendentes, ela retornará um código indicando "sem bilhetes" (o valor exato pode ser consultado nos exemplos da SDK ou no documento de erros).
- **Parâmetros (C#):**

```
// Exemplo de parâmetros para a função ColetarBilhete
// Os dados do bilhete são retornados por referência ou via PChar/StringBuilder
EasyInner.ColetarBilhete(
  int NumeroInner,      // [Entrada] Número da catraca (Expedidora)
  ref byte Tipo,        // [Saída] Tipo do bilhete (dispensa, status do mecanismo, etc.)
  ref byte Dia,         // [Saída] Dia do evento
  ref byte Mes,         // [Saída] Mês do evento
  ref byte Ano,         // [Saída] Ano do evento
  ref byte Hora,        // [Saída] Hora do evento
  ref byte Minuto,      // [Saída] Minuto do evento
  StringBuilder Cartao  // [Saída] Número do cartão dispensado (ou informação relevante)
);
```

- **NumeroInner:** O número de identificação da Catraca Expedidora.
- **Tipo:** Indica o tipo específico do bilhete coletado (detalhado na próxima subseção).
- **Dia, Mes, Ano, Hora, Minuto:** Data e hora do evento registrado.
- **Cartao:** Contém o número do cartão que foi dispensado (se aplicável ao tipo de bilhete). Conforme a descrição da DLL (página 102), este é o número do cartão do usuário.

- **Exemplo de Uso (C# - dentro de um estado ESTADO_COLETAR_BILHETES):**

```
// Supondo que 'InnerAtual' representa a Catraca Expedidora
// e que 'sbCartao' é um StringBuilder inicializado

byte tipoBilhete = 0;
byte dia = 0, mes = 0, ano = 0, hora = 0, minuto = 0;
StringBuilder sbCartao = new StringBuilder(20);

// Tenta coletar um bilhete da expedidora
int ret = EasyInner.ColetarBilhete(
  InnerAtual.Numero,
  ref tipoBilhete,
  ref dia, ref mes, ref ano, ref hora, ref minuto,
  sbCartao
);

if (ret == (int)Enumeradores.Retorno.RET_COMANDO_OK)
{
  // Bilhete coletado com sucesso!
  Console.WriteLine($"Bilhete da catraca {InnerAtual.Numero}: Tipo={tipoBilhete},
Cartão={sbCartao.ToString()}, Hora={hora}:{minuto}");

  // Processar o tipo de bilhete (ver 5.2.2) e registrar no seu sistema
  // Continuar no mesmo estado para coletar o próximo bilhete
  InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_COLETAR_BILHETES;
}
else if (ret == (int)Enumeradores.Retorno.RET_SEM_BILHETES) // Exemplo de retorno para "sem bilhetes"
{
  // Não há bilhetes pendentes nesta catraca
  Console.WriteLine($"Catraca {InnerAtual.Numero}: Sem bilhetes para coletar no momento.");
  // Máquina de estados: Transitar para um estado de espera ou Polling (se aplicável)
  // ou voltar para o início do loop de estados
  InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_POLLING; // Exemplo: se a expedidora também fizer
```

```
polling
}
else
{
  // Erro na comunicação, transitar para ESTADO_RECONECTAR
  Console.WriteLine($"Erro ao coletar bilhete da catraca {InnerAtual.Numero}: {ret}");
  InnerAtual.EstadoAtual = Enumeradores.EstadosInner.ESTADO_RECONECTAR;
}
```

- **Fluxo na Máquina de Estados:** Em um loop da máquina de estados, o estado ESTADO_COLETAR_BILHETES deve chamar ColetarBilhete repetidamente para uma catraca Expedidora até receber o retorno indicando que não há mais bilhetes. Em seguida, pode-se passar para um estado de espera antes de tentar coletar novamente, ou para o próximo estado do ciclo geral.

5.2.2 Tipos de Bilhetes da Expedidora

O parâmetro Tipo retornado por EasyInner.ColetarBilhete() informa o tipo de evento registrado. Alguns tipos de bilhetes são específicos do funcionamento da Catraca Expedidora ou indicam o status do mecanismo.

Valor do Tipo	Significado	Notas
0 a 9	Funções registradas pelo cartão	Registros de acesso ou eventos associados a funções configuradas.
10	Entrada pelo cartão	Bilhete gerado após um acesso de entrada bem-sucedido por cartão.
11	Saída pelo cartão	Bilhete gerado após um acesso de saída bem-sucedido por cartão.
12	Tentativa de entrada negada pelo cartão	Registro de uma tentativa de entrada que foi negada.
13	Tentativa de saída negada pelo cartão	Registro de uma tentativa de saída que foi negada.
100 a 109	Funções registradas pelo teclado	Registros de acesso ou eventos associados a funções digitadas no teclado.
110	Entrada pelo teclado	Bilhete gerado após um acesso de entrada bem-sucedido por teclado.
111	Saída pelo teclado	Bilhete gerado após um acesso de saída bem-sucedido por teclado.
112	Tentativa de entrada negada pelo teclado	Registro de uma tentativa de entrada via teclado que foi negada.
113	Tentativa de saída negada pelo teclado	Registro de uma tentativa de saída via teclado que foi

		negada.
003	Mecanismo com cartões (Bilhete de Sinalização)	Indica que há cartões disponíveis no mecanismo de dispensa.
004	Mecanismo com poucos cartões (Bilhete de Sinalização)	Alerta que a quantidade de cartões no mecanismo está baixa.
005	Mecanismo sem cartões (Bilhete de Sinalização)	Indica que o mecanismo de dispensa está vazio.
010	Entrada depois que a catraca foi liberada (cartão)	Bilhete gerado após o giro de entrada, quando o acesso foi liberado.
012	Timeout (usuário pegou cartão, mas não entrou)	O usuário retirou o cartão, mas não realizou o giro dentro do tempo limite.
013	Cartão retirado não validado (catraca não liberada)	Um cartão foi retirado, mas não foi validado e a catraca não liberou o acesso.
128	Bilhete repetido recebido do Inner	Indica que a função já retornou este bilhete em uma coleta anterior.

Fluxograma do Ciclo da Expedidora:

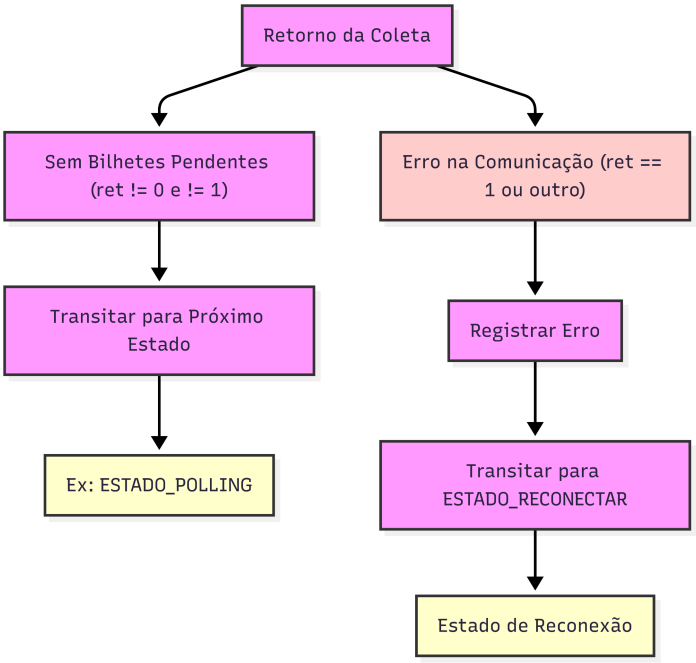
a) Início da Coleta e Análise do Retorno



b) Coleta com Sucesso (Bilhete Coletado)



c) Coleta Finalizada ou Erros



Em um sistema com Catraca Expedidora e Catraca com Urna, o Software de Integração coletará os bilhetes da Expedidora para saber quais cartões foram dispensados (Tipo 010, 012, 013, etc.) e gerenciar o estado desses cartões no seu sistema (ex: comanda aberta). Na saída, a Catraca com Urna operando online enviará o número do cartão depositado (Origem 3/7) para o software validar a comanda e liberar a saída.

5.3 Catraca com Urna Coletora

A **Catraca com Urna Coletora** é utilizada primariamente para controlar o acesso de **saída**, com a funcionalidade adicional de recolher o cartão ou comanda utilizado pelo usuário. É comum em sistemas onde o cartão retirado na entrada precisa ser devolvido na saída, validando o fluxo e impedindo que o mesmo cartão seja reutilizado para uma nova entrada sem o devido registro ou pagamento.

5.3.1 Fluxo de Operação e Validação de Saída

O funcionamento da Catraca com Urna é tipicamente no **modo online**. Isso permite que o seu Software de Integração valide o cartão (ou comanda) na saída (ex: verificar se o pagamento foi efetuado, se o tempo de permanência é válido, etc.) e, somente após a validação, autorize o recolhimento do cartão na urna e a liberação do giro da catraca.

A interação ocorre em tempo real através da função EasyInner.ReceberDadosOnLine() (seção 4.3.1) para receber os eventos, e comandos de liberação/acionamento.

Fluxo Básico de Saída com Urna no Modo Online:

1. **Usuário Apresenta o Cartão/Comanda:** O usuário insere o cartão na fenda do leitor (geralmente o Leitor 2, configurado para saída e urna) ou o aproxima do leitor configurado para saída.
2. **Catraca Envia Evento de Leitura:** A catraca lê o cartão e envia um evento para o Software de Integração via EasyInner.ReceberDadosOnLine().
    - **Origem:** A Origem do evento será **3** (ORIGEM_LEITOR2) se o cartão for lido pelo Leitor 2 (fenda/proximidade) ou **2** (ORIGEM_LEITOR1) se lido pelo Leitor 1 (se configurado para saída também).
    - **Cartao:** O parâmetro Cartao conterá o número do cartão/comanda lido.
3. **Software Valida o Acesso de Saída:** Sua aplicação recebe o evento (Origem 2 ou 3) e o número do cartão. Neste ponto (no estado ESTADO_VALIDAR_ACESSO ou similar), sua lógica de negócio valida se a saída é permitida (ex: consultando o estado da comanda no banco de dados).
4. **Software Aciona o Relé da Urna:** Se o acesso for validado, sua aplicação envia o comando para acionar o relé associado à urna (geralmente o Relé 2).
    - **Comando:** EasyInner.AcionarRele2(NumeroInner, Tempo) ou, se configurado via ConfigurarAcionamento2 com uma função que libera urna e tempo, a urna será acionada automaticamente após a validação no software.
    - **Propósito:** Abrir a fenda da urna para que o cartão seja depositado. O tempo definido no comando Tempo (na configuração do acionamento) determina por quanto tempo a fenda permanecerá aberta.
5. **Usuário Deposita o Cartão na Urna:** O usuário insere o cartão na fenda aberta da urna.
6. **Catraca Envia Evento de Cartão Recolhido:** A catraca detecta que o cartão foi recolhido pela urna e envia um novo evento para o software via EasyInner.ReceberDadosOnLine().
    - **Origem:** A Origem deste evento será **7** (ORIGEM_CARTAO_RECOLHIDO_URNA).
7. **Software Libera o Giro:** Ao receber o evento de Origem 7 (cartão recolhido), sua aplicação envia o comando para liberar o giro da catraca para o sentido de saída.

- **Comando:** EasyInner.LiberarCatracaSaida(NumeroInner) ou EasyInner.LiberarCatracaSaidaInvertida(NumeroInner), dependendo da instalação física da catraca.

8. **Usuário Realiza o Giro:** O usuário passa pela catraca.
9. **Catraca Envia Evento de Giro Confirmado:** A catraca detecta o giro completo e envia um evento.
    - **Origem:** A Origem deste evento será **6** (ORIGEM_GIRO_CATRACA_TOPDATA).
10. **Software Registra Saída Efetiva:** Ao receber o evento de Origem 6, sua aplicação registra a saída efetiva do usuário em seu sistema.

Tratamento de Erros Comuns no Fluxo da Urna:

- **Timeout de Acionamento (Origem 5):** Se após acionar o relé da urna (passo 4), o usuário não depositar o cartão dentro do Tempo configurado, a catraca enviará um evento com Origem = 5. Sua aplicação deve tratar isso (ex: exibir mensagem de erro no display, bloquear nova tentativa imediata).
- **Urna Cheia (Origem 20):** Se o usuário tentar depositar um cartão e a urna estiver cheia, a catraca pode enviar um evento com Origem = 20. Sua aplicação deve notificar o operador.
- **Giro Não Realizado (Origem 5 após Liberação):** Se após enviar o comando de liberação do giro (passo 7), o usuário não girar a catraca dentro do tempo de liberação (configurado no acionamento que liberou o giro), a catraca enviará um evento com Origem = 5.

Configuração da Urna via SDK:

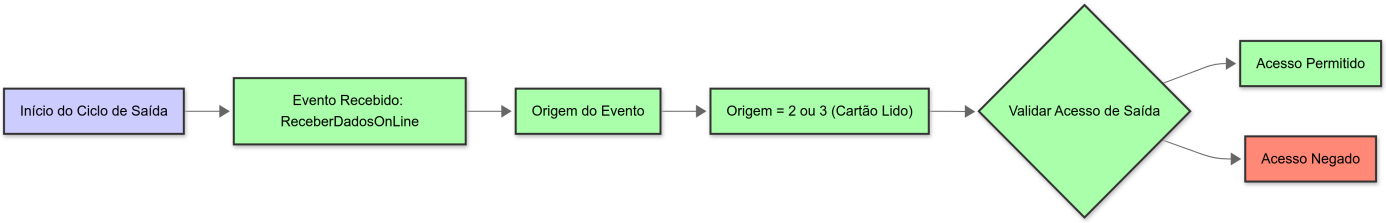
A configuração da funcionalidade de urna na catraca envolve principalmente os comandos de configuração de acionamento (Relé 2 para a urna) e a definição da operação do Leitor 2 (fenda da urna). Isso é feito durante a fase de envio de configurações (Capítulo 4.1).

- EasyInner.ConfigurarAcionamento2(Funcao, Tempo): Defina a função do Relé 2 para um valor que acione a urna (ex: FncAcionamento_ACIONA_REGISTRO_SAIDA ou outro que sua lógica use para acionar o relé da urna) e defina o tempo de abertura da fenda.
- EasyInner.ConfigurarLeitor2(Operacao): Defina a operação do Leitor 2 como somente saída (EntradasOff_LEITOR_SOMENTE_SAIDA) ou outra operação que se adeque ao seu fluxo.

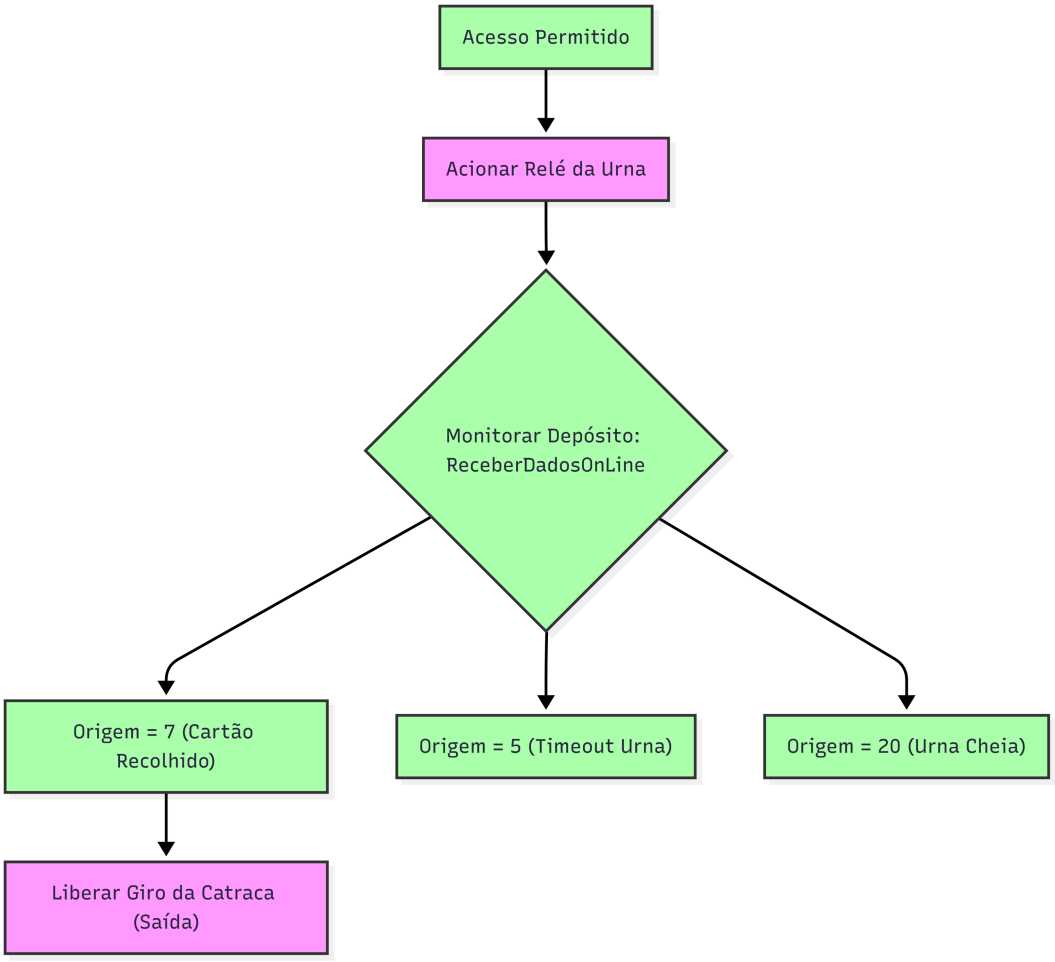
- Configuração da funcionalidade de Urna na catraca via menu master.

Fluxograma do Ciclo da Urna :

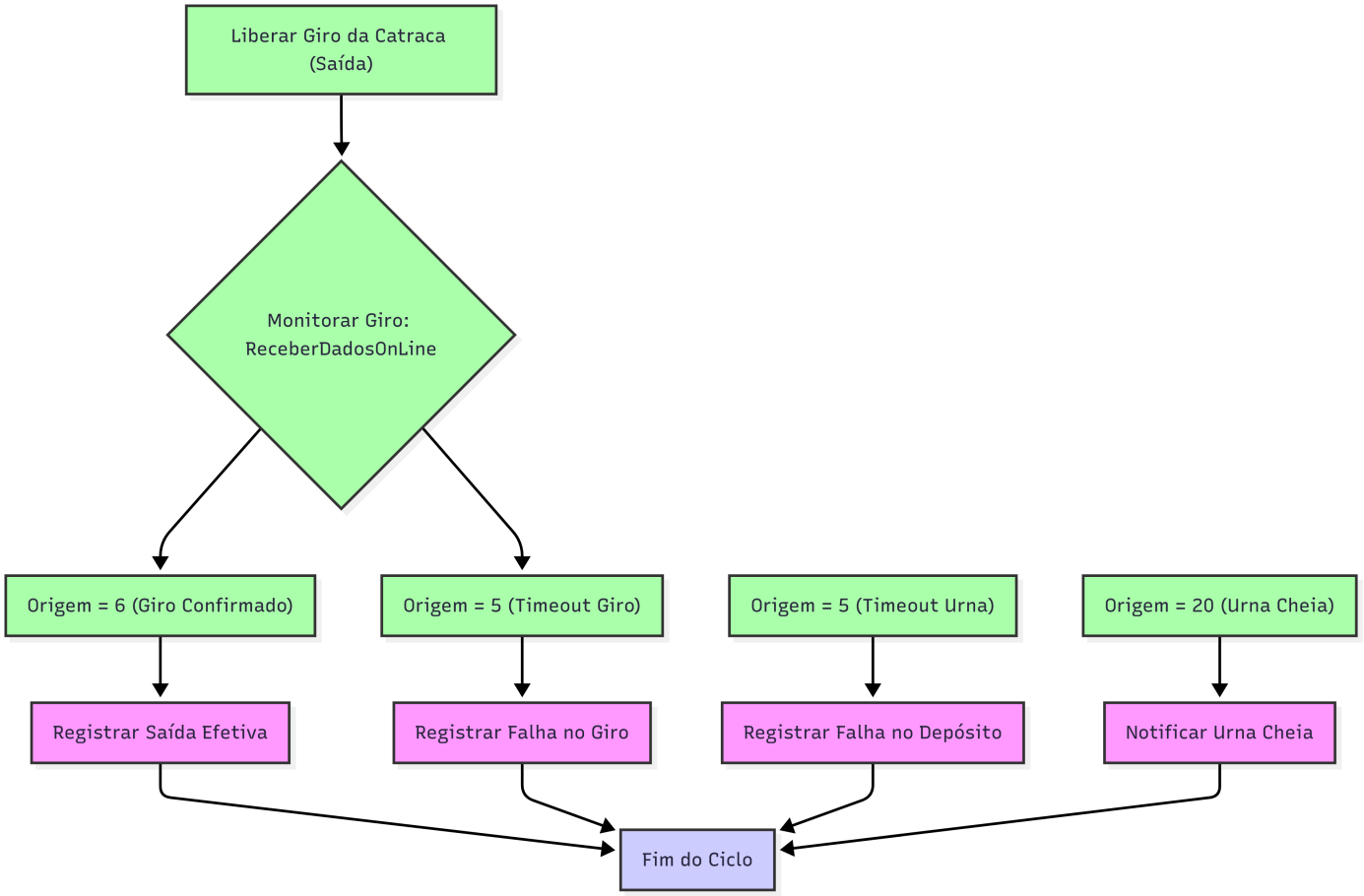
a) Início do Ciclo e Validação de Acesso



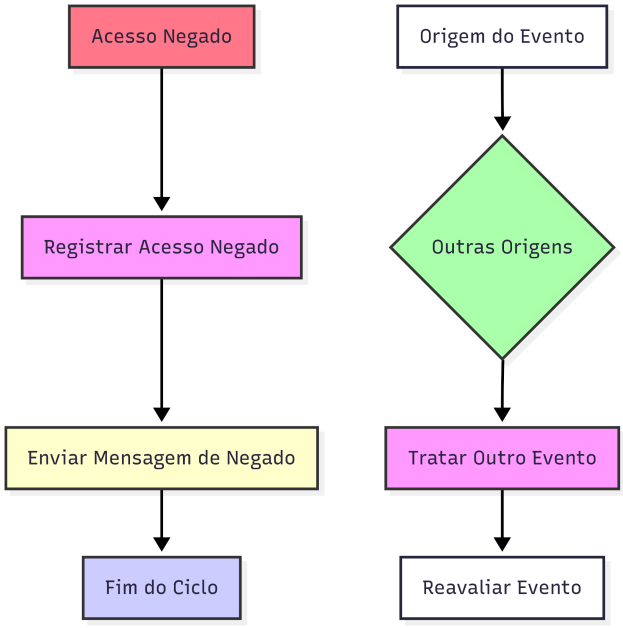
b) Acesso Permitido: Depósito de Cartão



c) Giro da Catraca e Conclusão



d) Acesso Negado ou Eventos Irrelevantes



A integração da Catraca com Urna no modo online oferece um controle preciso sobre o fluxo de saída, garantindo que a coleta de cartões ocorra somente após a validação do acesso pelo seu sistema.

5.4 Integração com Leitor Facial

A integração com os leitores faciais Topdata, como o modelo T4, difere fundamentalmente da comunicação com as catracas através da **SDK Inner Acesso**. Embora leitores faciais possam ser utilizados em conjunto com catracas, eles possuem seu próprio método de comunicação e um SDK dedicado.

Diferente da comunicação via EasyInner.dll (que usa TCP/IP direto com protocolo proprietário), os leitores faciais modernos se comunicam via **WebSocket**. Este é um protocolo diferente que permite uma comunicação bidirecional mais eficiente, baseado na troca de mensagens em formato JSON.

- **Comunicação Independente:** A comunicação com o leitor facial é feita diretamente entre o software de integração e o leitor, não passando pela EasyInner.dll ou pela placa controladora da catraca para a troca de mensagens de controle.
- **Protocolo WebSocket e JSON:** A interação é baseada no envio e recebimento de mensagens JSON, que contêm comandos e dados específicos para o gerenciamento do leitor facial (cadastro de usuários, recebimento de eventos de acesso facial, configuração, etc.).
- **Configuração de Rede do Leitor Facial:** O leitor facial precisa ser configurado com seu próprio endereço IP, máscara, gateway e, crucialmente, o IP e a porta do servidor onde o software de integração para o facial está rodando (geralmente porta 7792). Essa configuração é feita diretamente no menu do leitor facial.

Importante: Como a comunicação do leitor facial não utiliza a EasyInner.dll, as funcionalidades e comandos para integrar com ele estão contidos em uma **SDK separada**.

Para obter todas as informações, documentação, exemplos de código e detalhes sobre a comunicação via WebSocket com o leitor facial, acesse o link abaixo:

- **Link para a SDK do Leitor Facial:**

Integradores Topdata® - FAQ e downloads para integração com produtos Topdata®

Recomendamos que, caso seu projeto envolva leitores faciais, você consulte a documentação e os exemplos disponíveis na SDK específica do leitor facial para compreender o protocolo de comunicação via WebSocket e as funcionalidades dedicadas a este tipo de dispositivo.

6 Boas Práticas e Dicas de Desenvolvimento

Desenvolver uma aplicação de integração com a **SDK Inner Acesso** envolve mais do que apenas chamar funções da EasyInner.dll. A adoção de boas práticas e a atenção a alguns detalhes podem otimizar o processo, aumentar a estabilidade da sua aplicação e facilitar a depuração.

6.1 Tratamento de Erros e Exceções

Ignorar retornos de erro ou não tratar exceções de forma adequada é uma receita para problemas em sistemas de integração com hardware.

- **Sempre Verifique os Retornos:** Conforme discutido na seção 2.3, cada função da EasyInner.dll retorna um código de status. Sua aplicação deve sempre verificar esse retorno e reagir apropriadamente (sucesso, erro, sem eventos, etc.).
- **Use Estruturas de Controle:** Utilize if/else, switch/case ou estruturas de controle equivalentes para lidar com os diferentes códigos de retorno.
- **Tratamento de Exceções:** Implemente blocos try-catch (ou o mecanismo de tratamento de exceções da sua linguagem) em torno das chamadas da DLL para capturar e tratar exceções inesperadas que possam ocorrer (ex: problemas de rede, permissão).
- **Consulte Documentação/Exemplos:** Em caso de erros específicos, consulte os exemplos da SDK para ver como eles tratam retornos similares e utilize o futuro documento de Erros Detalhados para entender o significado dos códigos.

6.2 Uso de Thread(s) para a Comunicação (Serializando o Acesso à DLL)

É fundamental compreender que a EasyInner.dll **não é thread-safe**. Isso significa que você **NUNCA** deve permitir que múltiplas threads da sua aplicação chamem funções da EasyInner.dll simultaneamente. O acesso à DLL deve ser **serializado**, ou seja, apenas uma thread por vez pode interagir com a biblioteca.

- **Uma Única Thread para a DLL:** A arquitetura recomendada é criar uma **única thread dedicada** em sua aplicação especificamente para toda a comunicação com a EasyInner.dll.

- **Máquina de Estados nessa Thread:** A lógica completa da **Máquina de Estados** (Capítulo 2), que gerencia a conexão, configuração, operação (polling, coleta, etc.) e reconexão de **TODAS** as catracas, deve ser executada inteiramente dentro dessa única thread de comunicação.
- **Por que uma Thread Separada?** Apesar de a DLL ser bloqueante, executar a lógica de comunicação (incluindo chamadas como EasyInner.ReceberDadosOnLine()) em uma thread separada garante que a **thread principal da interface gráfica (UI) não seja bloqueada**. Isso mantém sua aplicação responsiva e evita a sensação de "travamento".
- **Comunicação entre Threads:** Se a sua interface gráfica ou outras partes da aplicação precisarem interagir com a thread de comunicação (ex: iniciar/parar o gerenciamento, solicitar o envio de uma lista, exibir um evento recebido), você deverá utilizar mecanismos de comunicação e sincronização entre threads seguros, como:
    - **Filas (Queues):** Enviar comandos da UI para a thread de comunicação colocando-os em uma fila. A thread de comunicação processa a fila no seu loop.
    - **Eventos:** Notificar a UI quando um evento importante ocorrer na thread de comunicação.
    - **Invoke/BeginInvoke (Windows Forms/WPF):** Para atualizar elementos da UI a partir da thread de comunicação de forma segura.

Não Faça:

- Não chame EasyInner.ReceberDadosOnLine() ou qualquer outra função da DLL diretamente de múltiplos botões na UI que rodam em threads diferentes.
- Não crie uma nova thread para cada catraca que chama a DLL independentemente.

Faça:

- Crie uma classe de gerenciamento de catracas.
- Dentro dessa classe, crie uma única thread para a comunicação.
- Implemente a Máquina de Estados dentro dessa thread.
- O loop principal da máquina de estados alterna o processamento entre todas as catracas gerenciadas nessa thread.
- Use mecanismos seguros para comunicação entre a thread de comunicação e a thread da UI.

A correta serialização do acesso à EasyInner.dll em uma única thread dedicada é vital para a estabilidade e o funcionamento correto da sua aplicação de integração.

6.3 Implementação de Logs Detalhados

Um sistema de log robusto é uma das ferramentas mais valiosas para depurar problemas em integrações, especialmente em ambientes de produção onde a depuração interativa pode ser difícil.

- **Registre Tudo Relevante:** Grave no log:
    - Início e fim da aplicação e da comunicação.
    - Transições de estado da máquina de estados.
    - Comandos enviados para a EasyInner.dll, incluindo os parâmetros.
    - Valores de retorno de todas as chamadas da EasyInner.dll.
    - Dados recebidos dos eventos (Origem, Cartao, data/hora).
    - Mensagens de erro, exceções e seus detalhes.
    - Eventos de rede (conexão estabelecida, desconexão, tentativas de reconexão).
- **Níveis de Log:** Considere usar diferentes níveis de log (INFO, DEBUG, WARNING, ERROR) para controlar a verbosidade.

6.4 Estabilidade da Rede e Conectividade

Problemas de rede são uma causa frequente de falhas em integrações de hardware. Embora a EasyInner.dll gerencie a comunicação, a qualidade e a configuração básica da rede onde as catracas e o servidor estão conectados são fundamentais.

- **Rede Estável:** Se possível, utilize uma rede estável e confiável para os equipamentos de controle de acesso.
- **IP Fixo:** Conforme recomendado na seção 3.1.1, utilize IP Fixo para as catracas para garantir que seus endereços não mudem.
- **Verificação de Cabos e Equipamentos:** Certifique-se de que os cabos de rede estão em boas condições e que switches/roteadores básicos estão funcionando corretamente.
- **Firewall Básico:** Confirme que as portas necessárias (padrão 3570) estão liberadas nos firewalls básicos do sistema operacional e da rede local (seção 3.2.1).

- **Monitoramento Básico da Rede:** Ferramentas simples como ping podem ajudar a verificar a conectividade básica entre o servidor e as catracas.
- **Lógica de Reconexão:** Implemente uma lógica robusta de reconexão na sua máquina de estados (ESTADO_RECONECTAR), com retentativas e tempos de espera adequados, pois mesmo em redes estáveis, falhas temporárias podem ocorrer.

6.5 Testes com Kit Integrador e Equipamento Real

Testar a integração é essencial.

- **Kit Integrador:** Se disponível, utilize o Kit Integrador (Capítulo 8.3) para desenvolver e testar sua aplicação sem a necessidade de uma catraca real. Este kit simula o comportamento da catraca.
- **Testes em Bancada:** Antes da instalação final no cliente, teste a comunicação com uma catraca real em um ambiente controlado (bancada).
- **Testes no Ambiente Final:** Realize testes completos no ambiente de rede final do cliente, verificando a conectividade, firewalls e o comportamento da aplicação sob as condições reais de rede.

6.6 Compilação da Aplicação em x86

Conforme destacado nos pré-requisitos (1.3.3), a EasyInner.dll é uma biblioteca de 32 bits.

- **Compile para x86:** Configure seu projeto na IDE para compilar para a plataforma de destino **x86**. Isso garante que sua aplicação rode como um processo de 32 bits e possa carregar a DLL corretamente. Evite "Any CPU" a menos que você tenha certeza de que a VM ou o ambiente de execução forçará a execução em 32 bits ao encontrar a dependência da DLL.

6.7 Gerenciando um Grande Número de Equipamentos

Conforme mencionado na introdução (Seção 1.1), a EasyInner.dll possui uma limitação prática de gerenciar **aproximadamente 30 equipamentos simultaneamente** a partir de uma única instância da DLL (em uma única thread de comunicação). Se o seu projeto envolve um número maior de catracas (ex: 50, 100, 200), a abordagem de ter apenas uma instância da sua aplicação e uma única thread de comunicação não será adequada devido a essa limitação e ao comportamento bloqueante da DLL.

- **Arquitetura com Múltiplas Instâncias:** Para gerenciar um grande número de equipamentos, a arquitetura recomendada é criar **múltiplas instâncias da sua aplicação de integração**.
    - Cada instância da aplicação seria responsável por gerenciar um **subconjunto** de catracas (ex: Instância 1 gerencia catracas de 1 a 30, Instância 2 gerencia de 31 a 60, e assim por diante).
    - Cada instância rodaria em seu próprio processo Windows e teria sua própria thread dedicada para comunicação com a EasyInner.dll, gerenciando apenas as catracas atribuídas a ela.
- **Considerações na Arquitetura Distribuída:**
    - **Balanceamento de Carga:** Distribua o número total de catracas de forma equilibrada entre as instâncias para evitar sobrecarga em uma única instância.
    - **Gerenciamento Central:** Você precisará de um componente central (pode ser um serviço, banco de dados ou outra aplicação) que saiba quais instâncias estão rodando e quais catracas cada instância está gerenciando. Este componente seria responsável por iniciar, monitorar e, se necessário, reiniciar as instâncias.
    - **Configuração das Catracas:** As catracas devem ser configuradas para apontar para o IP e Porta da instância da aplicação responsável por gerenciá-las. Isso pode exigir que cada instância escute em uma porta TCP diferente (ex: Instância 1 na porta 3570, Instância 2 na 3571).
- **Alternativa para o Limite:** Em cenários com grande volume de catracas e necessidade de alta performance, a integração direta via protocolo TCP/IP (sem a DLL, conforme documentação de baixo nível e solicitação de NDA) pode ser uma alternativa. Essa abordagem permite maior controle sobre o gerenciamento de conexões e threads, mas exige um desenvolvimento de comunicação mais complexo por parte do integrador.

Lidar com um grande número de equipamentos requer uma arquitetura distribuída e um planejamento cuidadoso para garantir a estabilidade e a escalabilidade da sua solução de integração.

A aplicação dessas boas práticas contribuirá significativamente para o sucesso da sua integração com a SDK Inner Acesso.

7 Troubleshooting: Diagnóstico e Solução de Problemas Comuns

Mesmo com um desenvolvimento cuidadoso, problemas podem ocorrer durante a integração com dispositivos de hardware. Este capítulo lista sintomas comuns e fornece passos para diagnosticar e solucionar as questões mais frequentes ao utilizar a **SDK Inner Acesso** e a EasyInner.dll.

7.1 Abordagem Geral para Solução de Problemas

Ao encontrar um problema, siga esta abordagem sistemática:

1. **Descreva o Problema:** Detalhe o sintoma exato, quando ocorre (em qual estado da máquina, após qual comando), se é intermitente ou constante.
2. **Verifique os Logs:** Analise os logs da sua aplicação (seção 6.3). Procure por códigos de erro retornados pela EasyInner.dll, mensagens de exceção ou falhas de comunicação.
3. **Confirme a Versão:** Verifique a versão do firmware da catraca (seção 5.1.1) e a versão da SDK/DLL que você está utilizando. Certifique-se de que são compatíveis (consulte os exemplos e suporte).
4. **Revise a Lógica:** Analise o trecho de código onde o problema ocorre e a lógica da máquina de estados (Capítulo 2) naquele ponto. A sequência de comandos está correta para o estado?

7.2 Problemas Comuns e Soluções

Aqui estão alguns sintomas frequentes e seus diagnósticos:

7.2.1 Problemas de Conexão Inicial

- **Sintoma:** EasyInner.AbrirPortaComunicacao() retorna erro (diferente de 0).
    - **Diagnóstico:** A porta TCP (padrão 3570) que sua aplicação tenta abrir já está em uso por outro processo no servidor, ou sua aplicação não tem permissão para abrir a porta.
    - **Solução:** Verifique com ferramentas como netstat -ano (Windows) qual processo está usando a porta. Feche o processo conflitante ou configure sua aplicação

para usar outra porta livre (e atualize a catraca para apontar para a nova porta). Execute sua aplicação com permissões adequadas (como administrador, se necessário).

- **Sintoma:** EasyInner.TestarConexaoInner(NumeroInner) retorna erro/timeout repetidamente; a catraca não aparece como conectada no seu software.
    - **Diagnóstico:** A catraca não está conseguindo se conectar ao seu servidor. As causas são geralmente de rede ou configuração incorreta na catraca.
    - **Solução:**
        - Confirme o **IP do Servidor** configurado na catraca (ela aponta para onde seu software está rodando?). Verifique se o IP está correto e na mesma sub-rede ou acessível (se estiver em rede remota, confirme que o roteamento básico está funcionando).
        - Confirme a **Porta TCP** configurada na catraca. Ela deve ser a mesma que sua aplicação está escutando (EasyInner.AbrirPortaComunicacao()).
        - Confirme o **IP da Catraca**, **Máscara de Sub-rede** e **Gateway** na catraca. Eles devem estar corretos para a rede onde ela está instalada.
        - Verifique o **Número do Inner** configurado na catraca (1-99).
        - Confirme que a **porta no firewall do servidor** (e firewalls básicos de rede local) está liberada para conexões de entrada dos IPs das catracas (seção 3.2.1).
        - Verifique a **conectividade básica** com ping para o IP da catraca a partir do servidor.
        - Confirme se a **catraca está ligada** e com os LEDs de rede indicando atividade.
        - Pode ser um **conflito de IP** na rede (duas máquinas com o mesmo IP).
- **Sintoma:** Erro "Can't load IA 32-bit .dll on a AMD 64-bit platform".
    - **Diagnóstico:** Sua aplicação está tentando carregar a EasyInner.dll (que é 32 bits) em um processo de 64 bits.
    - **Solução:** Configure seu projeto na IDE para compilar e rodar na plataforma de destino **x86** (32 bits) (seção 1.3.3 e 6.6).

7.2.2 Comandos da SDK não Funcionam

- **Sintoma:** Funções como EasyInner.EnviarConfiguracoes() ou EasyInner.EnviarListaAcesso() retornam erro 1 (Erro Genérico) ou outros códigos de erro (diferentes de 8).
    - **Diagnóstico:** O comando não foi aceito pela catraca. Isso pode ser por diversos motivos.
    - **Solução:**
        - A catraca está conectada e no estado correto para receber o comando? (Verifique sua máquina de estados e se TestarConexaoInner retornou sucesso).
        - A sequência de comandos antes de EnviarConfiguracoes está correta? (Embora a ordem das Configurar/Definir não importe, é preciso chamá-las antes de EnviarConfiguracoes).
        - Os parâmetros passados para as funções são válidos? (Ex: número de Inner correto, valores dentro dos limites).
        - Para envio de listas (EnviarListaAcesso), o buffer foi limpo previamente com EasyInner.ApagarListaAcesso()? A lista foi populada com InserirUsuarioListaAcesso?
        - A catraca mudou para o modo offline por timeout de comunicação (falta de PingOnline)? O comando que você está enviando só funciona no modo online?
- **Sintoma:** EasyInner.AbrirPortaComunicacao() ou outras funções retornam **erro 8 (GPF - General Protection Fault)**. A aplicação pode travar ou fechar inesperadamente.
    - **Diagnóstico:** Este é um erro crítico no carregamento ou uso da DLL. As causas mais comuns são:
        - **Incompatibilidade de Arquitetura:** Sua aplicação está compilada/rodando em 64 bits, mas a EasyInner.dll é 32 bits.
        - **.NET Framework:** O .NET Framework 3.5 (ou a versão compatível exigida pela DLL) não está instalado ou não está habilitado corretamente no sistema operacional Windows.

- **DLL Não Encontrada ou Corrompida:** A EasyInner.dll ou outras DLLs de suporte da Topdata não estão no caminho onde o sistema as procura, ou os arquivos estão corrompidos.
- **Incompatibilidade de Versões:** Conflito entre versões da EasyInner.dll e outras DLLs da Topdata.

- **Solução:**
  - **Verifique a Arquitetura:** Configure seu projeto na IDE para compilar e rodar na plataforma de destino **x86** (32 bits) (seção 1.3.3 e 6.6).
  - **Verifique o .NET Framework:** Confirme se o .NET Framework 3.5 (ou a versão necessária) está instalado e habilitado no Painel de Controle do Windows ("Ativar ou desativar recursos do Windows").
  - **Reinstale a SDK:** Baixe a versão mais recente da SDK Inner Acesso do portal do integrador e execute o instalador para garantir que as DLLs estão corretamente instaladas e registradas.
  - **Verifique o Caminho da DLL:** Embora o instalador geralmente cuide disso, confirme se a EasyInner.dll está em um diretório padrão do sistema (System32/SysWOW64) ou no mesmo diretório do executável da sua aplicação.

- **Sintoma:** Ao passar o cartão no leitor, a aplicação trava ou fecha inesperadamente (diferente do erro 8).
  - **Diagnóstico:** Pode ser um problema na lógica da sua aplicação ao processar o evento recebido, ou um problema no leitor/catraca que causa um comportamento inesperado na DLL.
  - **Solução:** Implemente tratamento de exceções (try-catch) em torno do código que processa os eventos recebidos (ReceberDadosOnLine). Analise os logs para ver o que foi recebido antes do travamento. Verifique se a lógica de processamento do evento (validação, liberação) está correta.

7.2.3 Leitores não Respondem ou Travam

- **Sintoma:** Catraca conectada, mas ao passar um cartão/digitar senha, nenhum evento chega no seu software (ReceberDadosOnLine não retorna sucesso/evento).

- **Diagnóstico:** O leitor físico está desabilitado ou configurado incorretamente, ou os eventos não estão sendo enviados para o software no modo online.
- **Solução:**
    - A catraca está em **modo online**? (Verifique a configuração enviada com EasyInner.ConfigurarInnerOnline()).
    - Os **leitores físicos** estão habilitados e configurados corretamente (entrada/saída) via EasyInner.ConfigurarLeitor1() / EasyInner.ConfigurarLeitor2() e EasyInner.EnviarConfiguracoes()? (Seção 4.1.6).
    - O **Tipo de Leitor** configurado via EasyInner.ConfigurarTipoLeitor() corresponde ao leitor físico conectado? (Seção 4.1.3).
    - A configuração da catraca permite que o evento de leitura seja enviado para o software? (Geralmente sim no modo online, mas verifique configurações específicas).
    - O leitor físico está funcionando? (Alguns leitores têm LEDs que acendem ao passar um cartão, teste em modo offline se possível).
- **Sintoma:** Ao passar o cartão no leitor, ele trava ou não aceita mais leituras até reiniciar a catraca.
    - **Diagnóstico:** No modo online, o software precisa "liberar" o leitor para a próxima leitura ou completar o ciclo de acesso. A falta dessa "liberação lógica" pode causar o travamento.
    - **Solução:** Confirme se após receber o evento de leitura (ReceberDadosOnLine), sua lógica de máquina de estados completa o ciclo esperado (validação -> liberação do giro -> registro) e se comandos como EasyInner.LiberarLeitor() ou EasyInner.EnviarConfiguracoesFuncoes() (se houver configuração bit-a-bit de leitores) estão sendo utilizados para indicar que o leitor está livre para a próxima leitura. Em alguns casos, a falta de um comando final após o giro pode ser a causa.

7.2.4 Problemas Específicos da Expedidora

- **Sintoma:** Catraca Expedidora conectada, mas não dispensa cartões.

- **Diagnóstico:** Falta de cartões no mecanismo, mecanismo travado, problema de configuração da dispensa na catraca, ou falha de hardware.
- **Solução:** Verifique fisicamente se há cartões no mecanismo. Remova e insira os cartões novamente. Verifique se há obstruções físicas. Reinicie a catraca. A configuração de dispensa é geralmente no menu da catraca. Se persistir, pode ser hardware.

- **Sintoma:** Software não recebe informações sobre cartões dispensados (EasyInner.ColetarBilhete() retorna "sem bilhetes" mesmo após dispensa).
  - **Diagnóstico:** Sua máquina de estados não está chamando EasyInner.ColetarBilhete() para a Catraca Expedidora ou está chamando com parâmetros incorretos. Os bilhetes podem não estar sendo gerados ou armazenados na catraca.
  - **Solução:** Verifique se a máquina de estados tem um estado dedicado para coletar bilhetes da Expedidora. Garanta que ColetarBilhete() é chamado regularmente para o NumeroInner correto da Expedidora. Confirme se a Catraca Expedidora está gerando bilhetes (verifique no display da catraca ou em um software de gerenciamento offline).

7.2.5 Problemas Específicos da Urna Coletora

- **Sintoma:** Cartão inserido na fenda da urna, mas o software não recebe o evento (Origem 3 ou 7).
  - **Diagnóstico:** O Leitor 2 (associado à fenda da urna) está desabilitado ou configurado incorretamente, problema físico no leitor ou sensor da urna, ou o software não está no estado ESTADO_POLLING recebendo eventos online.
  - **Solução:** Confirme a configuração do Leitor 2 (EasyInner.ConfigurarLeitor2()) para entrada/saída adequada. Verifique a conexão física do leitor da urna. Assegure-se de que a catraca está no modo online e seu software está chamando ReceberDadosOnLine().
- **Sintoma:** Software valida o acesso, envia EasyInner.AcionarRele2(), mas a fenda da urna não abre para receber o cartão.
  - **Diagnóstico:** O Relé 2 não está configurado para a função de urna/acionamento, problema no tempo configurado, ou falha no relé físico.

- **Solução:** Verifique a configuração do EasyInner.ConfigurarAcionamento2(). O tempo deve ser suficiente para o usuário depositar o cartão. Teste a funcionalidade do relé via software de gerenciamento ou Interpretador de Comandos.

- **Sintoma:** Cartão é depositado (Origem 7 recebido), software envia EasyInner.LiberarCatracaSaida(), mas o giro não é liberado.
  - **Diagnóstico:** Configuração incorreta dos acionamentos que controlam o mecanismo de giro (Acionamento 1 e/ou 2) ou falha mecânica/elétrica da catraca.
  - **Solução:** Revise as configurações de EasyInner.ConfigurarAcionamento1() e EasyInner.ConfigurarAcionamento2() para garantir que a liberação de saída está configurada corretamente após o depósito. Verifique a parte mecânica da catraca.

Tabela: Resumo de Problemas Comuns, Causas e Soluções Sugeridas

Sintoma Comum na Aplicação	Causa(s) Provável(eis)	Solução(ões) Sugerida(s)	Referência(s)
EasyInner.AbrirPortaComunicacao() retorna erro (não zero)	Porta TCP em uso por outro processo; Falta de permissão.	Verifique e libere a porta com netstat ou reinicie o PC/serviço. Execute com permissões de administrador. Use outra porta e atualize a catraca.	Cap. 3.3.2, 7.2.1
TestarConexaoInner() retorna erro/timeout; catraca não conecta	Configuração de rede incorreta na catraca (IP Servidor, Porta, IP, Máscara, Gateway); Firewall bloqueando; Problema de cabeamento/físico; Catraca desligada.	Verifique todas as configurações de rede na catraca. Confirme regras de firewall. Teste ping da catraca para o servidor e vice-versa. Verifique cabos/LEDs. Catraca ligada?	Cap. 3.1, 3.2, 7.2.1
Erro "Can't load IA 32-bit .dll on a AMD 64-bit platform"	Aplicação compilada/rodando em 64 bits, tentando carregar EasyInner.dll (32 bits).	Configure o projeto na IDE para compilar e rodar na plataforma de destino **x86** (32 bits).	Cap. 1.3.3, 6.6, 7.2.2
EasyInner.EnviarConfiguracoes() retorna erro 1.	Sequência de comandos incorreta antes do envio; Timeout na comunicação; Parâmetros inválidos.	Revise a lógica da sua máquina de estados. Verifique a estabilidade da rede. Confirme parâmetros.	Cap. 4.1.7, 7.2.2

EasyInner.EnviarListaAcesso() retorna erro 1 ou outro.	Buffer não limpo (ApagarListaAcesso); Lista não populada; Catraca não no modo Offline/estado correto; Erro de comunicação.	Chame ApagarListaAcesso() antes de montar a lista. Verifique se InserirUsuarioListaAcesso foi usado. Catraca no modo offline/estado de lista? Estabilidade da rede.	Cap. 4.2, 7.2.2
Aplicação trava/fecha após chamar função da DLL (Erro 8/GPF)	Incompatibilidade x86/x64; DLL corrompida/não encontrada; Falta .NET Framework.	Confirme compilação x86. Reinstale a SDK. Ative .NET Framework 3.5.	Cap. 1.3.3, 6.6, 7.2.2
Catraca conectada, mas eventos de leitura não chegam (ReceberDadosOnLine não retorna evento)	Catraca não no modo online; Leitor físico desabilitado/mal configurado/desconectado; Eventos não enviados para o software.	Confirme catraca no modo online (ConfigurarInnerOnline). Verifique configuração e status dos leitores (ConfigurarLeitor1/2, EasyInner.EnviarConfiguracoes). Teste leitor físico.	Cap. 2.2.1, 4.1.6, 7.2.3
Ao passar cartão no leitor, catraca/leitor trava.	Software não "libera" o leitor para próxima leitura; Ciclo de acesso não finalizado pela lógica da máquina.	Garanta que a lógica da máquina de estados complete o ciclo após a leitura (validação -> liberação -> registro). Utilize comandos como EasyInner.LiberarLeitor() se aplicável.	Cap. 7.2.3
Catraca Expedidora não dispensa cartão.	Sem cartões; Mecanismo travado; Configuração de dispensa incorreta; Falha hardware.	Verifique cartões e obstruções físicas. Teste configuração de dispensa (menu catraca). Reinicie. Contate Suporte Técnico.	Cap. 5.2, 7.2.4
ColetarBilhete() retorna "sem bilhetes" mesmo após dispensa.	Máquina de estados não chamando ColetarBilhete() para Expedidora; Catraca não gerando bilhetes; Configuração incorreta da Expedidora.	Verifique lógica da máquina de estados para coleta na Expedidora. Confirme se catraca gera bilhetes (display/software offline). Reveja configuração enviada.	Cap. 5.2.1, 7.2.4
Cartão inserido na urna, mas evento não chega (ReceberDadosOnLine sem Origem 3/7).	Leitor 2 (urna) desabilitado/mal configurado/desconectado; Software não em ESTADO_POLLING.	Confirme configuração do Leitor 2. Verifique conexão física do leitor. Assegure software em ESTADO_POLLING e ReceberDadosOnLine.	Cap. 5.3.1, 7.2.5

Software envia AcionarRele2(), mas urna não abre.	Relé 2 não configurado para função de urna; Tempo configurado muito curto; Falha relé físico.	Verifique configuração de ConfigurarAcionamento2(). Aumente o tempo. Teste o relé.	Cap. 4.1.5, 5.3.1, 7.2.5
Cartão depositado (Origem 7 recebido), software envia LiberarCatracaSaida(), mas giro não libera.	Configuração dos acionamentos para giro incorreta; Falha mecânica/elétrica catraca.	Revise ConfigurarAcionamento1()/2() para liberação de saída após depósito. Verifique mecanismo da catraca.	Cap. 4.1.5, 5.3.1, 7.2.5

Esta tabela resume os problemas mais comuns abordados na seção 7.2. Ela pode ser uma referência rápida para diagnóstico inicial.

7.3 Ferramentas e Técnicas de Debugging

- **Exemplos SDK:** Compile e execute os exemplos fornecidos pela Topdata (padrão e os específicos para Expedidora/Urna). Compare o comportamento com o da sua aplicação. Eles são uma referência validada do funcionamento esperado.
- **Logs Detalhados:** Implemente um sistema de logs robusto (seção 6.3). Eles fornecem um rastro completo das operações e retornos da DLL, essenciais para identificar onde a falha ocorre.
- **Wireshark:** Para análise avançada do tráfego de rede TCP/IP entre o servidor e as catracas. Permite ver os pacotes sendo trocados e diagnosticar problemas de conexão, timeouts ou falhas no protocolo (requer conhecimento em análise de rede).

7.4 Quando Contatar o Suporte Topdata

O suporte da Topdata está disponível para auxiliar integradores. É importante direcionar sua dúvida para o canal correto e fornecer as informações necessárias.

- **Suporte a Integradores (desenvolvimento@topdata.com.br):** Contate-nos (a equipe de Desenvolvimento) se tiver dúvidas específicas sobre:
  - Uso correto das funções da SDK Inner Acesso (EasyInner.dll).
  - Interpretação de códigos de retorno ou do comportamento da DLL.
  - Adaptação ou dúvidas sobre os exemplos SDK.
  - Problemas com a lógica da máquina de estados em relação à comunicação.

- **Suporte Técnico (suporte@topdata.com.br):** Contate-os para:
    - Problemas suspeitos de hardware (catraca não liga, leitores/sensores não acendem, mecanismo travado).
    - Dúvidas sobre configuração via menu da catraca ou web server.
    - Questões sobre firmware (versão, compatibilidade, atualização).
    - Problemas com o Kit Integrador (hardware do kit).
    - Dúvidas sobre o uso dos softwares Topdata (Gerenciador de Inners, TopAcesso, etc.).
    - Problemas de rede no local do cliente que fogem do diagnóstico básico (Firewall, NAT complexo, etc.).

Ao contatar o suporte (especialmente por e-mail), forneça o máximo de detalhes possível: modelo da catraca, versão do firmware (se souber), versão da SDK/DLL, descrição clara e reproduzível do problema, logs da sua aplicação (com retornos da DLL!), e os passos que levam ao erro.

Este capítulo visa ser um guia prático para ajudar o integrador a resolver problemas de forma autônoma ou saber quando e como escalar para o suporte especializado.

8 Suporte e Recursos Adicionais

Ao longo do seu processo de integração com a **SDK Inner Acesso**, podem surgir dúvidas, necessidade de suporte técnico ou a busca por recursos adicionais para aprimorar sua solução. Este capítulo detalha os canais de suporte da Topdata e onde encontrar materiais relevantes.

8.1 Canais de Suporte Topdata

A Topdata oferece diferentes canais de suporte, especializados em áreas distintas para melhor atendê-lo. É importante direcionar sua solicitação para o canal correto:

- **Suporte a Integradores (Desenvolvimento):** Este canal é o ponto de contato principal para **dúvidas relacionadas ao desenvolvimento de software** utilizando a SDK Inner Acesso. Contate-nos (a equipe de Desenvolvimento) para questões sobre:
    - Uso e interpretação das funções da EasyInner.dll.
    - Adaptação e dúvidas sobre os exemplos de código da SDK.
    - Implementação da máquina de estados.
    - Códigos de retorno da DLL e troubleshooting a nível de software/protocolo.
    - Melhores práticas de integração de software.
    - Problemas com a lógica da sua aplicação em conjunto com a SDK.
    - Problemas com o Kit Integrador (simulador de catraca).
- **Suporte Técnico:** Este canal é o ponto de contato para **dúvidas e problemas relacionados ao hardware e firmware** dos equipamentos. Contate o Suporte Técnico para questões sobre:
    - Instalação física e configuração inicial da catraca (menu no display, WebServer).
    - Problemas de funcionamento do hardware (catraca não liga, leitor não lê, mecanismo travado, problemas elétricos/cabeamento).
    - Atualização de firmware da catraca.
    - Configuração de rede diretamente na catraca (IP, Máscara, Gateway, IP Servidor, Porta).

- Uso e configuração dos softwares Topdata (Gerenciador de Inners, TopAcesso, TopPortaria) como produtos finais (não para integração).

8.2 Como Solicitar Suporte (Cadastro de Integrador, Informações Necessárias)

Para receber suporte dedicado e gratuito da equipe de Desenvolvimento ou Suporte Técnico, sua empresa precisa estar cadastrada como **Integradora** no sistema da Topdata.

1. **Cadastro de Integrador:** Se ainda não for cadastrado, preencha o formulário de cadastro de integradores fornecido pela Topdata. Este formulário geralmente solicita informações sobre sua empresa, experiência com integrações e os produtos Topdata de interesse.
    - [Sugestão: Mencionar onde encontrar o formulário - geralmente no portal do integrador ou fornecido via email.]
2. **Formas de Contato:** O suporte a integradores é realizado **prioritariamente via e-mail**. Isso permite um registro completo das dúvidas, respostas e histórico do atendimento. Contatos telefônicos geralmente são agendados previamente via e-mail para questões que exigem discussão em tempo real ou acesso remoto.
3. **Informações Essenciais ao Solicitar Suporte:** Ao enviar um e-mail de suporte, forneça o máximo de detalhes possível para agilizar o diagnóstico:
    - **Identificação:** Nome da sua empresa e CNPJ.
    - **Equipamento:** Modelo exato da catraca/equipamento (ex: Fit 4 Urna Prox), e se possível, a versão do firmware (vista no display ou via ReceberVersaoFirmware).
    - **SDK:** Versão da SDK Inner Acesso e da EasyInner.dll utilizada.
    - **Problema:** Descrição clara e detalhada do sintoma.
    - **Passos para Reproduzir:** Como reproduzir o problema.
    - **Contexto:** Em qual estado da máquina de estados o problema ocorre, qual comando da DLL foi chamado.
    - **Logs:** Inclua logs detalhados da sua aplicação (com retornos da DLL!), mensagens de erro e exceções. Se for problema de comunicação, logs de rede (Wireshark) podem ser úteis.
    - **Código:** Anexe um trecho de código relevante (se a dúvida for sobre o uso de uma função).

8.3 Kit Integrador: Como Obter

Para facilitar o desenvolvimento e teste da sua aplicação sem a necessidade inicial de uma catraca física, a Topdata oferece um **Kit Integrador**. Este kit simula o comportamento de uma catraca, permitindo que você teste a comunicação e a lógica da sua aplicação em bancada.

- **Como Solicitar:** O Kit Integrador deve ser solicitado através do canal de **Suporte Técnico** (suporte@topdata.com.br). Eles poderão informar sobre a disponibilidade, os requisitos e o processo para a obtenção do kit.
- **Utilização:** O Kit Integrador é uma ferramenta de desenvolvimento. Ele permite testar a maioria das funcionalidades de comunicação da EasyInner.dll, mas o comportamento em um ambiente real com múltiplas catracas e infraestrutura de rede pode variar.

8.4 Links Úteis

Acesse o portal do integrador da Topdata para encontrar os recursos mencionados neste manual e outros materiais relevantes:

- **Portal do Integrador Topdata:** Integradores Topdata® - FAQ e downloads para integração com produtos Topdata®
- **SDK Inner Acesso (Catracas e Coletores):** Download do instalador, exemplos e documentação.
  Integração com Catraca e Coletor de Acesso - Integradores Topdata®
- **SDK Leitor Facial:** Download do instalador, exemplos e documentação específica para leitores faciais (comunicação WebSocket).

Integração com Leitor Facial - Integradores Topdata®

- **Gerenciador de Inners:** Software para gerenciamento offline e coleta de bilhetes (pode ser útil para verificar o status da catraca ou coletar logs diretamente do equipamento).
  Gerenciador de Inners • Suporte Topdata ®
- **Outros Softwares (TopAcesso, TopPortaria):** Para gerenciamento de acesso como produtos finais.

Suporte Topdata ® • Manuais, Downloads e FAQ sobre produtos Topdata®

Utilizar os recursos disponíveis e contatar o suporte quando necessário são passos essenciais para uma integração bem-sucedida.

9 Histórico de Revisões

Revisão	Data	Descrição
00	23/07/2025	Primeira versão